



PyQGIS 3.44 developer cookbook

QGIS Project

2026 년 08 월 09 일

1	소개	3
1.1	파이썬 콘솔에서의 스크립트 작업	4
1.2	파이썬 플러그인	4
1.3	QGIS 구동 시 파이썬 코드 실행	5
1.4	파이썬 응용 프로그램	6
1.5	PyQt 및 SIP 에 대한 기술 노트	9
2	프로젝트 불러오기	11
2.1	경로 오류 해결하기	12
2.2	속도 향상을 위해 플러그 사용하기	13
3	레이어 불러오기	15
3.1	벡터 레이어	15
3.2	래스터 레이어	18
3.3	QgsProject 인스턴스	21
4	TOC(Table Of Contents) 에 접근하기	23
4.1	QgsProject 클래스	23
4.2	QgsLayerTreeGroup class	24
5	래스터 레이어 사용하기	27
5.1	레이어 상세 정보	27
5.2	렌더링 작업자	28
5.3	값 쿼리하기	30
5.4	래스터 데이터 편집하기	30
6	벡터 레이어 사용하기	33
6.1	속성에 대한 정보 검색하기	34
6.2	벡터 레이어 작업 반복하기	35
6.3	피쳐 선택	36
6.4	벡터 레이어 수정하기	38
6.5	공간 인덱스 사용하기	42
6.6	QgsVectorLayerUtils 클래스	43
6.7	벡터 레이어 생성하기	43
6.8	벡터 레이어의 표현 (심볼)	48
6.9	남은 이야기들	57

7	도형 다루기	59
7.1	도형 작성하기	60
7.2	도형에 접근하기	60
7.3	도형 숨어 및 연산	62
8	투영체 지원	65
8.1	좌표계	65
8.2	좌표계 변환	67
9	맵 캔버스 사용하기	69
9.1	맵 캔버스 내장시키기	70
9.2	고무줄과 꼭짓점 마커	71
9.3	캔버스에서 맵 도구 사용하기	72
9.4	사용자 정의 맵 도구 작성하기	74
9.5	사용자 정의 맵 캔버스 항목 작성하기	75
10	맵 렌더링 및 출력하기	77
10.1	단순 렌더링	78
10.2	서로 다른 좌표계를 가진 레이어들을 렌더링하기	79
10.3	인쇄 조판을 사용해서 산출하기	79
11	값을 필터링하고 계산하는 표현식	83
11.1	표현식 파싱하기	84
11.2	표현식 평가하기	85
11.3	표현식 오류 처리하기	87
12	읽기 및 저장하기 설정	89
13	사용자에게 정보 전달하기	93
13.1	메시지 보이기: QgsMessageBar 클래스	94
13.2	진행률 보이기	97
13.3	로그 작성하기	97
14	인증 인프라스트럭처	101
14.1	소개	102
14.2	용어 해설	102
14.3	QgsAuthManager: 입구점	103
14.4	플러그인이 인증 인프라스트럭처를 사용하도록 조정하기	106
14.5	인증 GUI	107
15	태스크 - 배경에서 무거운 작업 하기	111
15.1	소개	111
15.2	예제	113
16	파이썬 플러그인 개발하기	119
16.1	파이썬 플러그인 구성하기	119
16.2	코드 조각	129
16.3	플러그인 작성 및 디버그 작업을 위한 IDE 설정	133
16.4	사용자 플러그인 배포하기	141
17	공간 처리 플러그인 작성하기	145
17.1	플러그인을 업데이트하기	145
18	플러그인 레이어 사용하기	149
18.1	QgsPluginLayer 하위 클래스 만들기	149

19	네트워크 분석 라이브러리	153
19.1	그래프 작성하기	153
19.2	그래프 분석	155
20	QGIS 서버와 파이썬	163
20.1	소개	163
20.2	서버 API 기본 사항	164
20.3	독립형 또는 내장형	164
20.4	서버 플러그인	165
21	PyQGIS 의 꼼수와 편법	177
21.1	사용자 인터페이스	177
21.2	설정	178
21.3	툴바	178
21.4	메뉴 그룹	178
21.5	캔버스	179
21.6	레이어	179
21.7	TOC(Table of Contents)	183
21.8	고급 TOC	183
21.9	공간 처리 알고리즘	187
21.10	장식자 (decorator)	188
21.11	작성자 (composer)	189
21.12	소스	189

이 문서는 교재와 참조 지침서 둘 다를 목표로 작성되었습니다. 이 문서가 가능한 모든 사용례를 모두 보여주지는 못하지만, 중요한 기능들을 살펴보기엔 충분할 것입니다.

자유 소프트웨어 재단이 발행한 버전 1.3 이상의 GNU 자유 문서 사용 허가서의 조건에 따라 이 문서를 장절항목 변경 없이, 앞표지 및 뒤표지 텍스트 추가 없이 복사, 배포, 그리고/또는 수정할 수 있습니다.

이 사용 허가서의 복사본은 부록 `gnu_fdl`에 포함되어 있습니다.

이 사용 허가서는 이 문서에 있는 모든 코드 조각들에도 적용됩니다.

파이썬을 처음 지원하기 시작한 것은 QGIS 0.9 버전부터입니다. QGIS 데스크탑에서 파이썬을 사용할 수 있는 몇 가지 방법 (을 다음 절들에서 설명합니다)이 있습니다:

- QGIS 안에 있는 파이썬 콘솔에서 명령어 실행
- 플러그인 생성 및 사용
- QGIS 구동 시 파이썬 코드 자동 실행
- 공간 처리 알고리즘 생성
- QGIS 에서 표현식 용 함수 생성
- QGIS API 를 바탕으로 사용자 정의 응용 프로그램 생성

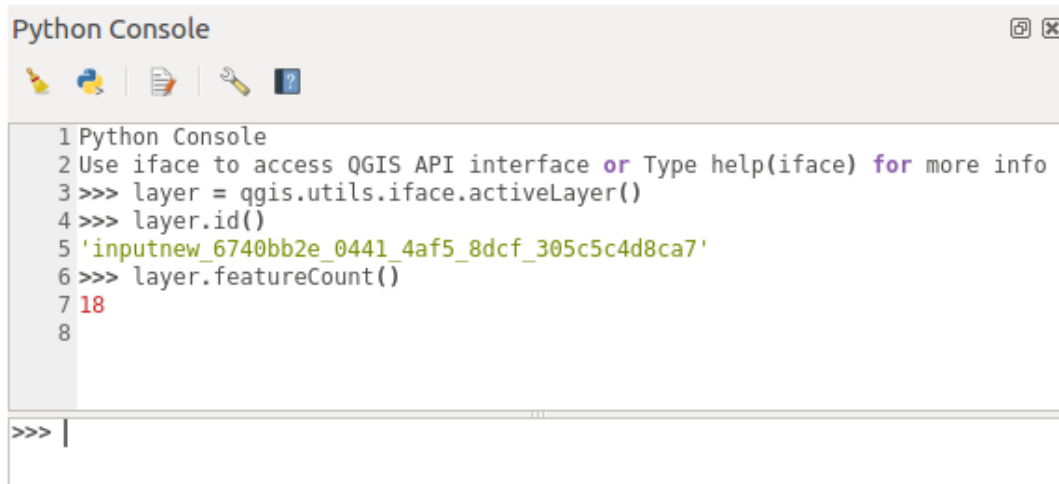
QGIS 서버에 대해서도 파이썬 바인딩을 사용할 수 있습니다. 이 사용법은 파이썬 플러그인 (*QGIS 서버와 파이썬* 참조) 과 파이썬 응용 프로그램에 QGIS 서버를 내장시키기 위해 사용할 수 있는 파이썬 바인딩을 포함합니다.

QGIS 라이브러리 클래스를 문서화한 완전한 QGIS C++ API 참조가 있습니다. 파이썬의 QGIS API (`pyqgis`) 는 C++ API 와 거의 동일합니다.

흔한 작업들을 수행하는 방법을 배우는 데 좋은 또다른 리소스는 플러그인 저장소 에서 기존 플러그인을 다운로드해서 그 코드를 공부하는 것입니다.

1.1 파이썬 콘솔에서의 스크립트 작업

QGIS 는 스크립트 작업을 위한 통합 파이썬 콘솔 을 제공합니다. *Plugins* ▢ *Python Console* 메뉴 항목을 클릭해서 열 수 있습니다.



```

1 Python Console
2 Use iface to access QGIS API interface or Type help(iface) for more info
3 >>> layer = qgis.utils.iface.activeLayer()
4 >>> layer.id()
5 'inputnew_6740bb2e_0441_4af5_8dcf_305c5c4d8ca7'
6 >>> layer.featureCount()
7 18
8
>>> |

```

그림 1.1: QGIS 파이썬 콘솔

앞의 스크린샷은 현재 레이어 목록에서 선택한 레이어를 가져오는 방법과, 레이어 ID 를 보는 방법, 그리고 벡터 레이어인 경우 피쳐 개수를 보는 선택적인 방법을 보여주고 있습니다. QGIS 환경과 쌍방향으로 작업하기 위한, *QgisInterface* 클래스의 인스턴스인 *iface* 변수가 있습니다. 이 인터페이스를 통해 QGIS 응용 프로그램의 맵 캔버스, 메뉴, 톨바, 그리고 기타 부분들에 접근할 수 있습니다.

사용자 편의성을 위해, 콘솔이 시작될 때 다음 선언문이 실행됩니다. (향후 더 많은 초기 명령어를 설정할 수 있게 될 것입니다.)

```

from qgis.core import *
import qgis.utils

```

이 콘솔을 자주 사용하는 사용자라면 (*Settings* ▢ *Keyboard shortcuts*...메뉴에서) 콘솔을 여는 단축키를 설정해두는 편이 유용할 수도 있습니다.

1.2 파이썬 플러그인

플러그인을 사용하면 QGIS 의 기능을 확장할 수 있습니다. 플러그인은 파이썬으로 작성할 수 있습니다. 파이썬 플러그인이 C++ 플러그인을 뛰어넘는 주요한 장점은 배포가 단순하다는 점과 (각 플랫폼 별로 컴파일해야 할 필요가 없습니다) 개발이 더 쉽다는 점입니다.

파이썬을 지원하기 시작한 후로 여러 기능을 커버하는 수많은 플러그인이 작성돼 왔습니다. 플러그인 설치자는 사용자가 파이썬 플러그인을 쉽게 가져오고, 업그레이드하고, 제거할 수 있게 해줍니다. 플러그인과 플러그인 개발에 대해 더 많이 알고 싶다면 [파이썬 플러그인](#) 페이지를 참조하세요.

파이썬으로 플러그인을 생성하는 일은 매우 간단합니다. 자세한 내용은 [파이썬 플러그인 개발하기](#) 강의를 참조하십시오.

참고

QGIS 서버 용 파이썬 플러그인도 사용할 수 있습니다. 더 자세한 내용은 [QGIS 서버와 파이썬](#) 을 참조하세요.

1.2.1 공간 처리 플러그인

Processing Plugins can be used to process data. They are easier to develop, more specific and more lightweight than Python Plugins. [공간 처리 플러그인 작성하기](#) explains when the use of Processing algorithms is appropriate and how to develop them.

1.3 QGIS 구동 시 파이썬 코드 실행

QGIS 를 구동할 때마다 파이썬 코드를 실행시킬 수 있는 서로 다른 방법들이 있습니다.

1. startup.py 스크립트 생성하기
2. 기존 파이썬 파일에 PYQGIS_STARTUP 환경 변수 설정하기
3. --code init_qgis.py 파라미터를 사용해서 구동 스크립트 지정하기

1.3.1 startup.py 파일

QGIS 를 구동할 때마다, 사용자의 파이썬 홈 디렉터리와 시스템 경로 목록에서 startup.py 라는 파일을 검색합니다. 해당 파일이 존재하는 경우, 내장 파이썬 해석기가 이 파일을 실행합니다.

사용자 홈 디렉터리 경로는 보통 다음 아래에서 찾을 수 있습니다:

- Linux: .local/share/QGIS/QGIS3
- Windows: AppData\Roaming\QGIS\QGIS3
- macOS: Library/Application Support/QGIS/QGIS3

기본 시스템 경로는 운영체제에 따라 달라집니다. 사용자에게 맞게 작동하는 경로를 찾으려면 파이썬 콘솔을 열고 `QStandardPaths.standardLocations(QStandardPaths.AppDataLocation)` 를 실행하십시오. 기본 디렉터리 목록이 출력될 것입니다.

QGIS 응용 프로그램의 구동 초기에, QGIS 에서 파이썬이 초기 설정되는 즉시 startup.py 스크립트를 실행합니다.

1.3.2 PYQGIS_STARTUP 환경 변수

PYQGIS_STARTUP 환경 변수를 기존 파이썬 파일의 경로로 설정하면 QGIS 초기 설정이 완료되기 직전에 파이썬 코드를 실행시킬 수 있습니다.

This code will run before QGIS initialization is complete. This method is very useful for cleaning sys.path, which may have undesirable paths, or for isolating/loading the initial environment without requiring a virtual environment, e.g. homebrew or MacPorts installs on Mac.

1.3.3 --code 파라미터

사용자가 QGIS 에 구동 파라미터로 실행되는 사용자 정의 코드를 제공할 수 있습니다. 파이썬 파일을, 예를 들어 `qgis_init.py` 파일을 생성한 다음, 명령 줄에서 `qgis --code qgis_init.py` 명령어를 실행해서 QGIS 를 구동하면 됩니다.

--code 파라미터를 통해 제공된 코드는 QGIS 초기 설정 단계 후반에, 응용 프로그램 구성 요소들을 불러온 후에 실행됩니다.

1.3.4 파이썬 용 추가 인자들

사용자의 --code 스크립트 또는 실행되는 다른 파이썬 코드에 추가 인자를 더하고 싶은 경우, --py-args 인자를 사용하면 됩니다. --py-args 뒤에 그리고 (존재하는 경우) -- 인자 앞에 오는 모든 인자는 파이썬으로 전달될 것이지만 QGIS 응용 프로그램 자체는 해당 인자를 무시할 것입니다.

다음 예시에서, `myfile.tif` 는 파이썬에서 `sys.argv` 를 통해 사용할 수 있지만 QGIS 가 `myfile.tif` 를 불러오지는 않을 것입니다. 반면 QGIS 는 `otherfile.tif` 를 불러올 것이지만 `sys.argv` 에는 없습니다.

```
qgis --code qgis_init.py --py-args myfile.tif -- otherfile.tif
```

파이썬 안에서 나온 모든 명령 줄 파라미터에 접근하고 싶은 경우, `QCoreApplication.arguments()` 를 사용하면 됩니다.

```
QgsApplication.instance().arguments()
```

1.4 파이썬 응용 프로그램

공간 처리 자동화를 위한 스크립트를 작성하는 것이 편리한 경우가 많습니다. PyQGIS 를 사용하면 이를 완벽하게 작업할 수 있습니다—`qgis.core` 모듈을 가져와서 초기 설정을 하거나 하면 공간 처리 작업을 할 준비가 끝납니다.

또는 사용자가 GIS 기능을 사용하는 대화형 응용 프로그램을 만들려 할 수도 있습니다—측정 작업을 수행하거나, 맵을 PDF 로 내보내거나, 등등과 같은 기능 말이죠. `qgis.gui` 모듈은 여러 GUI 구성 요소를 제공합니다. 가장 주목할 만한 요소는 맵 캔버스 위젯으로, 응용 프로그램에 통합되어 확대/축소, 이동, 그리고/또는 다른 사용자 정의 맵 도구들을 지원합니다.

PyQGIS 사용자 정의 응용 프로그램 또는 독립 실행형 스크립트는 투영체 정보와 벡터 및 래스터 레이어를 읽어오기 위한 제공자 같은 QGIS 리소스의 정확한 위치를 찾을 수 있도록 반드시 환경설정해줘야만 합니다. 사용자 응용 프로그램 또는 스크립트의 시작 부분에 몇 줄을 추가해주면 QGIS 리소스를 초기 설정할 수 있습니다. 사용자 정의 응용 프로그램 또는 독립 실행형 스크립트를 위해 QGIS 를 초기 설정해주는 코드는 비슷합니다. 다음은 그 각각의 예시입니다.

i 참고

사용자 스크립트에 `qgis.py` 라는 파일 이름을 사용하지 마십시오. 스크립트의 파일 이름이 바인딩을 가릴 것이기 때문에 파이썬이 바인딩을 가져올 수 없을 것입니다.

1.4.1 독립 실행형 스크립트에서 PyQGIS 사용하기

독립 실행형 스크립트를 시작하려면, 스크립트 시작 부분에 QGIS 리소스를 초기 설정하십시오:

```

1 from qgis.core import *
2
3 # Supply path to qgis install location
4 QgsApplication.setPrefixPath("/path/to/qgis/installation", True)
5
6 # Create a reference to the QgsApplication. Setting the
7 # second argument to False disables the GUI.
8 qgs = QgsApplication([], False)
9
10 # Load providers
11 qgs.initQgis()
12
13 # Write your code here to load some layers, use processing
14 # algorithms, etc.
15
16 # Finally, exitQgis() is called to remove the
17 # provider and layer registries from memory
18 qgs.exitQgis()
```

먼저 `qgis.core` 모듈을 가져온 다음 앞에 붙는 (prefix) 경로를 환경설정합니다. 이 접두 경로는 QGIS 가 사용자 시스템 상에 설치돼 있는 위치입니다. 스크립트에서 `setPrefixPath()` 메소드를 호출해서 접두 경로를 환경설정합니다. `setPrefixPath()` 의 두 번째 인자를 `True` 로 설정해서 기본 경로들을 사용한다고 지정합니다.

QGIS 의 설치 경로는 플랫폼에 따라 다릅니다. 사용자 시스템에서 설치 경로를 찾는 가장 쉬운 방법은 QGIS 안에서파이썬 콘솔에서의 스크립트 작업을 사용해서 다음을 실행한 다음 출력물을 살펴보는 것입니다:

```
QgsApplication.prefixPath()
```

접두 경로를 환경설정한 다음, `qgs` 변수에 `QgsApplication` 클래스를 가리키는 참조를 저장합니다. 두 번째 인자를 `False` 로 설정해서 GUI 를 사용할 계획이 없다고 지정합니다. 우리는 지금 독립 실행형 스크립트를 작성하고 있기 때문이죠. `QgsApplication` 클래스를 환경설정한 다음, `initQgis()` 메소드를 호출해서 QGIS 데이터 제공자와 레이어 레지스트리를 불러옵니다.

```
qgs.initQgis()
```

QGIS 를 초기 설정하고 나면, 스크립트의 다음 부분을 작성할 준비가 끝납니다. 마지막으로 `exitQgis()` 메소드를 호출해서 메모리로부터 데이터 제공자와 레이어 레지스트리를 제거하는 것으로 마무리합니다.

```
qgs.exitQgis()
```

1.4.2 사용자 정의 응용 프로그램에서 PyQGIS 사용하기

독립 실행형 스크립트에서 PyQGIS 사용하기 와 사용자 정의 PyQGIS 응용 프로그램의 유일한 차이점은 `QgsApplication` 클래스를 인스턴스화할 때의 두 번째 인자뿐입니다. `False` 대신 `True` 를 전달해서 GUI 를 사용할 계획이라고 나타내십시오.

```

1 from qgis.core import *
2
3 # Supply the path to the qgis install location
4 QgsApplication.setPrefixPath("/path/to/qgis/installation", True)
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

5
6 # Create a reference to the QgsApplication.
7 # Setting the second argument to True enables the GUI. We need
8 # this since this is a custom application.
9
10 qgs = QgsApplication([], True)
11
12 # load providers
13 qgs.initQgis()
14
15 # Write your code here to load some layers, use processing
16 # algorithms, etc.
17
18 # Finally, exitQgis() is called to remove the
19 # provider and layer registries from memory
20 qgs.exitQgis()

```

이제 QGIS API 를 작업할 수 있습니다—레이어를 불러와서 어떤 공간 처리를 하거나 GUI 를 맵 캔버스와 함께 열거나, 가능성은 무궁무진합니다. :-)

1.4.3 사용자 정의 응용 프로그램 실행하기

QGIS 라이브러리 및 알맞은 파이썬 모듈이 잘 알려진 위치에 없는 경우, 사용자 시스템에 이들을 검색해야 할 위치를 알려줘야 합니다—그렇지 않으면 파이썬이 불평할 것입니다:

```

>>> import qgis.core
ImportError: No module named qgis.core

```

PYTHONPATH 환경 변수를 설정하면 이 문제를 해결할 수 있습니다. 다음 명령어들에서, <qgispath> 를 사용자의 실제 QGIS 설치 경로로 대체해야 할 것입니다:

- 리눅스: **export PYTHONPATH=/<qgispath>/share/qgis/python**
- 윈도우: **set PYTHONPATH=c:\<qgispath>\python**
- 맥 OS: **export PYTHONPATH=/<qgispath>/Contents/Resources/python**

이제 PyQGIS 모듈들을 가리키는 경로를 지정했지만, 이 모듈들은 qgis_core 와 qgis_gui 라이브러리에 의존하고 있습니다. (파이썬 모듈은 래퍼(wrapper) 역할만 할 뿐입니다.) 운영체제가 이 라이브러리들을 가리키는 경로를 모르고 있을 수도 있기 때문에, 이 경우 다시 가져오기 오류 메시지를 보게 될 것입니다. (이 메시지는 사용자 시스템에 따라 달라질 수도 있습니다.):

```

>>> import qgis.core
ImportError: libqgis_core.so.3.2.0: cannot open shared object file:
No such file or directory

```

동적 링커(dynamic linker)의 검색 경로에 QGIS 라이브러리가 있는 디렉토리를 추가해서 이 문제를 해결하십시오:

- 리눅스: **export LD_LIBRARY_PATH=/<qgispath>/lib**
- 윈도우: **set PATH=C:\<qgispath>\bin;C:\<qgispath>\apps\<qgisrelease>\bin;%PATH%**, 이때 <qgisrelease> 를 사용자가 대상으로 하는 배포판 유형 (예: qgis-ltr, qgis, qgis-dev) 으로 대체해야 할 것입니다.

이러한 명령어를 구동 과정을 처리할 부트스트랩(bootstrap) 스크립트에 넣어 둘 수 있습니다. PyQGIS 를 이용하는 사용자 정의 응용 프로그램을 배포하는 경우, 보통 다음 두 가지 방법을 쓸 수 있습니다:

- 사용자 응용 프로그램을 설치하기 전에 필수적으로 QGIS 를 설치해야 한다고 요청하는 방법입니다. 응용 프로그램 설치자가 QGIS 라이브러리의 기본 위치를 검색하게 하고, 찾지 못했을 경우 사용자가 경로를 설정할 수 있도록 해줘야 합니다. 이 방법은 좀 더 간단하다는 장점이 있지만, 사용자가 더 많은 단계를 거쳐야 합니다.
- QGIS 를 응용 프로그램과 함께 패키징하는 방법입니다. 응용 프로그램을 배포하는 데 더 많은 노력이 필요하고 용량이 더 커지게 되지만, 사용자가 추가로 다른 소프트웨어를 다운로드해 설치해야 하는 부담이 줄어들게 됩니다.

이 두 가지 배포 모델을 섞을 수도 있습니다. 윈도우와 맥 OS 에서는 독립 설치형 응용 프로그램을 제공하지만, 리눅스에서는 사용자와 사용자의 패키지 관리자에 GIS 설치를 맡기도록 할 수 있습니다.

1.5 PyQt 및 SIP 에 대한 기술 노트

우리는 스크립트 작업에 가장 선호되는 언어 가운데 하나이기 때문에 파이썬을 사용하기로 결정했습니다. QGIS 3 버전의 PyQGIS 바인딩은 SIP 및 PyQt5 에 의존하고 있습니다. 좀 더 널리 쓰이는 SWIG 대신 SIP 를 사용하는 이유는 QGIS 코드가 Qt 라이브러리에 의존하고 있기 때문입니다. SIP 를 사용해서 Qt(PyQt) 에 파이썬을 바인딩하므로 PyQGIS 를 PyQt 와 완벽하게 통합할 수 있는 것입니다.

CHAPTER 2

프로젝트 불러오기

💡 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (  
2     Qgis,  
3     QgsProject,  
4     QgsPathResolver  
5 )  
6  
7 from qgis.gui import (  
8     QgsLayerTreeMapCanvasBridge,  
9 )
```

플러그인으로부터 기존 프로젝트를, 또는 (더 자주) 독립 설치형 QGIS 파이썬 응용 프로그램을 개발할 때, 기존 프로젝트를 불러와야 하는 경우가 많습니다. (파이썬 응용 프로그램 참조)

현재 QGIS 응용 프로그램으로 프로젝트를 불러오려면 `QgsProject` 클래스의 인스턴스를 생성해야 합니다. 이 클래스는 싱글턴 (singleton) 클래스이기 때문에, 생성하려면 이 클래스의 `instance()` 메소드를 사용해야만 합니다. 이 클래스의 `read()` 메소드를 호출해서 불러올 프로젝트의 경로를 넘겨줄 수 있습니다:

```
1 # If you are not inside a QGIS console you first need to import  
2 # qgis and PyQt classes you will use in this script as shown below:  
3 from qgis.core import QgsProject  
4 # Get the project instance  
5 project = QgsProject.instance()  
6 # Print the current project file name (might be empty in case no projects have been  
7 #   ↳ loaded)  
8 # print(project.fileName())  
9  
10 # Load another project  
11 project.read('testdata/01_project.qgs')  
print(project.fileName())
```

```
testdata/01_project.qgs
```

프로젝트를 (예를 들어 몇몇 레이어를 추가하거나 제거하기 위해) 수정하고 변경 사항을 저장해야 한다면, 프로젝트 인스턴스의 `write()` 메소드를 호출하십시오. `write()` 메소드도 프로젝트를 새 위치에 저장하기 위한 선택적인 경로를 입력받습니다.

```
# Save the project to the same
project.write()
# ... or to a new file
project.write('testdata/my_new_qgis_project.qgs')
```

`read()` 와 `write()` 두 함수 모두 작업 수행이 성공적이었는지 확인할 수 있는 불 (boolean) 값을 반환합니다.

참고

QGIS 독립 설치형 응용 프로그램을 작성하고 있는 경우, 불러온 프로젝트를 캔버스와 동기화시키려면 다음 예시에서처럼 `QgsLayerTreeMapCanvasBridge` 클래스의 인스턴스를 생성해야 합니다:

```
bridge = QgsLayerTreeMapCanvasBridge( \
    QgsProject.instance().layerTreeRoot(), canvas)
# Now you can safely load your project and see it in the canvas
project.read('testdata/my_new_qgis_project.qgs')
```

2.1 경로 오류 해결하기

프로젝트에 불러온 레이어들이 또다른 위치로 옮겨지는 경우가 있을 수 있습니다. 이런 프로젝트를 다시 불러오는 경우 모든 레이어 경로가 깨지게 됩니다. `QgsPathResolver` 클래스를 사용하면 프로젝트 안에서 레이어 경로를 다시 작성할 수 있습니다.

이 클래스의 `setPathPreprocessor()` 메소드는 사용자 정의 경로 전처리기 (pre-processor) 함수를 설정해서, 경로와 데이터 소스를 파일 참조 또는 레이어 소스로 해석하기 전에 조작할 수 있게 해줍니다.

처리기 (processor) 함수는 반드시 (원본 파일 경로 또는 데이터 소스를 표현하는) 단일 문자열 인자를 받아서 해당 경로를 처리한 버전을 반환해야만 합니다. 오류가 발생한 어떤 레이어 처리자 (handler) 보다도 먼저 이 경로 전처리기 함수를 호출합니다. 전처리기를 여러 개 설정한 경우, 원래 전처리기들이 설정된 순서대로 차례차례 호출할 것입니다.

다음은 몇몇 활용 사례입니다:

1. 변경된 경로를 대체:

```
def my_processor(path):
    return path.replace('c:/Users/ClintBarton/Documents/Projects', 'x:/Projects/')

QgsPathResolver.setPathPreprocessor(my_processor)
```

2. 새 데이터베이스 호스트 주소로 대체:

```
def my_processor(path):
    return path.replace('host=10.1.1.115', 'host=10.1.1.116')

QgsPathResolver.setPathPreprocessor(my_processor)
```

3. 저장된 데이터베이스 인증 정보를 새 정보로 대체:

```

1 def my_processor(path):
2     path= path.replace("user='gis_team'", "user='team_awesome'")
3     path = path.replace("password='cats'", "password='g7as!m*'"')
4     return path
5
6 QgsPathResolver.setPathPreprocessor(my_processor)

```

마찬가지로, 경로 작성자 함수에 `setPathWriter()` 메소드를 사용할 수 있습니다.

경로를 변수로 대체하는 예시입니다:

```

def my_processor(path):
    return path.replace('c:/Users/ClintBarton/Documents/Projects', '$projectdir$')

QgsPathResolver.setPathWriter(my_processor)

```

두 메소드 모두 전처리기 또는 메소드가 추가한 작성자를 제거하는 데 사용할 수 있는 id 를 반환합니다. `removePathPreprocessor()` 및 `removePathWriter()` 를 참조하세요.

2.2 속도 향상을 위해 플래그 사용하기

완전하게 기능하는 프로젝트를 사용할 필요는 없고 특정한 이유 때문에 프로젝트에 접근만 하면 되는 경우, 플래그가 도움이 될 수도 있습니다. `ProjectReadFlag` 클래스 아래에서 전체 플래그 목록을 찾아볼 수 있습니다. 플래그 여러 개를 함께 추가할 수도 있습니다.

예를 들어 실제 레이어와 데이터에는 관심이 없고 (예를 들면 조판 또는 3D 뷰 설정 때문에) 단순히 프로젝트에 접근만 하고 싶은 경우, 데이터 무결성 검증 단계를 뛰어넘고 오류가 발생한 레이어 대화창이 열리는 일을 방지하려면 `DontResolveLayers` 플래그를 다음 코드처럼 사용하면 됩니다:

```

readflags = Qgs.ProjectReadFlags()
readflags |= Qgs.ProjectReadFlag.DontResolveLayers
project = QgsProject.instance()
project.read('C:/Users/ClintBarton/Documents/Projects/mysweetproject.qgs', readflags)

```

플래그를 더 많이 추가하려면 반드시 파이썬 비트 연산자 (bitwise operator) `OR(|)` 를 사용해야만 합니다.

레이어 불러오기

 힌트

이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
import os # This is needed in the pyqgis console also
from qgis.core import (
    QgsVectorLayer
)
```

데이터를 담고 있는 레이어를 열어봅시다. QGIS 는 벡터 및 래스터 레이어를 인식합니다. 추가적으로 사용자 정의 레이어 유형도 사용할 수 있지만, 이 문서에서는 다루지 않을 것입니다.

3.1 벡터 레이어

To create and add a vector layer instance to the project, specify the layer's data source identifier. The data source identifier is a string and it is specific to each vector data provider. An optional layer name is used for identifying the layer in the *Layers* panel. It is important to check whether the layer has been loaded successfully. If it was not, an invalid layer instance is returned.

GeoPackage 벡터 레이어의 경우:

```
1 # get the path to a geopackage
2 path_to_gpkg = "testdata/data/data.gpkg"
3 # append the layername part
4 gpkg_airports_layer = path_to_gpkg + "|layername=airports"
5 vlayer = QgsVectorLayer(gpkg_airports_layer, "Airports layer", "ogr")
6 if not vlayer.isValid():
7     print("Layer failed to load!")
8 else:
9     QgsProject.instance().addMapLayer(vlayer)
```

The quickest way to open and display a vector layer in QGIS is the `addVectorLayer()` method of the `QgisInterface` class:

```
vlayer = iface.addVectorLayer(gpkg_airports_layer, "Airports layer", "ogr")
if not vlayer:
    print("Layer failed to load!")
```

이렇게 하면 한번에 새 레이어를 생성해서 현재 QGIS 프로젝트에 추가합니다. (레이어 목록에 새 레이어를 나타나게 합니다.) 이 함수는 레이어 인스턴스를 반환하거나 레이어를 불러올 수 없는 경우 `None` 을 반환합니다.

다음 목록은 벡터 데이터 제공자들을 통해 여러 데이터 소스에 접근하는 방법을 보여줍니다:

- The ogr provider from the GDAL library supports a **wide variety of formats**, also called drivers in GDAL speak. Examples are ESRI Shapefile, Geopackage, Flatgeobuf, Geojson, ... For single-file formats the filepath usually suffices as uri. For geopackages or dxf, a pipe separated suffix allows to specify the layer to load.

– for ESRI Shapefile:

```
uri = "testdata/airports.shp"
vlayer = QgsVectorLayer(uri, "layer_name_you_like", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

– for Geopackage (note the internal options in data source uri):

```
uri = "testdata/data/data.gpkg|layername=airports"
vlayer = QgsVectorLayer(uri, "layer_name_you_like", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

Note that if your GeoPackage contains relationships between layers. Simply using `addMapLayer` does not automatically create these relationships between the layers in your QGIS project. Add the following code to automatically find the relationships in the database and add appropriate ones to the project properties.

```
1 from qgis.core import QgsRelationManager
2
3 project = QgsProject.instance()
4 relation_manager = project.relationManager()
5 existing_relations = list(relation_manager.relations().values())
6
7 layers = [
8     layer for layer in project.mapLayers().values() if layer.type() == layer.
9     ↳ VectorLayer
10 ]
11 discovered_relations = QgsRelationManager.discoverRelations(existing_
12     ↳ relations, layers)
13 for relation in discovered_relations:
14     relation_manager.addRelation(relation)
```

– DXF 파일의 경우 (데이터 소스 URI 에 있는 내부 옵션들을 주목하십시오):

```
uri = "testdata/sample.dxf|layername=entities|geometrytype=Polygon"
vlayer = QgsVectorLayer(uri, "layer_name_you_like", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

- PostGIS 데이터베이스—데이터 소스는 PostgreSQL 데이터베이스와의 연결을 생성하는 데 필요한 모든 정보를 가진 문자열입니다.

`QgsDataSourceUri` 클래스가 이 문자열을 생성할 수 있습니다. QGIS 가 PostgreSQL 를 지원하도록 QGIS 를 컴파일해야 한다는 점을 기억하십시오. 그렇지 않으면 이 제공자를 사용할 수 없습니다:

```

1 uri = QgsDataSourceUri()
2 # set host name, port, database name, username and password
3 uri.setConnection("localhost", "5432", "dbname", "johny", "xxx")
4 # set database schema, table name, geometry column and optionally
5 # subset (WHERE clause)
6 uri.setDataSource("public", "roads", "the_geom", "cityid = 2643", "primary_key_
   ↳field")
7
8 vlayer = QgsVectorLayer(uri.uri(False), "layer name you like", "postgres")

```

참고

uri.uri(False) 에 전달된 False 인자가 인증 환경설정 파라미터들이 확장되지 못하게 막습니다. 사용자가 어떤 인증 환경설정도 사용하지 않는 경우, 이 인자는 어떤 영향도 미치지 않습니다.

- CSV 파일 또는 기타 구분된 텍스트 파일들—쌍반점 (;) 을 구분자로 사용하고 “x” 필드에 X 좌표를 그리고 “y” 필드에 Y 좌표를 가진 파일을 열려면 다음과 같은 코드를 사용해야 할 것입니다:

```

uri = "file:///testdata/delimited_xy.csv?delimiter={}&xField={}&yField={}".
   ↳format(os.getcwd(), ";", "x", "y")
vlayer = QgsVectorLayer(uri, "layer name you like", "delimitedtext")
QgsProject.instance().addMapLayer(vlayer)

```

참고

제공자 문자열은 URL 로써 구성되어 있기 때문에, 경로 앞에 반드시 file:// 을 붙여야만 합니다. 또 x 및 y 필드의 대체물로서 WKT(Well Known Text) 서식 도형을 사용할 수 있으며, 좌표제도 지정할 수 있습니다. 다음은 그 예시입니다:

```

uri = "file:///some/path/file.csv?delimiter={}&crs=epsg:4723&wktField={}".
   ↳format(";", "shape")

```

- GPX 파일—“gpx”데이터 제공자는 GPX 파일에서 트랙, 루트, 웨이포인트를 읽어옵니다. GPX 파일을 열려면, 데이터 유형 (트랙/루트/웨이포인트) 을 URL 의 일부분으로 지정해줘야 합니다:

```

uri = "testdata/layers.gpx?type=track"
vlayer = QgsVectorLayer(uri, "layer name you like", "gpx")
QgsProject.instance().addMapLayer(vlayer)

```

- SpatiaLite 데이터베이스—PostGIS 데이터베이스와 유사하게, QgsDataSourceUri 클래스를 사용해서 데이터 소스 식별자를 생성할 수 있습니다:

```

1 uri = QgsDataSourceUri()
2 uri.setDatabase('/home/martin/test-2.3.sqlite')
3 schema = ''
4 table = 'Towns'
5 geom_column = 'Geometry'
6 uri.setDataSource(schema, table, geom_column)
7
8 display_name = 'Towns'
9 vlayer = QgsVectorLayer(uri.uri(), display_name, 'spatialite')
10 QgsProject.instance().addMapLayer(vlayer)

```

- GDAL 을 통한 MySQL WKB 기반 도형—데이터 소스는 테이블과의 연결 문자열입니다:

```
uri = "MySQL:dbname,host=localhost,port=3306,user=root,password=xxx|layername=my_
↳table"
vlayer = QgsVectorLayer( uri, "my table", "ogr" )
QgsProject.instance().addMapLayer(vlayer)
```

- WFS 연결—URI 와 WFS 제공자를 사용해서 연결을 정의합니다:

```
uri = "https://demo.mapserver.org/cgi-bin/wfs?service=WFS&version=2.0.0&
↳request=GetFeature&typename=ms:cities"
vlayer = QgsVectorLayer(uri, "my wfs layer", "WFS")
```

표준 urllib 라이브러리를 사용해서 URI 를 생성할 수 있습니다:

```
1 import urllib
2
3 params = {
4     'service': 'WFS',
5     'version': '2.0.0',
6     'request': 'GetFeature',
7     'typename': 'ms:cities',
8     'srsname': "EPSG:4326"
9 }
10 uri2 = 'https://demo.mapserver.org/cgi-bin/wfs?' + urllib.parse.unquote(urllib.
↳parse.urlencode(params))
```

참고

다음 예시처럼 `QgsVectorLayer` 클래스 인스턴스에 대해 `setDataSource()` 메소드를 호출하면 기존 레이어의 데이터 소스를 변경할 수 있습니다:

```
1 uri = "https://demo.mapserver.org/cgi-bin/wfs?service=WFS&version=2.0.0&
↳request=GetFeature&typename=ms:cities"
2 provider_options = QgsDataProvider.ProviderOptions()
3 # Use project's transform context
4 provider_options.transformContext = QgsProject.instance().transformContext()
5 vlayer.setDataSource(uri, "layer name you like", "WFS", provider_options)
6
7 del(vlayer)
```

3.2 래스터 레이어

래스터 파일에 접근하기 위해 GDAL 라이브러리를 사용합니다. GDAL 라이브러리는 광범위한 파일 포맷들을 지원합니다. 어떤 파일을 여는 데 문제가 있는 경우, 사용자의 GDAL 라이브러리가 그 특정 포맷을 지원하고 있는지 확인하십시오. (기본적으로 모든 포맷을 지원하지는 않습니다.) 파일로부터 래스터를 불러오려면, 래스터의 파일 이름과 표시 이름을 지정하십시오:

```
1 # get the path to a tif file e.g. /home/project/data/srtm.tif
2 path_to_tif = "qgis-projects/python_cookbook/data/srtm.tif"
3 rlayer = QgsRasterLayer(path_to_tif, "SRTM layer name")
4 if not rlayer.isValid():
5     print("Layer failed to load!")
```


GeoPackage 에서 래스터를 불러오려면:

```

1 # get the path to a geopackage e.g. /home/project/data/data.gpkg
2 path_to_gpkg = os.path.join(os.getcwd(), "testdata", "sublayers.gpkg")
3 # gpkg_raster_layer = "GPKG:/home/project/data/data.gpkg:srtm"
4 gpkg_raster_layer = "GPKG:" + path_to_gpkg + ":srtm"
5
6 rlayer = QgsRasterLayer(gpkg_raster_layer, "layer name you like", "gdal")
7
8 if not rlayer.isValid():
9     print("Layer failed to load!")

```

벡터 레이어와 비슷하게, `QgisInterface` 클래스 객체의 `addRasterLayer` 함수를 사용해서 래스터 레이어를 불러올 수 있습니다:

```
iface.addRasterLayer(path_to_tif, "layer name you like")
```

이렇게 하면 한번에 새 레이어를 생성해서 현재 프로젝트에 추가합니다. (레이어 목록에 새 레이어를 나타나게 합니다.)

PostGIS 래스터를 불러오려면:

PostGIS 벡터와 비슷하게, URI 문자열을 사용해서 프로젝트에 PostGIS 래스터를 추가할 수 있습니다. 데이터베이스 연결 파라미터에 대해 재사용 가능한 문자열 목록 (dictionary) 을 유지하는 편이 효율적입니다. 이렇게 하면 적용 가능한 연결에 대한 목록을 쉽게 편집할 수 있습니다. 그 다음 'postgresraster' 제공자 메타데이터 객체를 사용해서 목록을 URI 로 인코딩합니다. 그러면 프로젝트에 래스터를 추가할 수 있습니다.

```

1 uri_config = {
2     # database parameters
3     'dbname': 'gis_db',          # The PostgreSQL database to connect to.
4     'host': 'localhost',        # The host IP address or localhost.
5     'port': '5432',             # The port to connect on.
6     'sslmode': QgsDataSourceUri.SslDisable, # SslAllow, SslPrefer, SslRequire,
    ↳ SslVerifyCa, SslVerifyFull
7     # user and password are not needed if stored in the authcfg or service
8     'authcfg': 'QconfigId',     # The QGIS authentication database ID holding connection
    ↳ details.
9     'service': None,            # The PostgreSQL service to be used for connection to
    ↳ the database.
10    'username': None,            # The PostgreSQL user name.
11    'password': None,            # The PostgreSQL password for the user.
12    # table and raster column details
13    'schema': 'public',          # The database schema that the table is located in.
14    'table': 'my_rasters',       # The database table to be loaded.
15    'geometrycolumn': 'rast',    # raster column in PostGIS table
16    'sql': None,                 # An SQL WHERE clause. It should be placed at the end of
    ↳ the string.
17    'key': None,                 # A key column from the table.
18    'srid': None,                # A string designating the SRID of the coordinate
    ↳ reference system.
19    'estimatedmetadata': 'False', # A boolean value telling if the metadata is
    ↳ estimated.
20    'type': None,                # A WKT string designating the WKB Type.
21    'selectatid': None,          # Set to True to disable selection by feature ID.
22    'options': None,             # other PostgreSQL connection options not in this list.
23    'enableTime': None,
24    'temporalDefaultTime': None,
25    'temporalFieldIndex': None,

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

26     'mode':'2',                # GDAL 'mode' parameter, 2 unions raster tiles, 1 adds_
    ↪ tiles separately (may require user input)
27 }
28 # remove any NULL parameters
29 uri_config = {key:val for key, val in uri_config.items() if val is not None}
30 # get the metadata for the raster provider and configure the URI
31 md = QgsProviderRegistry.instance().providerMetadata('postgresraster')
32 uri = QgsDataSourceUri(md.encodeUri(uri_config))
33
34 # the raster can then be loaded into the project
35 rlayer = iface.addRasterLayer(uri.uri(False), "raster layer name", "postgresraster")

```

WCS 서비스로부터도 래스터 레이어를 생성할 수 있습니다:

```

layer_name = 'modis'
url = "https://demo.mapserver.org/cgi-bin/wcs?identifier={}".format(layer_name)
rlayer = QgsRasterLayer(uri, 'my wcs layer', 'wcs')

```

다음에서는 WCS URI 가 담을 수 있는 파라미터들을 설명합니다:

WCS URI 는 & 문자로 구분된 키 = 값 쌍으로 이루어집니다. 이는 URL 에 있는 동일한 방식으로 인코딩된 쿼리 문자열과 동일한 서식입니다. 특수 문자들이 제대로 인코딩되도록 보장하려면 `QgsDataSourceUri` 클래스를 사용해서 URI 를 작성해야 할 것입니다.

- **url** (필수): WCS 서버 URL 입니다. WCS 각 버전이 **GetCapabilities** 버전에 대해 서로 다른 파라미터 이름을 사용하고 있기 때문에, URL 에 **VERSION** 을 사용하면 안 됩니다. 파라미터 버전을 참조하세요.
- **identifier** (필수): 커버리지 이름
- **time** (선택적): 시점 (time position) 또는 기간 (time period) (beginPosition/endTimePosition[/timeResolution])
- **format** (선택적): 지원하는 포맷 이름입니다. 기본값은 이름에 `tif` 가 있는 경우 첫 번째로 지원하는 포맷 또는 그렇지 않은 경우 첫 번째로 지원하는 포맷입니다.
- **crs** (선택적) : 기관:ID 형식의, 예를 들면 EPSG:4326 같은 좌표계입니다. 기본값은 지원하는 경우 EPSG:4326, 지원하지 않는 경우 첫 번째로 지원하는 좌표계입니다.
- **username** (선택적): 기본 인증을 위한 사용자 이름입니다.
- **password** (선택적): 기본 인증을 위한 비밀번호입니다.
- **IgnoreGetMapUrl** (선택적, 필수): 지정한 (1 로 설정한) 경우, **GetCapabilities** 가 노출하는 **GetCoverage** URL 을 무시합니다. 서버를 제대로 환경설정하지 않은 경우 필요할 수도 있습니다.
- **InvertAxisOrientation** (선택적, 필수): 지정한 (1 로 설정한) 경우, **GetCoverage** 요청에 있는 축의 순서를 바꿉니다. 서버가 사용하는 축 순서가 틀린 경우 지리 좌표계에 대해 필요할 수도 있습니다.
- **IgnoreAxisOrientation** (선택적, 필수): 지정한 (1 로 설정한) 경우, 지리 좌표계에 대해 WCS 표준을 준수하는 축 순서로 바꾸지 않습니다.
- **cache** (선택적): `QNetworkRequest::CacheLoadControl` 에서 설명하는 대로 캐시 불러오기를 제어하지만, `AlwaysCache` 로 실패한 경우 요청을 `PreferCache` 로 다시 보냅니다. 다음 값들 가운데 하나로 설정할 수 있습니다: `AlwaysCache`, `PreferCache`, `PreferNetwork`, `AlwaysNetwork`. 기본값은 `AlwaysCache` 입니다.

아니면 WMS 서버에서 래스터 레이어를 불러올 수도 있습니다. 하지만 현재 API 에서 **GetCapabilities** 응답에 접근할 수가 없습니다—사용자가 원하는 레이어가 어떤 레이어인지 알고 있어야 합니다:

```

urlWithParams = "crs=EPSG:4326&format=image/png&layers=continents&styles&url=https://
    ↪ demo.mapserver.org/cgi-bin/wms"
rlayer = QgsRasterLayer(urlWithParams, 'some layer name', 'wms')

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
if not rlayer.isValid():
    print("Layer failed to load!")
```

3.3 QgsProject 인스턴스

렌더링 작업에 열려져 있는 레이어를 사용하고 싶다면, `QgsProject` 클래스 인스턴스에 해당 레이어를 추가해야 한다는 사실을 기억하십시오. `QgsProject` 클래스 인스턴스는 레이어의 소유권을 가지며, 이후 응용 프로그램의 어느 부분에서든 레이어의 유일 ID 를 통해 해당 레이어에 접근할 수 있습니다. 프로젝트에서 레이어를 제거하는 경우, 유일 ID 도 삭제됩니다. 사용자는 QGIS 인터페이스에서 또는 `removeMapLayer()` 메소드를 사용하는 파이썬 코드를 통해 레이어를 제거할 수 있습니다.

`addMapLayer()` 메소드를 사용하면 현재 프로젝트에 레이어를 추가할 수 있습니다:

```
QgsProject.instance().addMapLayer(rlayer)
```

절대 위치에 레이어를 추가하려면:

```
1 # first add the layer without showing it
2 QgsProject.instance().addMapLayer(rlayer, False)
3 # obtain the layer tree of the top-level group in the project
4 layerTree = iface.layerTreeCanvasBridge().rootGroup()
5 # the position is a number starting from 0, with -1 an alias for the end
6 layerTree.insertChildNode(-1, QgsLayerTreeLayer(rlayer))
```

`removeMapLayer()` 메소드를 사용해서 레이어를 삭제하고 싶다면:

```
# QgsProject.instance().removeMapLayer(layer_id)
QgsProject.instance().removeMapLayer(rlayer.id())
```

앞의 코드에서는 (레이어의 `id()` 메소드를 호출해서 얻을 수 있는) 레이어 ID 를 전달했지만, 레이어 객체 자체도 전달할 수 있습니다.

불러온 레이어와 레이어 ID 의 목록을 원한다면, `mapLayers()` 메소드를 사용하십시오:

```
QgsProject.instance().mapLayers()
```

TOC(Table Of Contents) 에 접근하기

힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
from qgis.core import (
    QgsProject,
    QgsVectorLayer,
)
```

서로 다른 클래스들을 사용해서 차례 (Table Of Contents) 에 불러온 모든 레이어에 접근할 수 있으며, 다음 클래스들을 통해 정보를 검색할 수 있습니다:

- `QgsProject`
- `QgsLayerTreeGroup`

4.1 QgsProject 클래스

`QgsProject` 클래스를 사용하면 차례 및 불러온 모든 레이어에 대한 정보를 검색할 수 있습니다.

불러온 레이어들을 가져오려면 `QgsProject` 클래스의 `instance` 를 생성하고 이 클래스의 메소드를 사용해야 합니다.

주 메소드는 `mapLayers()` 입니다. 이 메소드가 불러온 레이어들의 목록 (dictionary) 을 반환할 것입니다:

```
layers = QgsProject.instance().mapLayers()
print(layers)
```

```
{'countries_89ae1b0f_f41b_4f42_bca4_caf55ddbe4b6': <QgsVectorLayer: 'countries' (ogr)>
→ }
```

이 목록의 keys 는 유일한 레이어 ID 이고 values 는 관련 객체입니다.

이제 레이어에 대한 다른 모든 정보를 간단하게 얻을 수 있습니다:

```
1 # list of layer names using list comprehension
2 l = [layer.name() for layer in QgsProject.instance().mapLayers().values()]
3 # dictionary with key = layer name and value = layer object
4 layers_list = {}
5 for l in QgsProject.instance().mapLayers().values():
6     layers_list[l.name()] = l
7
8 print(layers_list)
```

```
{'countries': <QgsVectorLayer: 'countries' (ogr)>}
```

레이어의 이름을 사용해서 차례도 쿼리할 수 있습니다:

```
country_layer = QgsProject.instance().mapLayersByName("countries")[0]
```

참고

이름이 일치하는 모든 레이어들의 목록을 반환하기 때문에, 이 이름을 가진 첫 번째 레이어를 가져오기 위해 [0] 인덱스를 사용합니다.

4.2 QgsLayerTreeGroup class

레이어 트리는 노드 (node) 들로 구성된 전통적인 트리 구조입니다. 현재 두 가지 노드 유형이 존재합니다: 그룹 노드 (QgsLayerTreeGroup) 와 레이어 노드 (QgsLayerTreeLayer) 입니다.

참고

for more information you can read these blog posts of Martin Dobias: [Part 1](#) [Part 2](#) [Part 3](#)

QgsProject 클래스의 layerTreeRoot() 메소드를 사용하면 프로젝트 레이어 트리에 쉽게 접근할 수 있습니다:

```
root = QgsProject.instance().layerTreeRoot()
```

root 는 그룹 노드이며 하위 (children) 노드를 가지고 있습니다:

```
root.children()
```

직계 하위 노드 목록을 반환합니다. 하위 그룹 노드는 그 직계 상위 노드로부터 접근해야 합니다.

하위 노드 가운데 하나를 검색할 수 있습니다:

```
child0 = root.children()[0]
print(child0)
```

```
<QgsLayerTreeLayer: countries>
```

레이어의 (유일) id 를 사용해서도 레이어를 검색할 수 있습니다:

```
ids = root.findLayerIds()
# access the first layer of the ids list
root.findLayer(ids[0])
```

그리고 그룹의 이름을 사용해서도 그룹을 검색할 수 있습니다:

```
root.findGroup('Group Name')
```

`QgsLayerTreeGroup` 클래스는 차례에 대한 더 많은 정보를 가져오는 데 사용할 수 있는 유용한 다른 메소드들을 다수 가지고 있습니다:

```
# list of all the checked layers in the TOC
checked_layers = root.checkedLayers()
print(checked_layers)
```

```
[<QgsVectorLayer: 'countries' (ogr)>]
```

이제 프로젝트의 레이어 트리에 레이어를 몇 개 추가해봅시다. 두 가지 방법이 있습니다:

1. 명시적 추가—`addLayer()` 또는 `insertLayer()` 함수를 사용합니다:

```
1 # create a temporary layer
2 layer1 = QgsVectorLayer("path_to_layer", "Layer 1", "memory")
3 # add the layer to the legend, last position
4 root.addLayer(layer1)
5 # add the layer at given position
6 root.insertLayer(5, layer1)
```

2. 암시적 추가—프로젝트의 레이어 트리가 레이어 레지스트리와 연결돼 있기 때문에 맵 레이어 레지스트리에 레이어를 추가해주는 것으로 충분합니다:

```
QgsProject.instance().addMapLayer(layer1)
```

`QgsVectorLayer` 와 `QgsLayerTreeLayer` 클래스를 쉽게 전환할 수 있습니다:

```
node_layer = root.findLayer(country_layer.id())
print("Layer node:", node_layer)
print("Map layer:", node_layer.layer())
```

```
Layer node: <QgsLayerTreeLayer: countries>
Map layer: <QgsVectorLayer: 'countries' (ogr)>
```

`addGroup()` 메소드를 사용하면 그룹을 쉽게 추가할 수 있습니다. 다음 예시에서, 전자는 차례의 마지막에 그룹을 추가할 것이지만 후자의 경우 기존 그룹 안에 또다른 그룹을 추가할 수 있습니다:

```
node_group1 = root.addGroup('Simple Group')
# add a sub-group to Simple Group
node_subgroup1 = node_group1.addGroup("I'm a sub group")
```

노드 및 그룹을 이동시키고 싶다면, 유용한 메소드들이 많이 있습니다.

기존 노드의 이동은 세 단계로 이루어집니다:

1. 기존 노드 복제
2. 복제한 노드를 원하는 위치로 이동
3. 원본 노드 삭제

```

1 # clone the group
2 cloned_group1 = node_group1.clone()
3 # move the node (along with sub-groups and layers) to the top
4 root.insertChildNode(0, cloned_group1)
5 # remove the original node
6 root.removeChildNode(node_group1)

```

범례에서 레이어를 이동시키는 작업은 좀 더 복잡합니다:

```

1 # get a QgsVectorLayer
2 vl = QgsProject.instance().mapLayersByName("countries")[0]
3 # create a QgsLayerTreeLayer object from vl by its id
4 myvl = root.findLayer(vl.id())
5 # clone the myvl QgsLayerTreeLayer object
6 myvlclone = myvl.clone()
7 # get the parent. If None (layer is not in group) returns ''
8 parent = myvl.parent()
9 # move the cloned layer to the top (0)
10 parent.insertChildNode(0, myvlclone)
11 # remove the original myvl
12 root.removeChildNode(myvl)

```

또는 기존 그룹으로 이동시키는 작업도 그렇죠:

```

1 # get a QgsVectorLayer
2 vl = QgsProject.instance().mapLayersByName("countries")[0]
3 # create a QgsLayerTreeLayer object from vl by its id
4 myvl = root.findLayer(vl.id())
5 # clone the myvl QgsLayerTreeLayer object
6 myvlclone = myvl.clone()
7 # create a new group
8 group1 = root.addGroup("Group1")
9 # get the parent. If None (layer is not in group) returns ''
10 parent = myvl.parent()
11 # move the cloned layer to the top (0)
12 group1.insertChildNode(0, myvlclone)
13 # remove the QgsLayerTreeLayer from its parent
14 parent.removeChildNode(myvl)

```

몇몇 다른 메소드들을 사용해서 그룹과 레이어를 수정할 수 있습니다:

```

1 node_group1 = root.findGroup("Group1")
2 # change the name of the group
3 node_group1.setName("Group X")
4 node_layer2 = root.findLayer(country_layer.id())
5 # change the name of the layer
6 node_layer2.setName("Layer X")
7 # change the visibility of a layer
8 node_group1.setItemVisibilityChecked(True)
9 node_layer2.setItemVisibilityChecked(False)
10 # expand/collapse the group view
11 node_group1.setExpanded(True)
12 node_group1.setExpanded(False)

```


래스터 레이어 사용하기

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```

1 from qgis.core import (
2     QgsRasterLayer,
3     QgsProject,
4     QgsPointXY,
5     QgsRaster,
6     QgsRasterShader,
7     QgsColorRampShader,
8     QgsSingleBandPseudoColorRenderer,
9     QgsSingleBandColorDataRenderer,
10    QgsSingleBandGrayRenderer,
11 )
12
13 from qgis.PyQt.QtGui import (
14     QColor,
15 )

```

5.1 레이어 상세 정보

래스터 레이어는 하나 또는 그 이상의 래스터 밴드로 이루어져 있습니다—이를 단일 밴드 그리고 다중 밴드 래스터라고 합니다. 밴드 하나는 값들의 행렬을 표현합니다. 컬러 이미지 (예: 항공 사진) 는 적색, 청색, 녹색 밴드로 이루어진 래스터입니다. 단일 밴드 래스터는 일반적으로 연속적인 변수들 (예: 표고) 또는 불연속적인 변수들 (예: 토지 이용도) 가운데 하나를 표현합니다. 또는 래스터 레이어에 색상표가 첨부되어 래스터 값들이 색상표에 저장된 색상들을 참조하는 경우도 있습니다.

다음 코드는 `rlayer` 가 `QgsRasterLayer` 클래스 객체라고 가정합니다.

```
rlayer = QgsProject.instance().mapLayersByName('srtm')[0]
# get the resolution of the raster in layer unit
print(rlayer.width(), rlayer.height())
```

```
919 619
```

```
# get the extent of the layer as QgsRectangle
print(rlayer.extent())
```

```
<QgsRectangle: 20.06856808199999875 -34.27001076999999896, 20.83945284300000012 -33.75077500700000144>
```

```
# get the extent of the layer as Strings
print(rlayer.extent().toString())
```

```
20.0685680819999988,-34.2700107699999990 : 20.8394528430000001,-33.7507750070000014
```

```
# get the raster type: 0 = GrayOrUndefined (single band), 1 = Palette (single band),
→ 2 = Multiband
print(rlayer.rasterType())
```

```
RasterLayerType.GrayOrUndefined
```

```
# get the total band count of the raster
print(rlayer.bandCount())
```

```
1
```

```
# get the first band name of the raster
print(rlayer.bandName(1))
```

```
Band 1: Height
```

```
# get all the available metadata as a QgsLayerMetadata object
print(rlayer.metadata())
```

```
<qgis._core.QgsLayerMetadata object at 0x13711d558>
```

5.2 렌더링 작업자

래스터 레이어를 불러왔을 때, 그 유형을 기반으로 기본 렌더링 작업자가 할당됩니다. 레이어 속성에서 또는 프로그래밍 방식으로 렌더링 작업자를 변경할 수 있습니다.

현재 렌더링 작업자를 쿼리하려면:

```
print(rlayer.renderer())
```

```
<qgis._core.QgsSingleBandGrayRenderer object at 0x7f471c1da8a0>
```

```
print(rlayer.renderer().type())
```

```
singlebandgray
```

렌더링 작업을 설정하려면 `QgsRasterLayer` 클래스의 `setRenderer()` 메소드를 사용하십시오. (`QgsRasterRenderer` 클래스에서 파생된) 렌더링 작업자 클래스가 여러 개 있습니다:

- `QgsHillshadeRenderer`
- `QgsMultiBandColorRenderer`
- `QgsPalettedRasterRenderer`
- `QgsRasterContourRenderer`
- `QgsSingleBandColorDataRenderer`
- `QgsSingleBandGrayRenderer`
- `QgsSingleBandPseudoColorRenderer`

단일 밴드 래스터는 회색조 (낮은 값은 검정색, 높은 값은 흰색) 또는 값에 색상을 할당하는 의사 색상 (pseudo-color) 알고리즘 가운데 하나로 그릴 수 있습니다. 색상표를 가진 단일 밴드 래스터는 색상표를 사용해서도 그릴 수 있습니다. 다중 밴드 래스터는 일반적으로 RGB 색상에 밴드들을 매핑시켜서 그립니다. 또는 밴드 단 한 개만 사용해서 그릴 수도 있습니다.

5.2.1 단일 밴드 래스터

단일 밴드 래스터 레이어를 (0에서 255까지의 픽셀 값들에 대응하는) 초록색에서 노란색 범위의 색상을 사용해서 렌더링하려 한다고 해봅시다. 첫 번째 단계는 `QgsRasterShader` 클래스 객체를 준비해서 그 셰이더 (shader) 함수를 환경설정하는 것입니다:

```
1 fcn = QgsColorRampShader()
2 fcn.setColorRampType(QgsColorRampShader.Interpolated)
3 lst = [ QgsColorRampShader.ColorRampItem(0, QColor(0,255,0)),
4         QgsColorRampShader.ColorRampItem(255, QColor(255,255,0)) ]
5 fcn.setColorRampItemList(lst)
6 shader = QgsRasterShader()
7 shader.setRasterShaderFunction(fcn)
```

셰이더는 자체 색상표가 지정하는 대로 색상을 매핑합니다. 이 색상표가 픽셀 값들과 그 관련 색상들의 목록을 제공합니다. 세 가지 보간 모드가 있습니다:

- 선형 (Interpolated): 픽셀 값 위아래 있는 색상표 항목들로부터 색상을 선형적으로 보간합니다.
- 불연속 (Discrete): 픽셀 값 이상의 값을 가진 색상표 항목들 가운데 픽셀 값과 가장 가까이 있는 색상표 항목의 색상을 사용합니다.
- 정확 (Exact): 색상을 보간하지 않고, 색상표 항목들과 동등한 값을 가진 픽셀들만 그릴 것입니다.

두 번째 단계에서는 래스터 레이어에 이 셰이더를 할당할 것입니다:

```
renderer = QgsSingleBandPseudoColorRenderer(rlayer.dataProvider(), 1, shader)
rlayer.setRenderer(renderer)
```

앞의 코드에 있는 숫자 1은 밴드 번호입니다. (래스터 밴드의 인덱스는 1부터 시작합니다.)

마지막으로 결과물을 보려면 `triggerRepaint()` 메소드를 사용해야 합니다:

```
rlayer.triggerRepaint()
```

5.2.2 다중 밴드 래스터

QGIS 는 컬러 이미지를 생성하기 위해 기본적으로 처음 3 개의 밴드를 적색, 녹색, 청색으로 매핑합니다. (이를 MultiBandColor 그리기 스타일이라 합니다.) 그러나 이 설정을 무시하고 싶은 경우가 있을 수도 있습니다. 다음은 적색 밴드 (1) 와 녹색 밴드 (2) 를 교환하는 코드입니다:

```
rlayer_multi = QgsProject.instance().mapLayersByName('multiband')[0]
rlayer_multi.renderer().setGreenBand(1)
rlayer_multi.renderer().setRedBand(2)
```

래스터를 가시화하는 데 밴드 하나만 필요한 경우, 단일 밴드 그리기를 회색조 또는 의사 색상 가운데 하나로 선택하면 됩니다.

맵을 업데이트해서 결과물을 보려면 `triggerRepaint()` 메소드를 사용해야 합니다:

```
rlayer_multi.triggerRepaint()
```

5.3 값 쿼리하기

`QgsRasterDataProvider` 클래스의 `sample()` 메소드를 사용하면 래스터 값을 쿼리할 수 있습니다. `QgsPointXY` 클래스와 여러분이 쿼리하려는 래스터 레이어의 밴드 번호를 지정해야 합니다. 이 메소드는 래스터 값과 쿼리 결과에 따라 `True` 또는 `False` 를 가진 튜플 (tuple) 을 반환합니다:

```
val, res = rlayer.dataProvider().sample(QgsPointXY(20.50, -34), 1)
```

래스터 값을 쿼리할 수 있는 또다른 메소드는 `QgsRasterIdentifyResult` 클래스 객체를 반환하는 `identify()` 메소드입니다.

```
ident = rlayer.dataProvider().identify(QgsPointXY(20.5, -34), QgsRaster.
    ↳IdentifyFormatValue)

if ident.isValid():
    print(ident.results())
```

```
{1: 323.0}
```

이 경우, `results()` 메소드가 밴드 인덱스를 키로 그리고 밴드 값을 값으로 가지는 목록 (dictionary) 을 반환합니다. 예를 들면 `{1: 323.0}` 같은 식으로 말이죠.

5.4 래스터 데이터 편집하기

`QgsRasterBlock` 클래스를 사용하면 래스터 레이어를 생성할 수 있습니다. 예를 들어 픽셀 당 1 바이트를 가진 2x2 래스터 블록을 생성하려면:

```
block = QgsRasterBlock(Qgis.Byte, 2, 2)
block.setData(b'\xaa\xbb\xcc\xdd')
```

`writeBlock()` 메소드 덕분에 래스터 픽셀을 덮어쓸 수 있습니다. 2x2 블록의 0,0 위치에 있는 기존 래스터 데이터를 덮어쓰려면:

```
provider = rlayer.dataProvider()
provider.setEditable(True)
provider.writeBlock(block, 1, 0, 0)
provider.setEditable(False)
```


벡터 레이어 사용하기

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (  
2     QgsApplication,  
3     QgsDataSourceUri,  
4     QgsCategorizedSymbolRenderer,  
5     QgsClassificationRange,  
6     QgsPointXY,  
7     QgsProject,  
8     QgsExpression,  
9     QgsField,  
10    QgsFields,  
11    QgsFeature,  
12    QgsFeatureRequest,  
13    QgsFeatureRenderer,  
14    QgsGeometry,  
15    QgsGraduatedSymbolRenderer,  
16    QgsMarkerSymbol,  
17    QgsMessageLog,  
18    QgsRectangle,  
19    QgsRendererCategory,  
20    QgsRendererRange,  
21    QgsSymbol,  
22    QgsVectorDataProvider,  
23    QgsVectorLayer,  
24    QgsVectorFileWriter,  
25    QgsWkbTypes,  
26    QgsSpatialIndex,  
27    QgsVectorLayerUtils  
28 )  
29  
30 from qgis.core.additions.edit import edit
```

```

31
32 from qgis.PyQt.QtGui import (
33     QColor,
34 )

```

이 장에서는 벡터 레이어에 대해 할 수 있는 여러 가지 작업들을 소개합니다.

이 장에서 소개하는 대부분의 작업은 `QgsVectorLayer` 클래스의 메소드들에 바탕을 두고 있습니다.

6.1 속성에 대한 정보 검색하기

`QgsVectorLayer` 클래스 객체에 대해 `fields()` 메소드를 호출하면 벡터 레이어와 연관된 필드들에 대한 정보를 검색할 수 있습니다:

```

vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports layer
↪", "ogr")
for field in vlayer.fields():
    print(field.name(), field.typeName())

```

```

1 fid Integer64
2 id Integer64
3 scalerank Integer64
4 featurecla String
5 type String
6 name String
7 abbrev String
8 location String
9 gps_code String
10 iata_code String
11 wikipedia String
12 natlscale Real

```

`displayField()` 와 `mapTipTemplate()` 메소드는 `maptips` 탭에서 쓰이는 필드와 템플릿에 대한 정보를 제공합니다.

벡터 레이어를 불러왔을 때 QGIS 는 언제나 필드 하나를 Display Name 으로 선택하는 반면, HTML Map Tip 은 기본적으로 비어 있습니다. 이 메소드들을 사용하면 둘 다 손쉽게 가져올 수 있습니다:

```

vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports layer
↪", "ogr")
print(vlayer.displayField())

```

```
name
```

참고

Display Name 을 필드에서 표현식으로 변경할 경우, `displayField()` 대신 `displayExpression()` 메소드를 사용해야 합니다.

6.2 벡터 레이어 작업 반복하기

벡터 레이어에 있는 피처들에 대한 작업을 반복하는 일은 가장 흔한 작업 가운데 하나입니다. 다음은 이런 작업을 수행하는 단순한 기본 코드의 예시로, 각 피처에 대한 몇몇 정보를 출력합니다. layer 변수가 `QgsVectorLayer` 클래스를 가지고 있다고 가정합니다.

```

1  # "layer" is a QgsVectorLayer instance
2  layer = iface.activeLayer()
3  features = layer.getFeatures()
4
5  for feature in features:
6      # retrieve every feature with its geometry and attributes
7      print("Feature ID: ", feature.id())
8      # fetch geometry
9      # show some information about the feature geometry
10     geom = feature.geometry()
11     geomSingleType = QgsWkbTypes.isSingleType(geom.wkbType())
12     if geom.type() == QgsWkbTypes.PointGeometry:
13         # the geometry type can be of single or multi type
14         if geomSingleType:
15             x = geom.asPoint()
16             print("Point: ", x)
17         else:
18             x = geom.asMultiPoint()
19             print("MultiPoint: ", x)
20     elif geom.type() == QgsWkbTypes.LineGeometry:
21         if geomSingleType:
22             x = geom.asPolyline()
23             print("Line: ", x, "length: ", geom.length())
24         else:
25             x = geom.asMultiPolyline()
26             print("MultiLine: ", x, "length: ", geom.length())
27     elif geom.type() == QgsWkbTypes.PolygonGeometry:
28         if geomSingleType:
29             x = geom.asPolygon()
30             print("Polygon: ", x, "Area: ", geom.area())
31         else:
32             x = geom.asMultiPolygon()
33             print("MultiPolygon: ", x, "Area: ", geom.area())
34     else:
35         print("Unknown or invalid geometry")
36     # fetch attributes
37     attrs = feature.attributes()
38     # attrs is a list. It contains all the attribute values of this feature
39     print(attrs)
40     # for this test only print the first feature
41     break

```

```

Feature ID: 1
Point:  <QgsPointXY: POINT(7 45)>
[1, 'First feature']

```

6.3 피처 선택

QGIS 데스크탑에서 피처를 여러 가지 방법으로 선택할 수 있습니다: 여러분은 피처 하나를 클릭하거나, 맵 캔버스 위에 직사각형을 그리거나, 또는 표현식 필터를 사용할 수 있습니다. 선택 집합에 여러분의 주의를 끌기 위해, 선택된 피처는 보통 다른 (기본값은 노란색) 색상으로 강조됩니다.

때로는 프로그래밍 방식으로 피처를 선택하거나 또는 기본 색상을 변경하는 편이 유용할 수도 있습니다.

피처를 모두 선택하려면, `selectAll()` 메소드를 사용하면 됩니다:

```
# Get the active layer (must be a vector layer)
layer = iface.activeLayer()
layer.selectAll()
```

표현식을 사용해서 선택하려면, `selectByExpression()` 메소드를 사용하십시오:

```
# Assumes that the active layer is points.shp file from the QGIS test suite
# (Class (string) and Heading (number) are attributes in points.shp)
layer = iface.activeLayer()
layer.selectByExpression('"Class"=\'B52\' and "Heading" > 10 and "Heading" <70',
↳QgsVectorLayer.SetSelection)
```

선택 집합의 색상을 변경하려면 다음 예시에서 볼 수 있는 바와 같이 `QgsMapCanvas` 클래스의 `setSelectionColor()` 메소드를 사용하면 됩니다:

```
iface.mapCanvas().setSelectionColor( QColor("red") )
```

지정한 레이어의 선택 피처 목록에 피처를 추가하려면, 피처 ID 목록에 피처 ID 를 전달하는 `select()` 메소드를 호출하면 됩니다:

```
1 selected_fid = []
2
3 # Get the first feature id from the layer
4 feature = next(layer.getFeatures())
5 if feature:
6     selected_fid.append(feature.id())
7
8 # Add that features to the selected list
9 layer.select(selected_fid)
```

선택 집합을 지우려면:

```
layer.removeSelection()
```

6.3.1 속성에 접근하기

속성은 속성 이름으로 참조할 수 있습니다:

```
print(feature['name'])
```

```
First feature
```

아니면, 인덱스로 속성을 참조할 수 있습니다. 이름을 사용하는 방법보다 좀 더 빠릅니다. 예를 들어 두 번째 속성을 가져오려면:

```
print(feature[1])
```

```
First feature
```

6.3.2 선택한 피처에 대한 작업 반복하기

선택한 피처들만 필요한 경우, 벡터 레이어에서 `selectedFeatures()` 메소드를 사용하면 됩니다:

```
selection = layer.selectedFeatures()
for feature in selection:
    # do whatever you need with the feature
    pass
```

6.3.3 피처 부분 집합에 대한 작업 반복하기

레이어에 있는 피처들 가운데 지정한 부분 집합에만, 예를 들면 지정한 영역 안에 있는 피처에만 작업을 반복하고 싶은 경우, `getFeatures()` 메소드 호출에 `QgsFeatureRequest` 클래스 객체를 추가해야 합니다. 다음은 그 예시입니다:

```
1 areaOfInterest = QgsRectangle(450290,400520, 450750,400780)
2
3 request = QgsFeatureRequest().setFilterRect(areaOfInterest)
4
5 for feature in layer.getFeatures(request):
6     # do whatever you need with the feature
7     pass
```

속도를 향상시키기 위해, 교차 여부를 확인하는 데 피처의 경계 상자만 사용하는 경우가 많습니다. 하지만 교차하는 피처들만 반환되도록 보장하는 `ExactIntersect` 플래그가 존재합니다:

```
request = QgsFeatureRequest().setFilterRect(areaOfInterest) \
    .setFlags(QgsFeatureRequest.ExactIntersect)
```

`setLimit()` 메소드를 사용하면 요청한 피처들의 개수를 제한할 수 있습니다. 다음은 그 예시입니다:

```
request = QgsFeatureRequest()
request.setLimit(2)
for feature in layer.getFeatures(request):
    print(feature)
```

```
<qgis._core.QgsFeature object at 0x7f9b78590948>
<qgis._core.QgsFeature object at 0x7faef5881670>
```

앞의 예시에서 볼 수 있는 것과 같은 공간 필터 대신 (또는 추가로) 속성 기반 필터가 필요한 경우, `QgsExpression` 클래스 객체를 작성해서 `QgsFeatureRequest` 작성자 (constructor) 클래스로 전달하면 됩니다. 다음은 그 예시입니다:

```
# The expression will filter the features where the field "location_name"
# contains the word "Lake" (case insensitive)
exp = QgsExpression('location_name ILIKE \'%Lake%\'' )
request = QgsFeatureRequest(exp)
```

`QgsExpression` 클래스가 지원하는 문법 (syntax) 에 대해 자세히 알고 싶다면 값을 필터링하고 계산하는 표현식을 참조하세요.

요청을 각 피처에 대해 검색된 데이터를 정의하기 위해 사용할 수 있기 때문에, 반복 작업자 (iterator) 가 모든 피처를 반환하긴 하지만 각 피처의 부분적인 데이터를 반환합니다.

```

1 # Only return selected fields to increase the "speed" of the request
2 request.setSubsetOfAttributes([0,2])
3
4 # More user friendly version
5 request.setSubsetOfAttributes(['name','id'],layer.fields())
6
7 # Don't return geometry objects to increase the "speed" of the request
8 request.setFlags(QgsFeatureRequest.NoGeometry)
9
10 # Fetch only the feature with id 45
11 request.setFilterFid(45)
12
13 # The options may be chained
14 request.setFilterRect(areaOfInterest).setFlags(QgsFeatureRequest.NoGeometry).
    ↳setFilterFid(45).setSubsetOfAttributes([0,2])

```

6.4 벡터 레이어 수정하기

벡터 데이터 제공자 대부분은 레이어 데이터의 편집을 지원합니다. 가능한 편집 작업의 일부만 지원하는 경우도 있습니다. 어떤 기능들을 지원하는지 알고 싶다면 `capabilities()` 함수를 사용해 보세요.

```

caps = layer.dataProvider().capabilities()
# Check if a particular capability is supported:
if caps & QgsVectorDataProvider.DeleteFeatures:
    print('The layer supports DeleteFeatures')

```

The layer supports DeleteFeatures

사용할 수 있는 케이퍼빌리티의 전체 목록을 보고 싶다면 `QgsVectorDataProvider` 의 API 문서 클래스를 참조해 주십시오.

레이어의 케이퍼빌리티를 심표로 구분된 설명 텍스트로 출력하려면 다음 예시처럼 `capabilitiesString()` 메소드를 사용하면 됩니다:

```

1 caps_string = layer.dataProvider().capabilitiesString()
2 # Print:
3 # 'Add Features, Delete Features, Change Attribute Values, Add Attributes,
4 # Delete Attributes, Rename Attributes, Fast Access to Features at ID,
5 # Presimplify Geometries, Presimplify Geometries with Validity Check,
6 # Transactions, Curved Geometries'

```

벡터 레이어 편집을 위한 다음 방법들 가운데 어떤 방법을 사용하든, 변경 사항은 기저 데이터 저장소 (파일, 데이터베이스 등등) 에 직접 반영됩니다. 일시적으로만 변경하고자 할 경우, 편집 버퍼로 수정 하는 방법을 설명하는 다음 절로 넘어가십시오.

참고

QGIS 안에서 (콘솔에서든 플러그인에서든) 작업하는 경우, 여러분이 도형의 스타일이나 속성을 변경한 결과를 보기 위해 맵 캔버스를 강제로 다시 그리게 해야 할 필요가 있을 수도 있습니다:

```
1 # If caching is enabled, a simple canvas refresh might not be sufficient
2 # to trigger a redraw and you must clear the cached image for the layer
3 if iface.mapCanvas().isCachingEnabled():
4     layer.triggerRepaint()
5 else:
6     iface.mapCanvas().refresh()
```

6.4.1 피처 추가하기

`QgsFeature` 클래스 인스턴스를 몇 개 생성한 다음 제공자의 `QgsVectorDataProvider` `addFeatures()` 메소드에 그 목록을 전달하십시오. 이 메소드가 결과 (True 또는 False) 와 추가된 피처 목록 (데이터 저장소가 설정한, 추가된 피처의 피처 ID) 이라는 값 2 개를 반환할 것입니다.

피처 속성을 설정하려면, (벡터 레이어의 `fields()` 메소드에서 얻을 수 있는) `QgsFields` 클래스 객체를 전달하는 피처를 초기화하거나, 또는 여러분이 추가하고 싶은 필드들의 개수를 전달하는 `initAttributes()` 메소드를 호출하면 됩니다.

```
1 if caps & QgsVectorDataProvider.AddFeatures:
2     feat = QgsFeature(layer.fields())
3     feat.setAttributes([0, 'hello'])
4     # Or set a single attribute by key or by index:
5     feat.setAttribute('name', 'hello')
6     feat.setAttribute(0, 'hello')
7     feat.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(123, 456)))
8     (res, outFeats) = layer.dataProvider().addFeatures([feat])
```

6.4.2 피처 삭제하기

피처를 몇 개 삭제하려면, 삭제하려는 피처의 ID 목록만 지정하면 됩니다.

```
1 if caps & QgsVectorDataProvider.DeleteFeatures:
2     res = layer.dataProvider().deleteFeatures([5, 10])
```

6.4.3 피처 수정하기

피처의 도형을 변경하거나 아니면 속성을 몇 개 변경할 수 있습니다. 다음 예시에서는 먼저 인덱스가 0 과 1 인 속성들의 값을 변경한 다음, 피처의 도형을 변경합니다.

```
1 fid = 100    # ID of the feature we will modify
2
3 if caps & QgsVectorDataProvider.ChangeAttributeValues:
4     attrs = { 0 : "hello", 1 : 123 }
5     layer.dataProvider().changeAttributeValues({ fid : attrs })
6
7 if caps & QgsVectorDataProvider.ChangeGeometries:
8     geom = QgsGeometry.fromPointXY(QgsPointXY(111,222))
9     layer.dataProvider().changeGeometryValues({ fid : geom })
```

 팁

도형만 편집하는 경우 `QgsVectorLayerEditUtils` 클래스 권장

도형만 변경해야 하는 경우, 도형을 편집하는 몇 가지 유용한 (꼭짓점 변위, 삽입, 이동 등등의) 메소드들을 제공하는 `QgsVectorLayerEditUtils` 클래스를 사용하는 편이 좋을 수도 있습니다.

6.4.4 편집 버퍼로 벡터 레이어 수정하기

QGIS 응용 프로그램 안에서 벡터를 편집할 때, 먼저 특정 레이어에 대한 편집 모드를 시작한 다음 어떤 수정 작업을 하고 마지막으로 변경 사항을 커밋 (또는 롤백) 해야 합니다. 여러분이 무엇을 수정했든, 모든 변경 사항은 커밋하기 전까지는 작성되지 않습니다—레이어의 인메모리 (in-memory) 편집 버퍼에 머물 뿐입니다. 이 기능을 프로그래밍 방식으로 사용할 수 있습니다—그저 데이터 제공자를 직접 사용하는 작업을 보완하는 벡터 레이어 편집의 또 다른 방법일 뿐입니다. 벡터 레이어 편집을 위한 몇몇 GUI 도구들을 제공할 때 이 옵션을 사용하십시오. 여러분이 커밋/롤백할지 여부를 결정할 수 있게 해주고, 실행 취소 (undo)/다시 실행 (redo) 기능을 사용할 수 있게 해주기 때문입니다. 변경 사항을 커밋하면, 데이터 제공자에 편집 버퍼에 있는 변경 사항을 전부 저장합니다.

이 메소드들은 제공자에서 볼 수 있는 메소드들과 비슷하지만, 제공자에서와는 달리 `QgsVectorLayer` 클래스 객체에 대해 호출됩니다.

이 메소드들이 작동하기 위해서는 레이어가 반드시 편집 모드 상태여야만 합니다. 편집 모드를 시작하려면, `startEditing()` 메소드를 사용하십시오. 편집을 끝내려면, `commitChanges()` 또는 `rollback()` 메소드를 사용하십시오. 전자는 데이터 소스에 여러분의 변경 사항을 전부 저장할 것이고, 후자는 변경 사항을 버리고 데이터 소스를 하나도 수정하지 않을 것입니다.

레이어가 편집 모드 상태인지 알아내려면, `isEditable()` 메소드를 사용하십시오.

다음은 이 편집 메소드들을 사용하는 방법을 보여주는 몇몇 예시입니다.

```
1 from qgis.PyQt.QtCore import QMetaType
2
3 feat1 = feat2 = QgsFeature(layer.fields())
4 fid = 99
5 feat1.setId(fid)
6
7 # add two features (QgsFeature instances)
8 layer.addFeatures([feat1, feat2])
9 # delete a feature with specified ID
10 layer.deleteFeature(fid)
11
12 # set new geometry (QgsGeometry instance) for a feature
13 geometry = QgsGeometry.fromWkt("POINT(7 45)")
14 layer.changeGeometry(fid, geometry)
15 # update an attribute with given field index (int) to a given value
16 fieldIndex = 1
17 value = 'My new name'
18 layer.changeAttributeValue(fid, fieldIndex, value)
19
20 # add new field
21 layer.addAttribute(QgsField("mytext", QMetaType.Type.QString))
22 # remove a field
23 layer.deleteAttribute(fieldIndex)
```

실행 취소/다시 실행이 제대로 작동하게 하려면, 앞에서 설명한 호출을 실행 취소 명령어 안에 래핑 (wrapping) 시켜야 합니다. (실행 취소/다시 실행에는 관심이 없고 변경하는 즉시 저장되기를 바라는 경우, 데이터 제공자로 편집하기 를 통해 더 쉽게 작업하게 될 것입니다.)

다음은 실행 취소 기능을 어떻게 사용할 수 있는지 보여주는 예시입니다:

```

1 layer.beginEditCommand("Feature triangulation")
2
3 # ... call layer's editing methods ...
4
5 if problem_occurred:
6     layer.destroyEditCommand()
7     # ... tell the user that there was a problem
8     # and return
9
10 # ... more editing ...
11
12 layer.endEditCommand()

```

`beginEditCommand()` 메소드가 내부 “활성” 명령어를 생성한 다음 벡터 레이어의 순차적인 변경 사항들을 기록할 것입니다. `endEditCommand()` 메소드를 호출하면 명령어를 실행 취소 스택으로 넘겨서 여러분이 GUI에서 실행 취소/다시 실행할 수 있게 될 것입니다. 벡터를 변경하는 도중 무언가 문제가 발생한 경우, `destroyEditCommand()` 메소드가 명령어를 제거하고 해당 명령어가 활성 상태인 동안 발생한 모든 변경 사항들을 롤백시킬 것입니다.

또한 다음 예시에서 볼 수 있는 바와 같이 `with edit(layer)` 선언문을 사용해서 커밋과 롤백을 좀 더 의미를 알 수 있는 (semantic) 코드 블록으로 래핑시킬 수 있습니다:

```

with edit(layer):
    feat = next(layer.getFeatures())
    feat[0] = 5
    layer.updateFeature(feat)

```

이 코드는 마지막에 `commitChanges()` 메소드를 자동 호출할 것입니다. 어떤 예외가 발생하는 경우, 이 코드는 모든 변경 사항을 롤백시키는 `rollback()` 메소드를 호출할 것입니다. `commitChanges()` 메소드가 실행되는 동안 문제가 발생하는 경우 (이 메소드가 거짓을 반환하는 경우) `QgsEditError` 예외가 발생할 것입니다.

6.4.5 필드 추가하기 및 제거하기

필드 (속성) 를 추가하려면, 필드를 정의한 리스트를 지정해야 합니다. 필드를 삭제하려면 필드 인덱스 목록만 넘겨주면 됩니다.

```

1 from qgis.PyQt.QtCore import QMetaType
2
3 if caps & QgsVectorDataProvider.AddAttributes:
4     res = layer.dataProvider().addAttributes(
5         [QgsField("mytext", QMetaType.Type.QString),
6          QgsField("myint", QMetaType.Type.Int)])
7
8 if caps & QgsVectorDataProvider.DeleteAttributes:
9     res = layer.dataProvider().deleteAttributes([0])

```



```

1 # Alternate methods for removing fields
2 # first create temporary fields to be removed (f1-3)
3 layer.dataProvider().addAttributes([QgsField("f1", QMetaType.Type.Int),
4                                     QgsField("f2", QMetaType.Type.Int),
5                                     QgsField("f3", QMetaType.Type.Int)])
6 layer.updateFields()
7 count=layer.fields().count() # count of layer fields
8 ind_list=list((count-3, count-2)) # create list

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

9
10 # remove a single field with an index
11 layer.dataProvider().deleteAttributes([count-1])
12
13 # remove multiple fields with a list of indices
14 layer.dataProvider().deleteAttributes(ind_list)

```

데이터 제공자에서 필드를 추가 또는 제거한 다음 레이어의 필드를 업데이트해야 합니다. 변경 사항이 자동적으로 반영되지 않기 때문입니다.

```
layer.updateFields()
```

💡 팁

with 기반 명령어를 사용해서 변경 사항을 직접 저장하기

with edit(layer): 선언문을 사용하면 마지막에 commitChanges() 메소드를 자동으로 호출해서 변경 사항을 커밋시킬 것입니다. 어떤 예외가 발생하는 경우, rollBack() 메소드를 호출해서 모든 변경 사항을 롤백시킬 것입니다. 편집 버퍼로 벡터 레이어 수정하기 를 참조하세요.

6.5 공간 인덱스 사용하기

벡터 레이어에 대해 자주 쿼리를 해야 할 경우 공간 인덱스를 사용하면 코드 실행 속도를 획기적으로 향상시킬 수 있습니다. 여러분이 보간 알고리즘을 작성하는데, 보간 값을 계산하기 위해 지정된 위치에서 가장 가까운 포인트 레이어의 포인트 10 개를 알아내야 한다고 한번 상상해보십시오. 공간 인덱스가 없다면, QGIS가 그 포인트 10 개를 찾는 유일한 방법은 해당 위치에서 모든 포인트까지의 거리를 각각 계산한 다음 그 거리들을 비교하는 것입니다. 특히 이 작업을 몇 군데의 위치에 대해 반복해야 할 경우 시간이 아주 오래 걸릴 수 있습니다. 레이어가 공간 인덱스를 가지고 있다면, 훨씬 효율적으로 작업할 수 있습니다.

공간 인덱스가 없는 레이어를 전화번호가 정렬되지도 색인되지도 않은 전화번호부라고 생각해보십시오. 어떤 사람의 전화번호를 찾으려면 처음부터 그 번호를 찾을 때까지 읽을 수밖에 없습니다.

QGIS 벡터 레이어에 공간 인덱스를 기본으로 생성하지는 않지만, 쉽게 생성할 수 있습니다. 여러분이 해야 할 일은 다음과 같습니다:

- QgsSpatialIndex 클래스를 사용해서 공간 인덱스를 생성합니다:

```
index = QgsSpatialIndex()
```

- 인덱스에 피처를 추가합니다—인덱스는 QgsFeature 클래스 객체를 받아 내부 데이터 구조에 추가합니다. 여러분이 객체를 직접 생성할 수도 있고, 또는 제공자의 getFeatures() 메소드를 호출했을 때 받은 객체를 사용할 수도 있습니다.

```
index.addFeature(feats)
```

- 아니면 벌크 불러오기를 사용해서 레이어의 피처들을 한번에 전부 불러올 수 있습니다.

```
index = QgsSpatialIndex(layer.getFeatures())
```

- 공간 인덱스가 값들로 채워지고 나면 쿼리를 해볼 수 있습니다.


```

1 # returns array of feature IDs of five nearest features
2 nearest = index.nearestNeighbor(QgsPointXY(25.4, 12.7), 5)
3
4 # returns array of IDs of features which intersect the rectangle
5 intersect = index.intersects(QgsRectangle(22.5, 15.3, 23.1, 17.2))

```

`QgsSpatialIndexKDBush` 클래스 공간 인덱스도 사용할 수 있습니다. 이 인덱스는 표준 `QgsSpatialIndex` 클래스 인덱스와 비슷하지만:

- 단일 포인트 피처 만 지원합니다.
- 정적입니다. (인덱스 작성 후에 인덱스에 어떤 피처도 추가할 수 없습니다.)
- 훨씬 빠릅니다!
- 추가적인 피처 요청 없이도 원본 피처의 포인트들을 직접 검색할 수 있습니다.
- 예를 들어 검색 포인트로부터 어떤 반경 안에 들어오는 모든 포인트를 반환받는다거나 하는, 진정한 거리 기반 검색을 지원합니다.

6.6 QgsVectorLayerUtils 클래스

`QgsVectorLayerUtils` 클래스는 벡터 레이어에 사용할 수 있는 몇몇 매우 유용한 메소드들을 담고 있습니다.

예를 들면 `createFeature()` 메소드는 벡터 레이어에 추가될, 레이어의 각 필드의 모든 최종 제약조건과 기본값을 유지하는 `QgsFeature` 클래스를 준비합니다:

```

vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports layer
↪", "ogr")
feat = QgsVectorLayerUtils.createFeature(vlayer)

```

`getValues()` 메소드는 필드 또는 표현식의 값을 빠르게 얻을 수 있게 해줍니다:

```

1 vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports layer
↪", "ogr")
2 # select only the first feature to make the output shorter
3 vlayer.selectByIds([1])
4 val = QgsVectorLayerUtils.getValues(vlayer, "NAME", selectedOnly=True)
5 print(val)

```

```
(['Sahnewal'], True)
```

6.7 벡터 레이어 생성하기

몇 가지 방법으로 벡터 레이어 데이터셋을 생성할 수 있습니다:

- `QgsVectorFileWriter` 클래스: 디스크에 벡터 파일을 작성할 수 있는 편리한 클래스입니다. 전체 벡터 레이어를 저장하는 `writeAsVectorFormatV3()` 메소드를 정적 호출하거나, 클래스의 인스턴스를 생성한 다음 상속한 `addFeature()` 메소드를 호출합니다. 이 클래스는 GDAL 이 지원하는 벡터 포맷들을 모두 (GeoPackage, 셰이프파일, GeoJSON, KML 및 기타 등등) 지원합니다.
- `QgsVectorLayer` 클래스: 데이터에 연결하고 접근하기 위한 데이터 소스의 지정 경로 (URL) 을 해석하는 데이터 제공자를 인스턴스화합니다. 이 클래스를 사용하면 메모리 기반 임시 레이어 (memory) 를 생성하고 GDAL 벡터 데이터셋 (ogr), 데이터베이스 (postgres, spatialite, mysql, mssql), 그리고 기타 등등 (wfs, gpx, delimitedtext, ...) 에 연결할 수 있습니다.

6.7.1 QgsVectorFileWriter 클래스의 인스턴스로부터

```

1  # SaveVectorOptions contains many settings for the writer process
2  save_options = QgsVectorFileWriter.SaveVectorOptions()
3  transform_context = QgsProject.instance().transformContext()
4  # Write to a GeoPackage (default)
5  error = QgsVectorFileWriter.writeAsVectorFormatV3(layer,
6                                                    "testdata/my_new_file.gpkg",
7                                                    transform_context,
8                                                    save_options)
9  if error[0] == QgsVectorFileWriter.NoError:
10     print("success!")
11  else:
12     print(error)

```

```

1  # Write to an ESRI Shapefile format dataset using UTF-8 text encoding
2  save_options = QgsVectorFileWriter.SaveVectorOptions()
3  save_options.driverName = "ESRI Shapefile"
4  save_options.fileEncoding = "UTF-8"
5  transform_context = QgsProject.instance().transformContext()
6  error = QgsVectorFileWriter.writeAsVectorFormatV3(layer,
7                                                    "testdata/my_new_shapefile",
8                                                    transform_context,
9                                                    save_options)
10  if error[0] == QgsVectorFileWriter.NoError:
11     print("success again!")
12  else:
13     print(error)

```

```

1  # Write to an ESRI GDB file
2  save_options = QgsVectorFileWriter.SaveVectorOptions()
3  save_options.driverName = "FileGDB"
4  # if no geometry
5  save_options.overrideGeometryType = QgsWkbTypes.Unknown
6  save_options.actionOnExistingFile = QgsVectorFileWriter.CreateOrOverwriteLayer
7  save_options.layerName = 'my_new_layer_name'
8  transform_context = QgsProject.instance().transformContext()
9  gdb_path = "testdata/my_example.gdb"
10  error = QgsVectorFileWriter.writeAsVectorFormatV3(layer,
11                                                    gdb_path,
12                                                    transform_context,
13                                                    save_options)
14  if error[0] == QgsVectorFileWriter.NoError:
15     print("success!")
16  else:
17     print(error)

```

또한 `FieldValueConverter` 클래스를 사용하면 필드들을 서로 다른 포맷들과 호환되도록 변환할 수 있습니다. 예를 들어 (PostgreSQL 에서) 배열 변수 유형을 텍스트 유형으로 변환하려면 다음과 같이 하면 됩니다:

```

1  LIST_FIELD_NAME = 'xxxx'
2
3  class ESRIValueConverter(QgsVectorFileWriter.FieldValueConverter):
4
5      def __init__(self, layer, list_field):
6          QgsVectorFileWriter.FieldValueConverter.__init__(self)
7          self.layer = layer

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

8     self.list_field_idx = self.layer.fields().indexOfName(list_field)
9
10    def convert(self, fieldIdxInLayer, value):
11        if fieldIdxInLayer == self.list_field_idx:
12            return QgsListFieldFormatter().representValue(layer=vlayer,
13                                                         fieldIndex=self.list_field_idx,
14                                                         config={},
15                                                         cache=None,
16                                                         value=value)
17        else:
18            return value
19
20    def fieldDefinition(self, field):
21        idx = self.layer.fields().indexOfName(field.name())
22        if idx == self.list_field_idx:
23            return QgsField(LIST_FIELD_NAME, QMetaType.Type.QString)
24        else:
25            return self.layer.fields()[idx]
26
27    converter = ESRIValueConverter(vlayer, LIST_FIELD_NAME)
28    opts = QgsVectorFileWriter.SaveVectorOptions()
29    opts.fieldValueConverter = converter

```

대상 좌표를 지정할 수도 있습니다—`QgsCoordinateReferenceSystem` 클래스의 무결한 인스턴스를 네 번째 파라미터로 전달하면, 레이어가 해당 좌표계로 변환됩니다.

무결한 드라이버 이름을 알고 싶다면 `supportedFiltersAndFormats()` 메소드를 호출하거나 `supported formats by OGR` (OGR 가 지원하는 포맷들) 문서를 살펴보십시오—“Code” 열에 이 값을 드라이버 이름으로 전달해야 할 것입니다.

선택한 피쳐들만 내보낼지, 생성 용 드라이버 전용 심화 옵션들을 전달할지, 작성기에 속성을 생성하지 말라고 할지 등등의 여부를 선택적으로 설정할 수 있습니다. 기타 (선택적) 파라미터들이 여럿 있습니다. 자세한 내용은 `QgsVectorFileWriter` 클래스 문서를 참조하세요.

6.7.2 피쳐로부터 직접

```

1  from qgis.PyQt.QtCore import QMetaType
2
3  # define fields for feature attributes. A QgsFields object is needed
4  fields = QgsFields()
5  fields.append(QgsField("first", QMetaType.Type.Int))
6  fields.append(QgsField("second", QMetaType.Type.QString))
7
8  """ create an instance of vector file writer, which will create the vector file.
9  Arguments:
10  1. path to new file (will fail if exists already)
11  2. field map
12  3. geometry type - from WKBTYP enum
13  4. layer's spatial reference (instance of
14     QgsCoordinateReferenceSystem)
15  5. coordinate transform context
16  6. save options (driver name for the output file, encoding etc.)
17  """
18
19  crs = QgsProject.instance().crs()

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

20 transform_context = QgsProject.instance().transformContext()
21 save_options = QgsVectorFileWriter.SaveVectorOptions()
22 save_options.driverName = "ESRI Shapefile"
23 save_options.fileEncoding = "UTF-8"
24
25 writer = QgsVectorFileWriter.create(
26     "testdata/my_new_shapefile.shp",
27     fields,
28     QgsWkbTypes.Point,
29     crs,
30     transform_context,
31     save_options
32 )
33
34 if writer.hasError() != QgsVectorFileWriter.NoError:
35     print("Error when creating shapefile: ", writer.errorMessage())
36
37 # add a feature
38 fet = QgsFeature()
39
40 fet.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(10,10)))
41 fet.setAttributes([1, "text"])
42 writer.addFeature(fet)
43
44 # delete the writer to flush features to disk
45 del writer

```

6.7.3 QgsVectorLayer 클래스 인스턴스로부터

`QgsVectorLayer` 클래스가 지원하는 모든 데이터 제공자들 가운데, 메모리 기반 레이어를 주목해봅시다. 메모리 제공자는 주로 플러그인이나 제 3 자 응용 프로그램 개발자들이 사용할 목적으로 설계되었습니다. 메모리 제공자는 디스크 상에 데이터를 저장하지 않기 때문에, 개발자들이 몇몇 임시 레이어에 대한 빠른 백엔드로 사용할 수 있기 때문입니다.

이 제공자는 문자열 (string), 정수 (int), 더블형 실수 (double) 유형의 필드를 지원합니다.

메모리 제공자는 공간 인덱스 작업도 지원하는데, 제공자의 `createSpatialIndex()` 함수를 호출하면 됩니다. 공간 인덱스를 생성하고 나면 좁은 지역 안에 있는 피쳐들을 더 빠르게 반복 작업할 수 있게 될 것입니다. (피쳐들을 전부 처리할 필요 없이 지정한 직사각형 안에 있는 피쳐들만 처리하면 되기 때문입니다.)

`QgsVectorLayer` 클래스 작성자 (constructor) 에 "memory" 를 제공자 문자열로 전달하면 메모리 제공자가 생성됩니다.

이 작성자는 레이어의 도형 유형을 정의하는 URI 도 받습니다. 도형 유형은 다음 가운데 하나일 수 있습니다: "Point", "LineString", "Polygon", "MultiPoint", "MultiLineString", "MultiPolygon" or "None"

좌표계, 필드, 메모리 기반 제공자의 인덱스 작업도 이 URI 로 설정할 수 있습니다. 문법은 다음과 같습니다:

crs=definition

좌표계를 지정합니다. 이때 좌표계 정의는 `QgsCoordinateReferenceSystem.createFromString()` 메소드가 받아들이는 어떤 양식도 될 수 있습니다.

index=yes

제공자가 공간 인덱스를 사용하도록 설정합니다.

field=name:type(length,precision)

레이어의 속성을 설정합니다. 속성의 명칭은 필수적이며, 선택적으로 유형 (정수, 더블형 실수, 문자 스트링), 길이 및 정밀도를 설정할 수 있습니다. 여러 개의 필드를 정의할 수도 있습니다.

다음은 이 모든 옵션들을 포함하는 URI 의 예시입니다.

```
"Point?crs=epsg:4326&field=id:integer&field=name:string(20)&index=yes"
```

다음은 메모리 기반 제공자를 생성하고 값을 채우는 코드의 예시입니다.

```
1 from qgis.PyQt.QtCore import QDateTime, QMetaType, Qt
2
3 # create layer
4 vl = QgsVectorLayer("Point", "temporary_points", "memory")
5 pr = vl.dataProvider()
6
7 # add fields
8 pr.addAttributes([QgsField("name", QMetaType.Type.QString),
9                     QgsField("age", QMetaType.Type.Int),
10                    QgsField("size", QMetaType.Type.Double),
11                    QgsField("birthday", QMetaType.Type.QDateTime)])
12 vl.updateFields() # tell the vector layer to fetch changes from the provider
13
14 # add a feature
15 fet = QgsFeature()
16 fet.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(10,10)))
17 fmt = Qt.DateFormat.ISODate
18 t = QDateTime.fromString("2000-01-01T12:00:00", fmt)
19 fet.setAttributes(["Johnny", 2, 0.3, t])
20 pr.addFeatures([fet])
21
22 # update layer's extent when new features have been added
23 # because change of extent in provider is not propagated to the layer
24 vl.updateExtents()
```

마지막으로, 모든 작업이 성공적이었는지 확인해봅시다.

```
1 # show some stats
2 print("fields:", len(pr.fields()))
3 print("features:", pr.featureCount())
4 e = vl.extent()
5 print("extent:", e.xMinimum(), e.yMinimum(), e.xMaximum(), e.yMaximum())
6
7 # iterate over features
8 features = vl.getFeatures()
9 for fet in features:
10     print("F:", fet.id(), fet.attributes(), fet.geometry().asPoint())
```

```
fields: 4
features: 1
extent: 10.0 10.0 10.0 10.0
F: 1 ['Johnny', 2, 0.3, PyQt5.QtCore.QDateTime(2000, 1, 1, 12, 0)] <QgsPointXY:
↳ POINT(10 10)>
```

6.8 벡터 레이어의 표현 (심볼)

벡터 레이어를 렌더링할 때, 레이어와 관련된 렌더링 작업자 와 심볼 이 데이터의 표현을 결정합니다. 심볼은 피처의 시각적 표현을 그리는 일을 담당하는 클래스이며, 렌더링 작업자는 특정 피처에 대해 어떤 심볼을 적용할지 결정합니다.

지정한 레이어의 렌더링 작업자는 다음과 같이 얻을 수 있습니다:

```
renderer = layer.renderer()
```

그리고 이를 참조해서 조금 더 나가봅시다.

```
print("Type:", renderer.type())
```

```
Type: singleSymbol
```

QGIS 핵심 라이브러리에서 알려진 몇몇 렌더링 작업자 유형을 사용할 수 있습니다:

유형	클래스	설명
singleSymbol	<code>QgsSingleSymbolRenderer</code>	모든 피처를 동일한 심볼로 렌더링합니다.
categorizedSymbol	<code>QgsCategorizedSymbolRenderer</code>	피처를 각 카테고리 별로 서로 다른 심볼로 렌더링합니다.
graduatedSymbol	<code>QgsGraduatedSymbolRenderer</code>	피처를 값의 범위에 따라 서로 다른 심볼로 렌더링합니다.

몇몇 사용자 정의 렌더링 작업자 유형이 있을 수도 있기 때문에, 이 유형들만 있을 거라고 절대로 가정하지 마십시오. 현재 사용할 수 있는 렌더링 작업자들을 알아내려면 응용 프로그램의 `QgsRendererRegistry` 클래스를 쿼리하면 됩니다:

```
print(QgsApplication.rendererRegistry().renderersList())
```

```
['nullSymbol', 'singleSymbol', 'categorizedSymbol', 'graduatedSymbol', 'RuleRenderer',  
→ 'pointDisplacement', 'pointCluster', 'mergedFeatureRenderer',  
→ 'invertedPolygonRenderer', 'heatmapRenderer', '25dRenderer', 'embeddedSymbol']
```

렌더링 작업자의 콘텐츠를 텍스트 형식으로 덤프받을 수도 있습니다—디버깅 작업에 유용할 수 있습니다.

```
renderer.dump()
```

```
SINGLE: MARKER SYMBOL (1 layers) color 190,207,80,255
```

6.8.1 단일 심볼 렌더링 작업자

`symbol()` 메소드를 호출하면 렌더링에 쓰인 심볼을 가져올 수 있고, `setSymbol()` 메소드로 심볼을 변경할 수 있습니다. (C++ 개발자는 기억하세요: 렌더링 작업자가 심볼을 소유합니다.)

`setSymbol()` 메소드를 호출해서 적당한 심볼의 인스턴스를 전달하면 특정 벡터 레이어가 사용하는 심볼을 변경할 수 있습니다. `QgsMarkerSymbol`, `QgsLineSymbol`, 그리고 `QgsFillSymbol` 클래스의 각각 대응하는 `createSimple()` 함수를 호출하면 각각 포인트, 라인, 폴리곤 레이어 용 심볼을 생성할 수 있습니다.

`createSimple()` 메소드에 전달되는 목록 (dictionary) 이 심볼의 스타일 속성을 설정합니다.

예를 들면 다음 코드 예시와 같이 `setSymbol()` 메소드를 호출해서 `QgsMarkerSymbol` 클래스의 인스턴스를 전달하면, 특정 포인트 레이어가 사용하는 심볼을 대체시킬 수 있습니다.

```
symbol = QgsMarkerSymbol.createSimple({'name': 'square', 'color': 'red'})
layer.renderer().setSymbol(symbol)
# show the change
layer.triggerRepaint()
```

`name` 은 마커의 형태를 의미하며, 다음 중 어느 것이라도 가능합니다.

- circle
- square
- cross
- rectangle
- diamond
- pentagon
- triangle
- equilateral_triangle
- star
- regular_star
- arrow
- filled_arrowhead
- x

심볼 인스턴스의 첫 번째 심볼 레이어에 대한 전체 속성 목록을 알고 싶다면 다음 코드 예시를 따라해보십시오:

```
print(layer.renderer().symbol().symbolLayers()[0].properties())
```

```
{'angle': '0', 'cap_style': 'square', 'color': '255,0,0,255,rgb:1,0,0,1', 'horizontal_
↪ anchor_point': '1', 'joinstyle': 'bevel', 'name': 'square', 'offset': '0,0',
↪ 'offset_map_unit_scale': '3x:0,0,0,0,0,0', 'offset_unit': 'MM', 'outline_color':
↪ '35,35,35,255,rgb:0.1372549,0.1372549,0.1372549,1', 'outline_style': 'solid',
↪ 'outline_width': '0', 'outline_width_map_unit_scale': '3x:0,0,0,0,0,0', 'outline_
↪ width_unit': 'MM', 'scale_method': 'diameter', 'size': '2', 'size_map_unit_scale':
↪ '3x:0,0,0,0,0,0', 'size_unit': 'MM', 'vertical_anchor_point': '1'}
```

여러분이 속성을 일부 변경하고 싶다면 다음이 유용할 것입니다:

```
1 # You can alter a single property...
2 layer.renderer().symbol().symbolLayer(0).setSize(3)
3 # ... but not all properties are accessible from methods,
4 # you can also replace the symbol completely:
5 props = layer.renderer().symbol().symbolLayer(0).properties()
6 props['color'] = 'yellow'
7 props['name'] = 'square'
8 layer.renderer().setSymbol(QgsMarkerSymbol.createSimple(props))
9 # show the changes
10 layer.triggerRepaint()
```

6.8.2 카테고리 심볼 렌더링 작업자

카테고리 렌더링 작업자를 사용할 때, 범주화에 사용되는 속성을 쿼리하고 설정할 수 있습니다: `classAttribute()` 및 `setClassAttribute()` 메소드를 사용하십시오.

카테고리 목록을 얻는 방법은 다음과 같습니다.

```
1 categorized_renderer = QgsCategorizedSymbolRenderer()
2 # Add a few categories
3 cat1 = QgsRendererCategory('1', QgsMarkerSymbol(), 'category 1')
4 cat2 = QgsRendererCategory('2', QgsMarkerSymbol(), 'category 2')
5 categorized_renderer.addCategory(cat1)
6 categorized_renderer.addCategory(cat2)
7
8 for cat in categorized_renderer.categories():
9     print("{}: {} :: {}".format(cat.value(), cat.label(), cat.symbol()))
```

```
1: category 1 :: <qgis._core.QgsMarkerSymbol object at 0x7f378ffcd9d8>
2: category 2 :: <qgis._core.QgsMarkerSymbol object at 0x7f378ffcd9d8>
```

이때 `value()` 메소드가 카테고리들을 구분하는 데 쓰이는 값이고, `label()` 메소드는 카테고리 설명에 쓰이는 텍스트이며, `symbol()` 메소드는 할당된 심볼을 반환합니다.

이 렌더링 작업자는 일반적으로 범주화에 사용된 원본 심볼과 색상표도 저장합니다: `sourceColorRamp()` 및 `sourceSymbol()` 메소드를 사용하십시오.

6.8.3 등급 심볼 렌더링 작업자

이 렌더링 작업자는 앞에서 설명한 카테고리 심볼 렌더링 작업자와 매우 비슷하지만, 범주 당 하나의 속성을 할당하는 대신 값의 범위에 따라 할당하기 때문에 숫자 속성에만 사용할 수 있습니다.

이 렌더링 작업자가 사용하는 범위에 대해 상세히 알아내려면 다음과 같은 방법을 사용합니다.

```
1 graduated_renderer = QgsGraduatedSymbolRenderer()
2 # Add a few categories
3 graduated_renderer.addClassRange(QgsRendererRange(QgsClassificationRange('class 0-100
4 ↳', 0, 100), QgsMarkerSymbol()))
5 graduated_renderer.addClassRange(QgsRendererRange(QgsClassificationRange('class 101-
6 ↳200', 101, 200), QgsMarkerSymbol()))
7
8 for ran in graduated_renderer.ranges():
9     print("{} - {}: {} {}".format(
10         ran.lowerValue(),
11         ran.upperValue(),
12         ran.label(),
13         ran.symbol()
14     ))
```

```
0.0 - 100.0: class 0-100 <qgis._core.QgsMarkerSymbol object at 0x7f8bad281b88>
101.0 - 200.0: class 101-200 <qgis._core.QgsMarkerSymbol object at 0x7f8bad281b88>
```

(범주화 속성 이름을 알아내기 위한) `classAttribute()` 메소드와 `sourceSymbol()` 및 `sourceColorRamp()` 메소드를 다시 사용하면 됩니다. 범위 생성 방법을 결정하는 추가적인 `mode()` 메소드도 있습니다: 등간격을 사용할지, 사분위를 사용할지, 또는 기타 다른 방법을 사용할지 결정할 수 있습니다.

사용자 정의 등급 심볼 렌더링 작업자를 생성하고자 할 경우 다음 예시에서처럼 하면 됩니다. (이 예시에서는 범주 2 개인 간단한 배열을 생성합니다.)


```

1 from qgis.PyQt import QtGui
2
3 myVectorLayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports",
4     ↳ "Airports layer", "ogr")
5 myTargetField = 'scalerank'
6 myRangeList = []
7 myOpacity = 1
8 # Make our first symbol and range...
9 myMin = 0.0
10 myMax = 50.0
11 myLabel = 'Group 1'
12 myColour = QtGui.QColor('#ffee00')
13 mySymbol1 = QgsSymbol.defaultSymbol(myVectorLayer.geometryType())
14 mySymbol1.setColor(myColour)
15 mySymbol1.setOpacity(myOpacity)
16 myRange1 = QgsRendererRange(myMin, myMax, mySymbol1, myLabel)
17 myRangeList.append(myRange1)
18 #now make another symbol and range...
19 myMin = 50.1
20 myMax = 100
21 myLabel = 'Group 2'
22 myColour = QtGui.QColor('#00eeff')
23 mySymbol2 = QgsSymbol.defaultSymbol(
24     myVectorLayer.geometryType())
25 mySymbol2.setColor(myColour)
26 mySymbol2.setOpacity(myOpacity)
27 myRange2 = QgsRendererRange(myMin, myMax, mySymbol2, myLabel)
28 myRangeList.append(myRange2)
29 myRenderer = QgsGraduatedSymbolRenderer('', myRangeList)
30 myClassificationMethod = QgsApplication.classificationMethodRegistry().method(
31     ↳ "EqualInterval")
32 myRenderer.setClassificationMethod(myClassificationMethod)
33 myRenderer.setClassAttribute(myTargetField)
34
35 myVectorLayer.setRenderer(myRenderer)

```

6.8.4 심볼 다루기

심볼을 표현하기 위한 `QgsSymbol` 기저 (base) 클래스는 파생 클래스 3 개를 가지고 있습니다:

- `QgsMarkerSymbol` —포인트 피쳐 용
- `QgsLineSymbol` —라인 피쳐 용
- `QgsFillSymbol` —폴리곤 피쳐 용

모든 심볼은 하나 이상의 (`QgsSymbolLayer` 클래스에서 파생된 클래스인) 심볼 레이어로 이루어집니다. 이 심볼 레이어들이 실제 렌더링 작업을 하며, 심볼 클래스 자체는 심볼 레이어 용 컨테이너 역할만 합니다.

(예를 들어 렌더링 작업자로부터 나온) 심볼 인스턴스를 가지고 있다면, 심볼을 탐색해볼 수 있습니다: `type()` 메소드는 심볼이 마커, 라인, 또는 채우기 심볼 가운데 어떤 유형인지 알려줍니다. 심볼의 짧은 설명을 반환하는 `dump()` 메소드도 있습니다. 심볼 레이어 목록을 보고 싶다면:

```

marker_symbol = QgsMarkerSymbol()
for i in range(marker_symbol.symbolLayerCount()):
    lyr = marker_symbol.symbolLayer(i)
    print("{}: {}".format(i, lyr.layerType()))

```

0: SimpleMarker

심볼의 색상을 알아내려면 `color()` 메소드를 사용하고 그 색상을 변경하려면 `setColor()` 메소드를 사용하십시오. 마커 심볼의 경우 추가적으로 `size()` 와 `angle()` 메소드를 사용해서 각각 심볼 크기와 회전 각도를 쿼리할 수 있습니다. 라인 심볼의 경우 `width()` 메소드가 라인 너비를 반환합니다.

기본적으로 크기 및 너비는 밀리미터 단위이며, 각도는 도 단위입니다.

심볼 레이어 다루기

앞에서 말했듯이, (`QgsSymbolLayer` 클래스의 하위 클래스인) 심볼 레이어가 피처의 모양을 결정합니다. 일반적인 사용례를 위한 기본 심볼 레이어 클래스가 몇 개 있습니다. 새로운 심볼 레이어 유형을 구현할 수 있기 때문에, 사용자가 피처를 어떻게 렌더링할지를 임의로 정의할 수 있습니다. `layerType()` 메소드가 심볼 레이어 클래스를 고유하게 식별합니다—기본 클래스는 `SimpleMarker`, `SimpleLine` 및 `SimpleFill` 심볼 레이어 유형입니다.

지정한 심볼 레이어 클래스에 생성할 수 있는 심볼 레이어 유형들의 전체 리스트를 다음 코드로 볼 수 있습니다:

```
1 from qgis.core import QgsSymbolLayerRegistry
2 myRegistry = QgsApplication.symbolLayerRegistry()
3 myMetadata = myRegistry.symbolLayerMetadata("SimpleFill")
4 for item in myRegistry.symbolLayersForType(QgsSymbol.Marker):
5     print(item)
```

```
1 AnimatedMarker
2 EllipseMarker
3 FilledMarker
4 FontMarker
5 GeometryGenerator
6 MaskMarker
7 RasterMarker
8 SimpleMarker
9 SvgMarker
10 VectorField
```

`QgsSymbolLayerRegistry` 클래스는 사용할 수 있는 모든 레이어 유형들의 데이터베이스를 관리합니다.

심볼 레이어 데이터에 접근하려면, 심볼 모양을 결정하는 속성들의 키-값 목록 (dictionary) 을 반환하는 `properties()` 메소드를 사용하십시오. 각각의 심볼 레이어 유형은 해당 유형이 사용하는 특정한 속성 집합을 가지고 있습니다. 또한 각각의 설정자 (setter) 대응 항목을 가진 일반적인 `color()`, `size()`, `angle()` 및 `width()` 메소드들도 있습니다. 물론 크기와 각도는 마커 심볼에만 사용할 수 있으며 너비는 라인 심볼에만 사용할 수 있습니다.

사용자 지정 심볼 레이어 유형 생성하기

데이터를 어떻게 렌더링할지 사용자 정의하고 싶다고 상상해보십시오. 사용자가 원하는 방식대로 피처를 그리는, 자신만의 심볼 레이어 클래스를 생성할 수 있습니다. 다음 코드는 지정된 반경으로 빨간색 원을 그리는 마커의 예시입니다:

```
1 from qgis.core import QgsMarkerSymbolLayer
2 from qgis.PyQt.QtGui import QColor
3
4 class FooSymbolLayer(QgsMarkerSymbolLayer):
5
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

6  def __init__(self, radius=4.0):
7      QgsMarkerSymbolLayer.__init__(self)
8      self.radius = radius
9      self.color = QColor(255,0,0)
10
11  def layerType(self):
12      return "FooMarker"
13
14  def properties(self):
15      return { "radius" : str(self.radius) }
16
17  def startRender(self, context):
18      pass
19
20  def stopRender(self, context):
21      pass
22
23  def renderPoint(self, point, context):
24      # Rendering depends on whether the symbol is selected (QGIS >= 1.5)
25      color = context.selectionColor() if context.selected() else self.color
26      p = context.renderContext().painter()
27      p.setPen(color)
28      p.drawEllipse(point, self.radius, self.radius)
29
30  def clone(self):
31      return FooSymbolLayer(self.radius)

```

`layerType()` 메소드가 심볼 레이어의 이름을 결정합니다. 이 이름은 전체 심볼 레이어들 사이에서 유일해야 합니다. `properties()` 메소드는 속성들의 지속성을 위해 쓰입니다. `clone()` 메소드는 반드시 심볼 레이어와 모든 속성이 정확하게 동일한 복제물을 반환해야만 합니다. 마지막으로 렌더링 메소드들이 있습니다: 첫 번째 피처를 렌더링하기 전에 `startRender()` 메소드를 호출하고, 렌더링을 완료했을 때 `stopRender()` 메소드를 호출하며, 렌더링을 하기 위해서는 `renderPoint()` 메소드를 호출합니다. 포인트 (들) 의 좌표는 이미 산출 좌표로 변환되어 있습니다.

폴리라인 및 폴리곤의 경우 유일한 차이점은 렌더링 메소드에 있을 것입니다: 라인의 경우 라인 목록을 받는 `renderPolyline()` 메소드를 사용하는 반면, 폴리곤의 경우 외곽 고리 상에 있는 포인트 목록을 첫 번째 파라미터로 그리고 내곽 고리의 포인트 목록 (또는 None) 을 두 번째 파라미터로 받는 `renderPolygon()` 메소드를 사용할 것입니다.

일반적으로 사용자가 모양을 변경할 수 있도록 심볼 레이어 유형의 속성을 설정하기 위한 GUI 를 추가하는 편이 좋습니다. 앞의 예시에서 사용자가 원의 반경을 설정하도록 할 수 있습니다. 다음 코드는 그런 위젯을 구현하는 예시입니다:

```

1  from qgis.gui import QgsSymbolLayerWidget
2
3  class FooSymbolLayerWidget(QgsSymbolLayerWidget):
4      def __init__(self, parent=None):
5          QgsSymbolLayerWidget.__init__(self, parent)
6
7          self.layer = None
8
9          # setup a simple UI
10         self.label = QLabel("Radius:")
11         self.spinRadius = QDoubleSpinBox()
12         self.hbox = QHBoxLayout()
13         self.hbox.addWidget(self.label)

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

14     self.hbox.addWidget(self.spinRadius)
15     self.setLayout(self.hbox)
16     self.connect(self.spinRadius, SIGNAL("valueChanged(double)"), \
17                 self.radiusChanged)
18
19     def setSymbolLayer(self, layer):
20         if layer.layerType() != "FooMarker":
21             return
22         self.layer = layer
23         self.spinRadius.setValue(layer.radius)
24
25     def symbolLayer(self):
26         return self.layer
27
28     def radiusChanged(self, value):
29         self.layer.radius = value
30         self.emit(SIGNAL("changed()"))

```

심볼 속성 대화창 안에 이 위젯을 내장시킬 수 있습니다. 심볼 속성 대화창에서 심볼 레이어 유형을 선택했을 때, 심볼 레이어의 인스턴스 및 심볼 레이어 위젯의 인스턴스를 생성합니다. 그 다음 `setSymbolLayer()` 메소드를 호출해서 위젯에 심볼 레이어를 할당합니다. 이때 위젯이 심볼 레이어의 속성을 반영하도록 UI 를 업데이트해야 합니다. 심볼 속성 대화창에서 심볼에 사용할 수 있도록 심볼 레이어를 다시 검색하는 데 `symbolLayer()` 메소드를 사용합니다.

속성을 변경할 때마다, 이 위젯은 심볼 속성 대화창이 심볼 미리보기를 업데이트할 수 있도록 `changed()` 신호를 보내야 합니다.

이제 마지막 단계만 남았습니다. QGIS 가 이 새 클래스들을 인식하도록 심볼 레이어를 레지스트리에 추가하는 일입니다. 심볼 레이어를 레지스트리에 추가하지 않고 사용할 수도 있지만, 예를 들어 사용자 정의 심볼 레이어를 담고 있는 프로젝트 파일을 불러오지 못한다든가 GUI 에서 레이어 속성을 편집할 수 없다는 등 몇몇 기능들을 사용할 수 없게 됩니다.

심볼 레이어에 대한 메타데이터도 생성해야 합니다:

```

1  from qgis.core import QgsSymbol, QgsSymbolLayerAbstractMetadata, \
2     ↪QgsSymbolLayerRegistry
3
4  class FooSymbolLayerMetadata(QgsSymbolLayerAbstractMetadata):
5
6      def __init__(self):
7          super().__init__("FooMarker", "My new Foo marker", QgsSymbol.Marker)
8
9      def createSymbolLayer(self, props):
10         radius = float(props["radius"]) if "radius" in props else 4.0
11         return FooSymbolLayer(radius)
12
13  fslmetadata = FooSymbolLayerMetadata()

```

```
QgsApplication.symbolLayerRegistry().addSymbolLayerType(fslmetadata)
```

상위 클래스의 작성자에 (레이어가 반환하는 유형과 동일한) 레이어 유형과 심볼 유형 (마커/라인/채우기) 을 전달해야 합니다. `createSymbolLayer()` 메소드가 props 목록 (dictionary) 에 지정된 속성들을 가진 심볼 레이어의 인스턴스 생성을 처리합니다. 그리고 해당 심볼 레이어 유형을 위한 설정 위젯을 반환하는 `createSymbolLayerWidget()` 메소드도 있습니다.

마지막 단계는 레지스트리에 이 심볼 레이어를 추가하는 일입니다. 이제 모두 끝났습니다.

6.8.5 사용자 정의 렌더링 작업자 생성하기

피처를 렌더링하는 데 어떻게 심볼을 선택할지에 대한 규칙을 마음대로 지정하고 싶을 경우 새로운 렌더링 작업자를 만드는 편이 유용할 수도 있습니다. 여러 필드들을 조합해서 심볼을 결정해야 하거나, 현재 축척에 따라 심볼 크기를 변경해야 하는 등의 경우에 새로운 렌더링 작업자를 만들면 좋습니다.

다음 코드는 마커 심볼 2 개를 생성해서 각 피처마다 랜덤하게 2 개 중 1 개를 할당하는 간단한 사용자 정의 렌더링 작업자의 예시입니다:

```

1 import random
2 from qgis.core import QgsWkbTypes, QgsSymbol, QgsFeatureRenderer
3
4
5 class RandomRenderer(QgsFeatureRenderer):
6     def __init__(self, syms=None):
7         super().__init__("RandomRenderer")
8         self.syms = syms if syms else [
9             QgsSymbol.defaultSymbol(QgsWkbTypes.geometryType(QgsWkbTypes.Point)),
10            QgsSymbol.defaultSymbol(QgsWkbTypes.geometryType(QgsWkbTypes.Point))
11        ]
12
13     def symbolForFeature(self, feature, context):
14         return random.choice(self.syms)
15
16     def startRender(self, context, fields):
17         super().startRender(context, fields)
18         for s in self.syms:
19             s.startRender(context, fields)
20
21     def stopRender(self, context):
22         super().stopRender(context)
23         for s in self.syms:
24             s.stopRender(context)
25
26     def usedAttributes(self, context):
27         return []
28
29     def clone(self):
30         return RandomRenderer(self.syms)

```

상위 `QgsFeatureRenderer` 클래스의 작성자는 (렌더링 작업자들 가운데 유일해야 하는) 렌더링 작업자 이름을 필요로 합니다. `symbolForFeature()` 가 특정 피처에 어떤 심볼을 사용할지 결정하는 메소드입니다. `startRender()` 와 `stopRender()` 메소드는 각각 심볼 렌더링 작업의 시작과 마무리를 처리합니다. `usedAttributes()` 메소드는 렌더링 작업자가 존재할 것으로 예상하는 필드 이름 목록을 반환할 수 있습니다. 마지막으로 `clone()` 메소드는 렌더링 작업자의 복제물을 반환할 것입니다.

심볼 레이어와 마찬가지로, 렌더링 작업자를 환경설정하기 위한 GUI를 추가할 수 있습니다. 이 GUI는 `QgsRendererWidget` 클래스로부터 파생시켜야 합니다. 다음 코드는 여러분이 첫 번째 심볼을 설정할 수 있게 해주는 버튼을 생성하는 예시입니다:

```

1 from qgis.gui import QgsRendererWidget, QgsColorButton
2
3
4 class RandomRendererWidget(QgsRendererWidget):
5     def __init__(self, layer, style, renderer):
6         super().__init__(layer, style)
7         if renderer is None or renderer.type() != "RandomRenderer":

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

8     self.r = RandomRenderer()
9     else:
10        self.r = renderer
11    # setup UI
12    self.btn1 = QgsColorButton()
13    self.btn1.setColor(self.r.syms[0].color())
14    self.vbox = QVBoxLayout()
15    self.vbox.addWidget(self.btn1)
16    self.setLayout(self.vbox)
17    self.btn1.colorChanged.connect(self.setColor1)
18
19    def setColor1(self):
20        color = self.btn1.color()
21        if not color.isValid(): return
22        self.r.syms[0].setColor(color)
23
24    def renderer(self):
25        return self.r

```

이 작성자는 활성 레이어의 인스턴스(`QgsVectorLayer`), 전체 수준 스타일(`QgsStyle`), 그리고 현재 렌더링 작업을 받습니다. 렌더링 작업자가 없거나 렌더링 작업자가 다른 유형인 경우, 새로운 사용자 정의 렌더링 작업자로 대체할 것입니다. 그렇지 않은 경우(이미 필요한 유형인) 현재 렌더링 작업자를 사용할 것입니다. 렌더링 작업자의 현재 상태를 보이도록 위젯의 내용을 업데이트해야 합니다. 렌더링 작업자 대화창이 승인되면, 현재 렌더링 작업자를 가져오기 위해 위젯의 `renderer()` 메소드를 호출합니다—레이어에 가져온 렌더링 작업자를 할당할 것입니다.

마지막으로 렌더링 작업자의 메타데이터를 생성하고 레지스트리에 렌더링 작업자를 등록해야 합니다. 이렇게 하지 않으면 렌더링 작업자와 함께 레이어를 불러올 수 없고, 사용자가 렌더링 작업자 목록에서 렌더링 작업자를 선택할 수 없습니다. 이제 `RandomRenderer` 예시 코드를 완성해봅시다:

```

1 from qgis.core import (
2     QgsRendererAbstractMetadata,
3     QgsRendererRegistry,
4     QgsApplication
5 )
6
7 class RandomRendererMetadata(QgsRendererAbstractMetadata):
8
9     def __init__(self):
10        super().__init__("RandomRenderer", "Random renderer")
11
12    def createRenderer(self, element):
13        return RandomRenderer()
14
15    def createRendererWidget(self, layer, style, renderer):
16        return RandomRendererWidget(layer, style, renderer)
17
18 rmetadata = RandomRendererMetadata()

```

```
QgsApplication.rendererRegistry().addRenderer(rrmetadata)
```

심볼 레이어의 경우와 비슷하게, 추상(`abstract`) 메타데이터 작성자도 렌더링 작업자 이름, 여러분이 볼 수 있는 이름, 그리고 선택적인 렌더링 작업자의 아이콘 이름을 기다립니다. `createRenderer()` 메소드가 DOM 트리로부터 렌더링 작업자의 상태를 복구하는 데 사용할 수 있는 `QDomElement` 클래스 인스턴스를 전달합니다. `createRendererWidget()` 메소드는 환경설정 위젯을 생성합니다. 렌더링 작업자가 GUI를 가지고 있지 않은 경우, 이 메소드를 사용할 필요가 없고 또는 이 메소드가 `None`을 반환할 수 있습니다.

렌더링 작업자에 아이콘을 연결하려면 `QgsRendererAbstractMetadata` 작성자에 아이콘을 세 번째 (선택적) 인자로 할당하면 됩니다—`RandomRendererMetadata` `__init__()` 함수에 있는 기저 클래스 작성자는 다음과 같이 보일 것입니다:

```
QgsRendererAbstractMetadata.__init__(self,
    "RandomRenderer",
    "Random renderer",
    QIcon(QPixmap("RandomRendererIcon.png", "png")))
```

The icon can also be associated at any later time using the `setIcon()` method of the metadata class. The icon can be loaded from a file (as shown above) or can be loaded from a [Qt resource](#) (PyQt5 includes .qrc compiler for Python).

6.9 남은 이야기들

할 일:

- 심볼 생성/수정하기
- 스타일 작업하기 (`QgsStyle` 클래스)
- 색상표 작업하기 (`QgsColorRamp` 클래스)
- 심볼 레이어 및 렌더링 작업자 레지스트리 탐색하기

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```

1 from qgis.core import (
2     QgsGeometry,
3     QgsGeometryCollection,
4     QgsPoint,
5     QgsPointXY,
6     QgsWkbTypes,
7     QgsProject,
8     QgsFeatureRequest,
9     QgsVectorLayer,
10    QgsDistanceArea,
11    QgsUnitTypes,
12    QgsCoordinateTransform,
13    QgsCoordinateReferenceSystem
14 )

```

공간 피처를 표현하는 포인트, 라인스트링, 그리고 폴리곤을 흔히 도형 (geometry) 이라고 부릅니다. QGIS 에서는 도형을 `QgsGeometry` 클래스를 사용해서 표현합니다.

도형 한 개가 실제로는 단순 (단일 부분, *single-part*) 도형의 집합인 경우가 종종 있습니다. 이런 도형을 다중 부분 (*multi-part*) 도형이라고 합니다. 다중 부분 도형이 한 가지 유형의 단순 도형으로만 이루어져 있을 경우 멀티포인트, 멀티라인스트링, 또는 멀티폴리곤이라고 부릅니다. 예를 들어 여러 개의 섬으로 이루어진 국가라면 멀티폴리곤으로 표현할 수 있습니다.

도형의 좌표는 어떤 좌표계라도 될 수 있습니다. 레이어에서 피처를 불러올 때, 해당 도형은 레이어의 좌표계를 따르는 좌표를 가지게 될 것입니다.

Description and specifications of all possible geometries construction and relationships are available in the [OGC Simple Feature Access Standards](#) for advanced details.

7.1 도형 작성하기

PyQGIS 는 도형을 생성하기 위한 옵션을 몇 개 제공합니다:

- 좌표로부터

```
1 gPnt = QgsGeometry.fromPointXY(QgsPointXY(1,1))
2 print(gPnt)
3 gLine = QgsGeometry.fromPolyline([QgsPoint(1, 1), QgsPoint(2, 2)])
4 print(gLine)
5 gPolygon = QgsGeometry.fromPolygonXY([[QgsPointXY(1, 1),
6     QgsPointXY(2, 2), QgsPointXY(2, 1)]]])
7 print(gPolygon)
```

`QgsPoint` 클래스 또는 `QgsPointXY` 클래스를 사용해서 좌표를 지정합니다. 이 두 클래스의 차이라고 하면 `QgsPoint` 클래스는 **M** 과 **Z** 차원을 지원한다는 점입니다.

폴리라인 (라인스트링) 은 포인트 목록으로 표현됩니다.

폴리곤은 선형 고리 (예: 닫힌 라인스트링들) 목록으로 표현됩니다. 첫 번째 고리 (ring) 가 외곽 고리 (경계) 이며, 있을 수도 있고 없을 수도 있는 그 다음 고리들은 폴리곤에 있는 구멍 (hole) 입니다. 몇몇 다른 프로그램과는 달리 QGIS 는 여러분을 위해 고리를 닫아줄 것이기 때문에 첫 번째 포인트를 마지막 포인트로 복제할 필요가 없다는 사실을 기억하세요.

다중 부분 도형은 한 단계 심화됩니다. 멀티포인트는 포인트의 목록, 멀티라인스트링은 라인스트링의 목록, 멀티폴리곤은 폴리곤의 목록입니다.

- WKT(well-known text) 로부터

```
geom = QgsGeometry.fromWkt("POINT(3 4)")
print(geom)
```

- WKB(well-known binary) 로부터

```
1 g = QgsGeometry()
2 wkb = bytes.fromhex("01010000000000000000000045400000000000001440")
3 g.fromWkb(wkb)
4
5 # print WKT representation of the geometry
6 print(g.asWkt())
```

7.2 도형에 접근하기

먼저 도형 유형을 알아야 합니다. `wkbType()` 메소드를 사용하면 됩니다. 이 메소드는 `QgsWkbTypes.Type` 클래스의 열거 목록 (enumeration) 으로부터 값을 반환합니다.

```
1 print(gPnt.wkbType())
2 # output: WkbType.Point
3 print(gLine.wkbType())
4 # output: WkbType.LineString
5 print(gPolygon.wkbType())
6 # output: WkbType.Polygon
```

아니면, `QgsWkbTypes.GeometryType` 메소드의 열거 목록으로부터 값을 반환하는 `type()` 메소드를 사용해도 됩니다.

```
print(gLine.type())
# output: GeometryType.Line
```

사람이 읽을 수 있는 도형 유형을 얻으려면 `displayString()` 함수를 사용하십시오.

```
1 print(QgsWkbTypes.displayString(gPnt.wkbType()))
2 # output: 'Point'
3 print(QgsWkbTypes.displayString(gLine.wkbType()))
4 # output: 'LineString'
5 print(QgsWkbTypes.displayString(gPolygon.wkbType()))
6 # output: 'Polygon'
```

도형이 다중 부분인지 아닌지 여부를 알려주는 `isMultipart()` 도우미 함수도 있습니다.

모든 벡터 유형 별로 도형에서 정보를 추출하기 위한 접근자 (accessor) 함수들이 있습니다. 다음은 이런 접근자를 어떻게 사용하는지 보여주는 예시입니다:

```
1 print(gPnt.asPoint())
2 # output: <QgsPointXY: POINT(1 1)>
3 print(gLine.asPolyline())
4 # output: [<QgsPointXY: POINT(1 1)>, <QgsPointXY: POINT(2 2)>]
5 print(gPolygon.asPolygon())
6 # output: [<QgsPointXY: POINT(1 1)>, <QgsPointXY: POINT(2 2)>, <QgsPointXY: POINT(2_
↪1)>, <QgsPointXY: POINT(1 1)>]
```

참고

(x,y) 튜플은 실제 튜플 (tuple) 이 아니라 `QgsPoint` 클래스 객체로, `x()` 와 `y()` 메소드를 사용해서 그 값들에 접근할 수 있습니다.

다중 부분 도형의 경우 비슷한 접근자 함수들이 존재합니다: `asMultiPoint()`, `asMultiPolyline()`, 그리고 `asMultiPolygon()` 메소드들입니다.

도형 유형과는 상관없이 도형의 모든 부분들을 반복해서 작업할 수 있습니다. 예를 들면:

```
geom = QgsGeometry.fromWkt( 'MultiPoint( 0 0, 1 1, 2 2)' )
for part in geom.parts():
    print(part.asWkt())
```

```
Point (0 0)
Point (1 1)
Point (2 2)
```

```
geom = QgsGeometry.fromWkt( 'LineString( 0 0, 10 10 )' )
for part in geom.parts():
    print(part.asWkt())
```

```
LineString (0 0, 10 10)
```

```
gc = QgsGeometryCollection()
gc.fromWkt( 'GeometryCollection( Point(1 2), Point(11 12), LineString(33 34, 44 45))' )
print(gc[1].asWkt())
```

```
Point (11 12)
```

`QgsGeometry.parts()` 메소드를 사용하면 도형의 각 부분을 수정할 수도 있습니다.

```
1 geom = QgsGeometry.fromWkt( 'MultiPoint( 0 0, 1 1, 2 2)' )
2 for part in geom.parts():
3     part.transform(QgsCoordinateTransform(
4         QgsCoordinateReferenceSystem("EPSG:4326"),
5         QgsCoordinateReferenceSystem("EPSG:3111"),
6         QgsProject.instance() )
7     )
8
9 print(geom.asWkt())
```

```
MultiPoint ((-10334728.12541878595948219 -5360106.25905461423099041), (-10462135.
↪16126426123082638 -5217485.4735023295506835), (-10589399.844444035589694977 -5072021.
↪45942386891692877))
```

7.3 도형 술어 및 연산

QGIS 는 도형 술어 (predicate) 같은 고급 도형 연산 (`contains()`, `intersects()`, ...) 과 집합 연산 (`combine()`, `difference()`, ...) 에 GEOS 라이브러리를 사용합니다. GEOS 라이브러리는 (폴리곤의 경우) 면적 또는 (폴리곤 및 라인의 경우) 길이 같은 도형의 기하 속성도 계산할 수 있습니다.

지정한 레이어에 있는 피처들을 반복 작업해서 피처들의 도형을 바탕으로 도형 계산을 수행하는 예시를 볼까요. 다음 코드는 예제 QGIS 프로젝트 안에서 `countries` 레이어에 있는 각 국가의 면적과 둘레를 계산해서 출력할 것입니다.

다음 코드는 `layer` 가 폴리곤 피처 유형인 `QgsVectorLayer` 클래스 객체라고 가정합니다.

```
1 # let's access the 'countries' layer
2 layer = QgsProject.instance().mapLayersByName('countries')[0]
3
4 # let's filter for countries that begin with Z, then get their features
5 query = '"name" LIKE \'Z%\''
6 features = layer.getFeatures(QgsFeatureRequest().setFilterExpression(query))
7
8 # now loop through the features, perform geometry computation and print the results
9 for f in features:
10     geom = f.geometry()
11     name = f.attribute('NAME')
12     print(name)
13     print('Area: ', geom.area())
14     print('Perimeter: ', geom.length())
```

```
1 Zambia
2 Area: 62.82279065343119
3 Perimeter: 50.65232014052552
4 Zimbabwe
5 Area: 33.41113559136517
6 Perimeter: 26.608288555013935
```

이제 도형들의 면적과 둘레를 계산해서 출력했습니다. 하지만 그 값들이 이상하다는 사실을 한 눈에 알아차렸겠죠. `QgsGeometry` 클래스에서 나온 `area()` 및 `length()` 메소드를 사용해서 면적과 둘레를 계산할 때 좌표계를

연산에 넣지 않았기 때문입니다. 좀 더 강력한 면적 및 거리 계산을 하고 싶다면, 타원체 기반 계산을 수행할 수 있는 `QgsDistanceArea` 클래스를 사용하면 됩니다:

다음 코드는 layer 가 폴리곤 피쳐 유형인 `QgsVectorLayer` 클래스 객체라고 가정합니다.

```

1 d = QgsDistanceArea()
2 d.setEllipsoid('WGS84')
3
4 layer = QgsProject.instance().mapLayersByName('countries')[0]
5
6 # let's filter for countries that begin with Z, then get their features
7 query = '"name" LIKE \'Z%\''
8 features = layer.getFeatures(QgsFeatureRequest().setFilterExpression(query))
9
10 for f in features:
11     geom = f.geometry()
12     name = f.attribute('NAME')
13     print(name)
14     print("Perimeter (m):", d.measurePerimeter(geom))
15     print("Area (m2):", d.measureArea(geom))
16
17     # let's calculate and print the area again, but this time in square kilometers
18     print("Area (km2):", d.convertAreaMeasurement(d.measureArea(geom), QgsUnitTypes.
    ↳AreaSquareKilometers))

```

```

1 Zambia
2 Perimeter (m): 5539361.250294601
3 Area (m2): 751989035032.9031
4 Area (km2): 751989.0350329031
5 Zimbabwe
6 Perimeter (m): 2865021.3325076113
7 Area (m2): 389267821381.6008
8 Area (km2): 389267.8213816008

```

아니면, 포인트 2 개 사이의 거리를 알고 싶을 수도 있을 겁니다.

```

1 d = QgsDistanceArea()
2 d.setEllipsoid('WGS84')
3
4 # Let's create two points.
5 # Santa claus is a workaholic and needs a summer break,
6 # lets see how far is Tenerife from his home
7 santa = QgsPointXY(25.847899, 66.543456)
8 tenerife = QgsPointXY(-16.5735, 28.0443)
9
10 print("Distance in meters: ", d.measureLine(santa, tenerife))

```

QGIS 에 포함되어 있는 알고리즘들의 수많은 예시를 찾아볼 수 있으며, 이 알고리즘들을 사용해서 벡터 데이터를 분석하고 변환할 수 있습니다. 다음 링크들은 그 가운데 몇몇 알고리즘들의 코드를 보여줍니다.

- `QgsDistanceArea` 클래스를 사용해서 거리 및 면적 계산: [거리 행렬 알고리즘](#)
- 라인을 폴리곤으로 알고리즘

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (
2     QgsCoordinateReferenceSystem,
3     QgsCoordinateTransform,
4     QgsProject,
5     QgsPointXY,
6 )
```

8.1 좌표계

`QgsCoordinateReferenceSystem` 클래스가 좌표계를 캡슐화합니다. 이 클래스의 인스턴스를 몇 가지 서로 다른 방법으로 생성할 수 있습니다:

- 좌표계 ID 로 좌표계 지정하기

```
# EPSG 4326 is allocated for WGS84
crs = QgsCoordinateReferenceSystem("EPSG:4326")
print(crs.isValid())
```

```
True
```

QGIS 는 다음 서식을 가진 서로 다른 좌표계 식별자들을 지원합니다:

- EPSG:<code>—EPSG 기관이 할당한 ID 입니다. `createFromOgcWms()` 메소드로 처리합니다.
- POSTGIS:<srid>—PostGIS 데이터베이스에서 쓰이는 ID 입니다. `createFromSrid()` 메소드로 처리합니다.

- INTERNAL:<srsid> —QGIS 데이터베이스에서 쓰이는 ID 입니다. `createFromSrsId()` 메소드로 처리합니다.
- PROJ:<proj> —`createFromProj()` 메소드로 처리합니다.
- WKT:<wkt> —`createFromWkt()` 메소드로 처리합니다.

접두어를 지정하지 않는 경우, WKT(well-known text) 정의라고 가정합니다.

- WKT 로 좌표계 지정하기

```
1 wkt = 'GEOGCS["WGS84", DATUM["WGS84", SPHEROID["WGS84", 6378137.0, 298.
    ↳257223563]],' \
2     'PRIMEM["Greenwich", 0.0], UNIT["degree",0.017453292519943295],' \
3     'AXIS["Longitude",EAST], AXIS["Latitude",NORTH]]'
4 crs = QgsCoordinateReferenceSystem(wkt)
5 print(crs.isValid())
```

True

- 무결하지 않은 좌표계를 생성한 다음 `create*` 함수들 가운데 하나를 사용해서 초기 설정합니다. 다음 예시에서는 투영체를 초기 설정하는 데 Proj 문자열을 사용합니다.

```
crs = QgsCoordinateReferenceSystem()
crs.createFromProj("+proj=longlat +ellps=WGS84 +datum=WGS84 +no_defs")
print(crs.isValid())
```

True

좌표계를 성공적으로 생성했는지 여부를 (예: 데이터베이스 검색) 확인하는 편이 좋습니다: `isValid()` 메소드가 True 를 반환해야만 합니다.

공간 참조 시스템 (spatial reference system) 을 초기 설정하기 위해 QGIS 가 `srs.db` 내부 데이터베이스에서 알맞은 값을 검색해야 한다는 사실을 기억하십시오. 즉 여러분이 독립 응용 프로그램을 생성한 경우 `QgsApplication.setPrefixPath()` 메소드로 경로를 정확하게 설정해야 합니다. 그렇지 않을 경우 이 데이터베이스를 찾지 못할 것입니다. 여러분이 QGIS 파이썬 콘솔에서 명령어를 실행하거나 또는 플러그인을 개발하는 중이라면 상관없습니다. 이미 모든 것이 설정되어 있기 때문입니다.

공간 참조 시스템 정보에 접근하기:

```
1 crs = QgsCoordinateReferenceSystem("EPSG:4326")
2
3 print("QGIS CRS ID:", crs.srsid())
4 print("PostGIS SRID:", crs.postgisSrid())
5 print("Description:", crs.description())
6 print("Projection Acronym:", crs.projectionAcronym())
7 print("Ellipsoid Acronym:", crs.ellipsoidAcronym())
8 print("Proj String:", crs.toProj())
9 # check whether it's geographic or projected coordinate system
10 print("Is geographic:", crs.isGeographic())
11 # check type of map units in this CRS (values defined in QGis::units enum)
12 print("Map units:", crs.mapUnits())
```

산출물:

```
1 QGIS CRS ID: 3452
2 PostGIS SRID: 4326
3 Description: WGS 84
4 Projection Acronym: longlat
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

5 Ellipsoid Acronym: EPSG:7030
6 Proj String: +proj=longlat +datum=WGS84 +no_defs
7 Is geographic: True
8 Map units: DistanceUnit.Degrees

```

8.2 좌표계 변환

`QgsCoordinateTransform` 클래스를 사용해서 서로 다른 공간 참조 시스템들을 변환시킬 수 있습니다. 이 클래스를 가장 쉽게 사용하는 방법은 소스 및 대상 좌표계를 생성한 다음 이 좌표계들과 현재 프로젝트를 가진 `QgsCoordinateTransform` 인스턴스를 작성하는 것입니다. 그 다음 `transform()` 함수를 반복 호출해서 변환을 수행합니다. 이 함수는 기본적으로 정방향 변환을 하지만, 역방향 변환도 할 수 있습니다.

```

1 crsSrc = QgsCoordinateReferenceSystem("EPSG:4326")      # WGS 84
2 crsDest = QgsCoordinateReferenceSystem("EPSG:32633")    # WGS 84 / UTM zone 33N
3 transformContext = QgsProject.instance().transformContext()
4 xform = QgsCoordinateTransform(crsSrc, crsDest, transformContext)
5
6 # forward transformation: src -> dest
7 pt1 = xform.transform(QgsPointXY(18,5))
8 print("Transformed point:", pt1)
9
10 # inverse transformation: dest -> src
11 pt2 = xform.transform(pt1, QgsCoordinateTransform.ReverseTransform)
12 print("Transformed back:", pt2)

```

산출물:

```

Transformed point: <QgsPointXY: POINT(832713.79873844375833869 553423.
↪98688333143945783)>
Transformed back: <QgsPointXY: POINT(18 4.99999999999999911)>

```


맵 캔버스 사용하기

💡 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```

1 from qgis.PyQt.QtGui import (
2     QColor,
3 )
4
5 from qgis.PyQt.QtCore import Qt, QRectF
6
7 from qgis.PyQt.QtWidgets import QMenu
8
9 from qgis.core import (
10     QgsVectorLayer,
11     QgsPoint,
12     QgsPointXY,
13     QgsProject,
14     QgsGeometry,
15     QgsMapRendererJob,
16     QgsWkbTypes,
17 )
18
19 from qgis.gui import (
20     QgsMapCanvas,
21     QgsVertexMarker,
22     QgsMapCanvasItem,
23     QgsMapMouseEvent,
24     QgsRubberBand,
25 )

```

맵 캔버스가 중첩하는 맵 레이어들로 구성된 맵을 보여주고 맵과 레이어들과 쌍방향 작업을 할 수 있게 해주기 때문에, 맵 캔버스 위젯은 아마도 QGIS 에서 가장 중요한 위젯일 겁니다. 맵 캔버스는 항상 현재 캔버스 범위로 정의되는 맵의 일부분을 보여줍니다. 쌍방향 작업은 맵 도구 를 사용해서 이루어집니다: 이동 (pan), 확대/축소

(zoom), 레이어 식별, 측정, 벡터 편집, 그리고 기타 작업들을 위한 도구들이 있습니다. 다른 그래픽 프로그램들처럼, 도구 하나는 항상 활성화되어 있는 상태이며 여러분은 사용할 수 있는 다른 도구들로 전환할 수 있습니다.

The map canvas is implemented with the `QgsMapCanvas` class in the `qgis.gui` module. The implementation is based on the Qt Graphics View framework. This framework generally provides a surface and a view where custom graphics items are placed and user can interact with them. We will assume that you are familiar enough with Qt to understand the concepts of the graphics scene, view and items. If not, please read the [overview of the framework](#).

맵을 이동시키거나 확대/축소할 (또는 새로고침을 촉발하는 기타 다른 액션을 취할) 때마다, 현재 범위 안에 있는 맵을 다시 렌더링합니다. 레이어를 (`QgsMapRendererJob` 클래스를 사용해서) 이미지로 렌더링해서, 캔버스에 이 이미지를 보이는 것입니다. `QgsMapCanvas` 클래스는 렌더링된 맵의 새로고침도 제어합니다. 배경 역할을 하는 이 항목 외에, 더 많은 맵 캔버스 항목들이 있을 수도 있습니다.

전형적인 맵 캔버스 항목이라고 하면 (측정, 벡터 편집 등등에 쓰이는) 고무줄 (rubber band) 또는 꼭짓점 마커 등등이 있습니다. 이런 캔버스 항목들은 보통 맵 도구들을 위한 가시적 피드백을 주기 위해 사용됩니다. 예를 들면 새 폴리곤을 생성할 때 맵 도구가 폴리곤의 현재 형태를 보여주는 고무줄 캔버스 항목을 생성합니다. 모든 맵 캔버스 항목은 기본 `QGraphicsItem` 객체에 더 많은 기능을 추가하는 `QgsMapCanvasItem` 클래스의 하위 클래스입니다.

요약하면 맵 캔버스 아키텍처는 다음 3 가지 개념으로 이루어집니다.

- 맵 캔버스—맵을 보여주는 데 쓰입니다.
- 맵 캔버스 항목—맵 캔버스에 보일 수 있는 추가적인 항목들입니다.
- 맵 도구—맵 캔버스와 쌍방향 작업을 하는 데 쓰입니다.

9.1 맵 캔버스 내장시키기

맵 캔버스는 다른 모든 Qt 위젯과 마찬가지로이기 때문에, 이를 사용하는 방법도 생성하고 보이는 방법처럼 간단합니다.

```
canvas = QgsMapCanvas()
canvas.show()
```

이 코드는 맵 캔버스를 가진 독립형 (standalone) 창을 생성합니다. 기존 위젯이나 창에 맵 캔버스 창을 내장시킬 수도 있습니다. .ui 파일과 Qt 설계자를 사용할 때, 양식 상에 `QWidget` 을 배치하고 새 클래스로 승격시키십시오: `QgsMapCanvas` 를 클래스 이름으로 그리고 `qgis.gui` 를 헤더 파일로 설정하십시오. `pyuic5` 유틸리티가 이를 처리할 것입니다. 캔버스를 내장시킬 수 있는 매우 편리한 방법입니다. 다른 방법은 맵 캔버스와 기타 위젯들을 (메인 창 또는 대화창의 하위 창으로써) 작성하는 코드를 직접 입력한 다음 조판 (layout) 을 생성하는 것입니다.

맵 캔버스는 기본적으로 검은색 배경이고 위신포 제거 (anti-aliasing) 를 사용하지 않습니다. 배경을 하얀색으로 설정하고 매끄러운 렌더링을 위해 위신포 제거를 활성화하려면:

```
canvas.setCanvasColor(Qt.white)
canvas.enableAntiAliasing(True)
```

(여러분이 궁금할 경우를 위해 알려드리자면, Qt 는 `PyQt.QtCore` 모듈에서 나온 것이고 `Qt.white` 는 사전 정의된 `QColor` 인스턴스들 가운데 하나입니다.)

이제 맵 레이어 몇 개를 추가할 때입니다. 먼저 레이어 하나를 열어서 현재 프로젝트에 추가할 것입니다. 그 다음 캔버스 범위를 설정하고 캔버스에 보일 레이어 목록을 설정할 것입니다.

```
1 vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports layer
  ↳", "ogr")
2 if not vlayer.isValid():
3     print("Layer failed to load!")
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

4
5 # add layer to the registry
6 QgsProject.instance().addMapLayer(vlayer)
7
8 # set extent to the extent of our layer
9 canvas.setExtent(vlayer.extent())
10
11 # set the map canvas layer set
12 canvas.setLayers([vlayer])

```

이 명령어들을 실행하면, 사용자가 불러온 레이어가 캔버스에 보일 것입니다.

9.2 고무줄과 꼭짓점 마커

캔버스에 있는 맵 최상단에 추가 데이터를 보이려면 맵 캔버스 항목을 사용하십시오. (`^writing-custom-map-canvas-items^` 에서 설명할) 사용자 정의 캔버스 항목 클래스를 생성할 수 있지만, 사용자 편의를 위한 유용한 캔버스 항목 클래스 2 개가 있습니다: 폴리라인 또는 폴리곤을 그리기 위한 `QgsRubberBand` 클래스와 포인트를 그리기 위한 `QgsVertexMarker` 클래스입니다. 두 클래스 모두 맵 좌표를 사용해서 작업하기 때문에, 캔버스를 이동시키거나 확대/축소할 때 형태를 자동으로 이동/크기 조정합니다.

폴리라인을 보이려면:

```

r = QgsRubberBand(canvas, QgsWkbTypes.LineGeometry) # line
points = [QgsPoint(-100, 45), QgsPoint(10, 60), QgsPoint(120, 45)]
r.setToGeometry(QgsGeometry.fromPolyline(points), None)

```

폴리곤을 보이려면:

```

r = QgsRubberBand(canvas, QgsWkbTypes.PolygonGeometry) # polygon
points = [[QgsPointXY(-100, 35), QgsPointXY(10, 50), QgsPointXY(120, 35)]]
r.setToGeometry(QgsGeometry.fromPolygonXY(points), None)

```

폴리곤의 포인트들이 평범한 목록이 아니라는 점을 기억하십시오. 사실 이 목록은 폴리곤의 선형 고리들을 담고 있는 고리 목록입니다. 첫 번째 고리는 외곽 경계이며, 그 다음 (있을 수도 있고 없을 수도 있는) 고리들은 폴리곤 내부의 구멍에 해당합니다.

고무줄을 사용자 정의할 수 있습니다. 즉 색상 및 선 너비를 변경할 수 있습니다.

```

r.setColor(QColor(0, 0, 255))
r.setWidth(3)

```

캔버스 항목은 캔버스 신(scene)에 바인딩되어 있습니다. 캔버스 항목을 일시적으로 숨기려면 (그리고 다시 보이려면) `hide()` 와 `show()` 함수 조합을 사용하십시오. 어떤 항목을 완전히 제거하려면, 캔버스 신에서 제거해야 합니다:

```

canvas.scene().removeItem(r)

```

(C++ 의 경우 항목을 그냥 삭제할 수 있지만, 파이썬의 경우 `del r` 명령어는 참조만 삭제할 뿐 실제 객체는 캔버스가 소유하고 있기 때문에 계속 남아 있을 것입니다.)

포인트도 고무줄을 사용해서 그릴 수 있지만, 이 작업에는 `QgsVertexMarker` 클래스가 더 적합합니다. (`QgsRubberBand` 클래스는 원하는 지점 주위에 직사각형을 그릴 뿐입니다.)

꼭짓점 마커는 다음과 같이 사용할 수 있습니다:

```
m = QgsVertexMarker(canvas)
m.setCenter(QgsPointXY(10, 40))
```

이 코드는 [10,45] 위치에 빨간색 십자표를 그릴 것입니다. 아이콘의 유형, 크기, 색상, 그리고 펜 너비를 사용자 정의할 수 있습니다:

```
m.setColor(QColor(0, 255, 0))
m.setIconSize(5)
m.setIconType(QgsVertexMarker.ICON_BOX) # or ICON_CROSS, ICON_X
m.setPenWidth(3)
```

꼭짓점 마커를 일시적으로 숨기거나 캔버스에서 제거하려면, 고무줄의 경우와 동일한 메소드들을 사용하십시오.

9.3 캔버스에서 맵 도구 사용하기

다음 예시는 맵 캔버스와 맵 이동 및 확대/축소를 위한 기본 맵 도구를 담고 있는 창일 작성합니다. 각 도구의 활성화를 위한 액션을 생성합니다: `QgsMapToolPan` 클래스는 이동을 수행하고, `QgsMapToolZoom` 클래스 인스턴스의 쌍으로 확대/축소를 수행합니다. 이 액션들을 체크할 수 있게 설정해서, 향후 액션의 체크/체크 해제 상태를 자동으로 처리할 수 있도록 도구에 할당합니다—어떤 맵 도구를 활성화시킬 때, 해당 도구의 액션을 선택 상태로 표시하고 이전 맵 도구의 액션을 선택 해제합니다. 맵 도구를 활성화시키는 메소드는 `setMapTool()` 입니다.

```
1 from qgis.gui import *
2 from qgis.PyQt.QtWidgets import QAction, QMainWindow
3 from qgis.PyQt.QtCore import Qt
4
5 class MyWnd(QMainWindow):
6     def __init__(self, layer):
7         QMainWindow.__init__(self)
8
9         self.canvas = QgsMapCanvas()
10        self.canvas.setCanvasColor(Qt.white)
11
12        self.canvas.setExtent(layer.extent())
13        self.canvas.setLayers([layer])
14
15        self.setCentralWidget(self.canvas)
16
17        self.actionZoomIn = QAction("Zoom in", self)
18        self.actionZoomOut = QAction("Zoom out", self)
19        self.actionPan = QAction("Pan", self)
20
21        self.actionZoomIn.setCheckable(True)
22        self.actionZoomOut.setCheckable(True)
23        self.actionPan.setCheckable(True)
24
25        self.actionZoomIn.triggered.connect(self.zoomIn)
26        self.actionZoomOut.triggered.connect(self.zoomOut)
27        self.actionPan.triggered.connect(self.pan)
28
29        self.toolbar = self.addToolBar("Canvas actions")
30        self.toolbar.addAction(self.actionZoomIn)
31        self.toolbar.addAction(self.actionZoomOut)
32        self.toolbar.addAction(self.actionPan)
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

33
34     # create the map tools
35     self.toolPan = QgsMapToolPan(self.canvas)
36     self.toolPan.setAction(self.actionPan)
37     self.toolZoomIn = QgsMapToolZoom(self.canvas, False) # false = in
38     self.toolZoomIn.setAction(self.actionZoomIn)
39     self.toolZoomOut = QgsMapToolZoom(self.canvas, True) # true = out
40     self.toolZoomOut.setAction(self.actionZoomOut)
41
42     self.pan()
43
44     def zoomIn(self):
45         self.canvas.setMapTool(self.toolZoomIn)
46
47     def zoomOut(self):
48         self.canvas.setMapTool(self.toolZoomOut)
49
50     def pan(self):
51         self.canvas.setMapTool(self.toolPan)

```

파이썬 콘솔 편집기에서 앞의 코드를 시도해볼 수 있습니다. 캔버스 창을 열려면, `MyWnd` 클래스를 초기 설정하는 다음 줄들을 추가하십시오. 새로 생성한 캔버스 위에 현재 선택된 레이어를 렌더링할 것입니다:

```

w = MyWnd(iface.activeLayer())
w.show()

```

9.3.1 QgsMapToolIdentifyFeature 를 사용해서 피쳐 선택하기

사용자에게 콜백 (callback) 함수로 전송될 피쳐 하나를 선택하도록 요청하려면, `QgsMapToolIdentifyFeature` 맵 도구를 사용하면 됩니다.

```

1  def callback(feature):
2      """Code called when the feature is selected by the user"""
3      print("You clicked on feature {}".format(feature.id()))
4
5  canvas = iface.mapCanvas()
6  feature_identifier = QgsMapToolIdentifyFeature(canvas)
7
8  # indicates the layer on which the selection will be done
9  feature_identifier.setLayer(vlayer)
10
11 # use the callback as a slot triggered when the user identifies a feature
12 feature_identifier.featureIdentified.connect(callback)
13
14 # activation of the map tool
15 canvas.setMapTool(feature_identifier)

```

9.3.2 맵 캔버스 컨텍스트 메뉴에 항목 추가하기

`contextMenuAboutToShow` 신호를 사용해서 맵 캔버스의 컨텍스트 메뉴에 추가할 수도 있는 항목들을 통해서도 맵 캔버스와의 쌍방향 작업을 수행할 수 있습니다.

다음은 맵 캔버스를 오른쪽 클릭할 때 나타나는 기본 항목에 *My menu* → *My Action* 액션을 추가하는 코드입니다.

```

1 # a slot to populate the context menu
2 def populateContextMenu(menu: QMenu, event: QgsMapMouseEvent):
3     subMenu = menu.addMenu('My Menu')
4     action = subMenu.addAction('My Action')
5     action.triggered.connect(lambda *args:
6                             print(f'Action triggered at {event.x()}, {event.y()}'))
7
8 canvas.contextMenuAboutToShow.connect(populateContextMenu)
9 canvas.show()

```

9.4 사용자 정의 맵 도구 작성하기

여러분이 캔버스 상에서 수행하는 액션에 사용자 정의 습성을 구현하기 위한 사용자 정의 도구를 작성할 수 있습니다.

맵 도구는 클래스든 파생 클래스든 `QgsMapTool` 클래스로부터 상속받아야 하며, 이미 앞에서 본 바와 같이 캔버스에서 `setMapTool()` 메소드를 사용해서 활성 도구로써 선택되어야 합니다.

다음 예시는 캔버스 상에서 클릭 & 드래그로 직사각형 범위를 정의하도록 해주는 맵 도구의 코드입니다. 직사각형을 정의하면, 콘솔에 그 경계 좌표를 출력합니다. 앞에서 설명했던 고무줄 기능을 사용해서 정의되고 있는 직사각형을 보일 것입니다.

```

1 class RectangleMapTool(QgsMapToolEmitPoint):
2     def __init__(self, canvas):
3         self.canvas = canvas
4         QgsMapToolEmitPoint.__init__(self, self.canvas)
5         self.rubberBand = QgsRubberBand(self.canvas, QgsWkbTypes.PolygonGeometry)
6         self.rubberBand.setColor(Qt.red)
7         self.rubberBand.setWidth(1)
8         self.reset()
9
10    def reset(self):
11        self.startPoint = self.endPoint = None
12        self.isEmittingPoint = False
13        self.rubberBand.reset(QgsWkbTypes.PolygonGeometry)
14
15    def canvasPressEvent(self, e):
16        self.startPoint = self.toMapCoordinates(e.pos())
17        self.endPoint = self.startPoint
18        self.isEmittingPoint = True
19        self.showRect(self.startPoint, self.endPoint)
20
21    def canvasReleaseEvent(self, e):
22        self.isEmittingPoint = False
23        r = self.rectangle()
24        if r is not None:
25            print("Rectangle:", r.xMinimum(),
26                  r.yMinimum(), r.xMaximum(), r.yMaximum())

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

27         )
28
29     def canvasMoveEvent(self, e):
30         if not self.isEmittingPoint:
31             return
32
33         self.endPoint = self.toMapCoordinates(e.pos())
34         self.showRect(self.startPoint, self.endPoint)
35
36     def showRect(self, startPoint, endPoint):
37         self.rubberBand.reset(QgsWkbTypes.PolygonGeometry)
38         if startPoint.x() == endPoint.x() or startPoint.y() == endPoint.y():
39             return
40
41         point1 = QgsPointXY(startPoint.x(), startPoint.y())
42         point2 = QgsPointXY(startPoint.x(), endPoint.y())
43         point3 = QgsPointXY(endPoint.x(), endPoint.y())
44         point4 = QgsPointXY(endPoint.x(), startPoint.y())
45
46         self.rubberBand.addPoint(point1, False)
47         self.rubberBand.addPoint(point2, False)
48         self.rubberBand.addPoint(point3, False)
49         self.rubberBand.addPoint(point4, True)      # true to update canvas
50         self.rubberBand.show()
51
52     def rectangle(self):
53         if self.startPoint is None or self.endPoint is None:
54             return None
55         elif (self.startPoint.x() == self.endPoint.x() or \
56               self.startPoint.y() == self.endPoint.y()):
57             return None
58
59         return QgsRectangle(self.startPoint, self.endPoint)
60
61     def deactivate(self):
62         QgsMapTool.deactivate(self)
63         self.deactivated.emit()

```

9.5 사용자 정의 맵 캔버스 항목 작성하기

다음은 원을 그리는 사용자 정의 캔버스 항목의 예시입니다:

```

1 class CircleCanvasItem(QgsMapCanvasItem):
2     def __init__(self, canvas):
3         super().__init__(canvas)
4         self.center = QgsPoint(0, 0)
5         self.size = 100
6
7     def setCenter(self, center):
8         self.center = center
9
10    def center(self):
11        return self.center
12

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
13 def setSize(self, size):
14     self.size = size
15
16 def size(self):
17     return self.size
18
19 def boundingRect(self):
20     return QRectF(self.center.x() - self.size/2,
21                   self.center.y() - self.size/2,
22                   self.center.x() + self.size/2,
23                   self.center.y() + self.size/2)
24
25 def paint(self, painter, option, widget):
26     path = QPainterPath()
27     path.moveTo(self.center.x(), self.center.y());
28     path.arcTo(self.boundingRect(), 0.0, 360.0)
29     painter.fillPath(path, QColor("red"))
30
31
32 # Using the custom item:
33 item = CircleCanvasItem(iface.mapCanvas())
34 item.setCenter(QgsPointXY(200,200))
35 item.setSize(80)
```

CHAPTER 10

맵 렌더링 및 출력하기

힌트

이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 import os
2
3 from qgis.core import (
4     QgsGeometry,
5     QgsMapSettings,
6     QgsPrintLayout,
7     QgsMapSettings,
8     QgsMapRendererParallelJob,
9     QgsLayoutItemLabel,
10    QgsLayoutItemLegend,
11    QgsLayoutItemMap,
12    QgsLayoutItemPolygon,
13    QgsLayoutItemScaleBar,
14    QgsLayoutExporter,
15    QgsLayoutItem,
16    QgsLayoutPoint,
17    QgsLayoutSize,
18    QgsUnitTypes,
19    QgsProject,
20    QgsFillSymbol,
21    QgsAbstractValidityCheck,
22    check,
23 )
24
25 from qgis.PyQt.QtGui import (
26     QPolygonF,
27     QColor,
28 )
29
30 from qgis.PyQt.QtCore import (
```

```

31     QPointF,
32     QRectF,
33     QSize,
34 )

```

입력 데이터를 맵으로 렌더링해야 하는 경우 일반적으로 두 가지 접근법이 있습니다: `QgsMapRendererJob` 클래스를 사용해서 렌더링하는 빠른 방법 또는 맵을 `QgsLayout` 클래스로 구성해서 좀 더 미세 조정된 산출물을 생성하는 방법입니다.

10.1 단순 렌더링

렌더링 설정을 정의하는 `QgsMapSettings` 객체를 생성한 다음 해당 설정으로 `QgsMapRendererJob` 클래스를 작성해서 렌더링을 수행합니다. 후자는 산출 이미지를 생성하는 데 쓰입니다.

다음은 그 예시입니다:

```

1  image_location = os.path.join(QgsProject.instance().homePath(), "render.png")
2
3  vlayer = iface.activeLayer()
4  settings = QgsMapSettings()
5  settings.setLayers([vlayer])
6  settings.setBackgroundColor(QColor(255, 255, 255))
7  settings.setOutputSize(QSize(800, 600))
8  settings.setExtent(vlayer.extent())
9
10 render = QgsMapRendererParallelJob(settings)
11
12 def finished():
13     img = render.renderedImage()
14     # save the image; e.g. img.save("/Users/myuser/render.png", "png")
15     img.save(image_location, "png")
16
17 render.finished.connect(finished)
18
19 # Start the rendering
20 render.start()
21
22 # The following loop is not normally required, we
23 # are using it here because this is a standalone example.
24 from qgis.PyQt.QtCore import QEventLoop
25 loop = QEventLoop()
26 render.finished.connect(loop.quit)
27 loop.exec_()

```

10.2 서로 다른 좌표계를 가진 레이어들을 렌더링하기

레이어가 하나 이상 있는데 그 레이어들이 서로 다른 좌표계를 가지고 있는 경우, 앞의 단순 예시는 아마도 작동하지 않을 것입니다. 범위 계산으로부터 올바른 값들을 얻으려면 대상 좌표계를 명확하게 설정해야 합니다:

```
layers = [iface.activeLayer()]
settings = QgsMapSettings()
settings.setLayers(layers)
settings.setDestinationCrs(layers[0].crs())
```

10.3 인쇄 조판을 사용해서 산출하기

앞에서 본 단순 렌더링보다 좀 더 세련된 산출물을 생성하고 싶은 경우, 인쇄 조판 (print layout) 이 매우 유용한 도구입니다. 맵 뷰, 라벨, 범례, 표 그리고 보통 종이 지도에서 볼 수 있는 기타 요소들로 이루어진 복잡한 맵 조판을 생성할 수 있습니다. 나중에 이 조판을 PDF, SVG, 래스터 이미지로 내보내거나 프린터에서 직접 인쇄할 수 있습니다.

조판은 클래스 여러 개들로 이루어집니다. 이 클래스들은 모두 핵심 라이브러리에 속해 있습니다. QGIS 응용 프로그램은 이런 요소들을 배치하기 위한 편리한 GUI 를 가지고 있지만, GUI 라이브러리에서 이 GUI 를 사용할 수는 없습니다. 여러분이 Qt 그래픽스 뷰 프레임워크 에 익숙하지 않은 경우, 조판이 이 프레임워크를 기반으로 하기 때문에, 지금 해당 문서를 읽어볼 것을 권장합니다.

조판의 중심 클래스는 Qt QGraphicsScene 클래스에서 파생된 QgsLayout 클래스입니다. 이 클래스의 인스턴스를 생성해봅시다:

```
project = QgsProject.instance()
layout = QgsPrintLayout(project)
layout.initializeDefaults()
```

이 코드는 몇몇 기본 설정으로, 그 중에서도 조판에 비어 있는 A4 페이지를 추가해서 조판을 초기 설정합니다. initializeDefaults() 메소드를 호출하지 않고서도 조판을 생성할 수 있지만, 여러분이 조판에 페이지를 직접 추가해야 할 것입니다.

앞의 코드는 GUI 에 가시화되지 않는 “임시” 조판을 생성합니다. 프로젝트 자체를 수정하지 않고, 또는 사용자에게 이런 변경 사항을 노출시키지 않고서도 몇몇 항목들을 빠르게 추가하고 내보낼 수 있는 편리한 방법입니다. 조판을 프로젝트와 함께 저장하거나 복원하고 조판 관리자에서 사용할 수 있기를 바라는 경우 다음 코드를 추가하십시오:

```
layout.setName("MyLayout")
project.layoutManager().addLayout(layout)
```

이제 조판에 여러 가지 (맵, 라벨, ...) 요소들을 추가할 수 있습니다. 이 객체들은 모두 기저 QgsLayoutItem 클래스에서 상속받은 클래스들로 표현됩니다.

다음은 조판에 추가할 수 있는 주요 조판 항목들 가운데 일부를 설명합니다.

- **map** — 사용자 정의 크기를 가진 맵을 생성하고 현재 맵 캔버스를 렌더링하는 코드입니다:

```
1 map = QgsLayoutItemMap(layout)
2 # Set map item position and size (by default, it is a 0 width/0 height item,
  ↳ placed at 0,0)
3 map.attemptMove(QgsLayoutPoint(5,5, QgsUnitTypes.LayoutMillimeters))
4 map.attemptResize(QgsLayoutSize(200,200, QgsUnitTypes.LayoutMillimeters))
5 # Provide an extent to render
6 map.zoomToExtent(iface.mapCanvas().extent())
7 layout.addLayoutItem(map)
```

- label —라벨을 보이게 할 수 있게 해주는 코드입니다. 글꼴, 색상, 정렬 및 여백을 수정할 수 있습니다:

```
label = QgsLayoutItemLabel(layout)
label.setText("Hello world")
label.adjustSizeToText()
layout.addLayoutItem(label)
```

- legend

```
legend = QgsLayoutItemLegend(layout)
legend.setLinkedMap(map) # map is an instance of QgsLayoutItemMap
layout.addLayoutItem(legend)
```

- 축척막대 (scale bar)

```
1 item = QgsLayoutItemScaleBar(layout)
2 item.setStyle('Numeric') # optionally modify the style
3 item.setLinkedMap(map) # map is an instance of QgsLayoutItemMap
4 item.applyDefaultSize()
5 layout.addLayoutItem(item)
```

- 노드 기반 형태

```
1 polygon = QPolygonF()
2 polygon.append(QPointF(0.0, 0.0))
3 polygon.append(QPointF(100.0, 0.0))
4 polygon.append(QPointF(200.0, 100.0))
5 polygon.append(QPointF(100.0, 200.0))
6
7 polygonItem = QgsLayoutItemPolygon(polygon, layout)
8 layout.addLayoutItem(polygonItem)
9
10 props = {}
11 props["color"] = "green"
12 props["style"] = "solid"
13 props["style_border"] = "solid"
14 props["color_border"] = "black"
15 props["width_border"] = "10.0"
16 props["joinstyle"] = "miter"
17
18 symbol = QgsFillSymbol.createSimple(props)
19 polygonItem.setSymbol(symbol)
```

조판에 항목을 추가하고 나면, 항목을 이동시키고 크기를 조정할 수 있습니다:

```
item.attemptMove(QgsLayoutPoint(1.4, 1.8, QgsUnitTypes.LayoutCentimeters))
item.attemptResize(QgsLayoutSize(2.8, 2.2, QgsUnitTypes.LayoutCentimeters))
```

기본적으로 각 항목 주위에 프레임을 그립니다. 이 프레임을 다음과 같이 제거할 수 있습니다:

```
# for a composer label
label.setFrameEnabled(False)
```

조판 항목을 직접 생성하는 방법 이외에, QGIS 는 본질적으로 .qpt 파일에 모든 조판 항목들을 (XML 문법을 사용해서) 저장한 조판 (composition) 인 조판 템플릿을 지원합니다.

이 조판이 준비되고 나면 (조판 항목들을 생성해서 조판에 추가하고 나면) 래스터 그리고/또는 벡터 산출물 생성을 시작할 수 있습니다.

10.3.1 조판 무결성 검증하기

조판은 상호 연결된 항목들의 집합으로 이루어져 있는데, 수정 작업 동안 이 연결이 깨지는 경우 (제거한 맵에 연결된 범례, 누락된 소스 파일을 가진 이미지 항목 등등) 가 있을 수도 있고, 또는 여러분이 조판 항목에 사용자 정의 제약조건을 적용시키고 싶을 수도 있습니다. 이런 경우 `QgsAbstractValidityCheck` 클래스가 도움이 될 것입니다.

기본 검증 코드는 다음처럼 보일 겁니다:

```
@check.register(type=QgsAbstractValidityCheck.TypeLayoutCheck)
def my_layout_check(context, feedback):
    results = ...
    return results
```

다음은 조판 맵 항목이 웹 메르카토르 투영법으로 설정되어 있을 때마다 경고 메시지를 띄우는 검증 코드의 예시입니다:

```
1 @check.register(type=QgsAbstractValidityCheck.TypeLayoutCheck)
2 def layout_map_crs_choice_check(context, feedback):
3     layout = context.layout
4     results = []
5     for i in layout.items():
6         if isinstance(i, QgsLayoutItemMap) and i.crs().authid() == 'EPSG:3857':
7             res = QgsValidityCheckResult()
8             res.type = QgsValidityCheckResult.Warning
9             res.title = 'Map projection is misleading'
10            res.detailedDescription = 'The projection for the map item {} is set to <i>Web_
11            ↳Mercator (EPSG:3857)</i> which misrepresents areas and shapes. Consider using an_
12            ↳appropriate local projection instead.'.format(i.displayName())
13            results.append(res)
14
15    return results
```

그리고 다음은 더 복잡한 예시입니다. 이 코드는 조판 맵 항목이 해당 맵 항목의 범위 바깥에서만 무결한 좌표계로 설정되어 있는 경우 경고 메시지를 띄웁니다:

```
1 @check.register(type=QgsAbstractValidityCheck.TypeLayoutCheck)
2 def layout_map_crs_area_check(context, feedback):
3     layout = context.layout
4     results = []
5     for i in layout.items():
6         if isinstance(i, QgsLayoutItemMap):
7             bounds = i.crs().bounds()
8             ct = QgsCoordinateTransform(QgsCoordinateReferenceSystem('EPSG:4326'), i.
9             ↳crs(), QgsProject.instance())
10            bounds_crs = ct.transformBoundingBox(bounds)
11
12            if not bounds_crs.contains(i.extent()):
13                res = QgsValidityCheckResult()
14                res.type = QgsValidityCheckResult.Warning
15                res.title = 'Map projection is incorrect'
16                res.detailedDescription = 'The projection for the map item {} is set_
17                ↳to \'{}\', which is not valid for the area displayed within the map.'.format(i.
18                ↳displayName(), i.crs().authid())
19                results.append(res)
20
21    return results
```

10.3.2 조판 내보내기

조판을 내보내려면, 반드시 `QgsLayoutExporter` 클래스를 사용해야만 합니다.

```
1 base_path = os.path.join(QgsProject.instance().homePath())
2 pdf_path = os.path.join(base_path, "output.pdf")
3
4 exporter = QgsLayoutExporter(layout)
5 exporter.exportToPdf(pdf_path, QgsLayoutExporter.PdfExportSettings())
```

여러분이 조판을 SVG 또는 이미지 파일로 아니면 PDF 파일로 내보내고 싶은 경우 각각 `exportToSvg()` 아니면 `exportToImage()` 메소드를 사용하십시오.

10.3.3 조판 지도책 내보내기

지도책 (atlas) 옵션이 환경설정되어 있고 활성화된 조판에서 모든 페이지를 내보내고자 하는 경우, 내보내기 작업자 (`QgsLayoutExporter` 클래스) 에서 `atlas()` 메소드를 약간 조정해서 사용해야 합니다. 다음 예시는 페이지들을 PNG 이미지로 내보내는 코드입니다:

```
exporter.exportToImage(layout.atlas(), base_path, 'png', QgsLayoutExporter.
↪ ImageExportSettings())
```

산출물이 기본 경로 폴더에 지도책에 대해 환경설정된 산출 파일 이름 표현식을 사용해서 저장될 것이라는 사실을 기억하십시오.

값을 필터링하고 계산하는 표현식

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```

1 from qgis.core import (
2     edit,
3     QgsExpression,
4     QgsExpressionContext,
5     QgsFeature,
6     QgsFeatureRequest,
7     QgsField,
8     QgsFields,
9     QgsVectorLayer,
10    QgsPointXY,
11    QgsGeometry,
12    QgsProject,
13    QgsExpressionContextUtils
14 )

```

QGIS 는 SQL 계열 표현식의 파싱을 일부분 지원합니다. SQL 문법 가운데 작은 부분만을 지원합니다. (True 또는 False 를 반환하는) 불 술어 (boolean predicate) 또는 (스칼라 값을 반환하는) 함수 가운데 하나로 표현식을 평가할 수 있습니다. 사용할 수 있는 함수의 전체 목록을 보고 싶다면 사용자 지침서에 있는 `vector_expressions` 을 참조하세요.

다음 기본 유형 3 개를 지원합니다:

- 숫자—정수와 십진수, 예: 123, 3.14
- 문자열—'hello world' 처럼 작은 따옴표로 둘러싸야 합니다.
- 열 (column) 참조—표현식을 평가할 때, 참조를 필드의 실제 값으로 대체합니다. 이름은 이스케이프시키지 않습니다.

다음과 같은 연산자들을 사용할 수 있습니다:

- 산술 연산자: +, -, *, /, ^
- 괄호: (1 + 1) * 3 처럼 연산의 우선 순위를 강제합니다.
- 단항 플러스 및 마이너스: -12, +5
- 수학 함수: sqrt, sin, cos, tan, asin, acos, atan
- 변환 함수: to_int, to_real, to_string, to_date
- 도형 함수: \$area, \$length
- 도형 처리 함수: \$x, \$y, \$geometry, num_geometries, centroid

다음과 같은 술어들을 지원합니다:

- 비교: =, !=, >, >=, <, <=
- 패턴 매칭: LIKE (% 와 _ 사용), ~ (정규 표현식)
- 논리 술어: AND, OR, NOT
- NULL 값 검증: IS NULL, IS NOT NULL

술어의 예:

- 1 + 2 = 3
- sin(angle) > 0
- 'Hello' LIKE 'He%'
- (x > 10 AND y > 10) OR z = 0

스칼라 표현식의 예:

- 2 ^ 10
- sqrt(val)
- \$length + 1

11.1 표현식 파싱하기

다음은 지정한 표현식을 정확하게 파싱할 수 있는지 여부를 검증하는 방법을 보여주는 예시입니다:

```

1 exp = QgsExpression('1 + 1 = 2')
2 assert(not exp.hasParserError())
3
4 exp = QgsExpression('1 + 1 = ')
5 assert(exp.hasParserError())
6
7 assert(exp.parserErrorMessage() == '\nIncomplete expression. You might not have
  ↳ finished the full expression.')
```

11.2 표현식 평가하기

서로 다른 맥락에서, 예를 들면 피처를 필터링하기 위해 또는 새 필드 값을 계산하기 위해 표현식을 사용할 수 있습니다. 어떤 경우라도, 표현식을 평가해야 합니다. 즉 단순 산술 표현식에서 집계 표현식까지 아우를 수 있는 범위에서, 지정된 계산 단계를 따라 그 값을 계산한다는 뜻입니다.

11.2.1 기본 표현식

다음 기본 표현식은 단순 산술 연산을 평가합니다:

```
exp = QgsExpression('2 * 3')
print(exp)
print(exp.evaluate())
```

```
<QgsExpression: '2 * 3'>
6
```

비교에도 1(True) 또는 0(False) 으로 평가하는 표현식을 사용할 수 있습니다:

```
exp = QgsExpression('1 + 1 = 2')
exp.evaluate()
# 1
```

11.2.2 피처를 사용하는 표현식

피처에 대한 표현식을 평가하려면, 표현식이 피처의 필드 값에 접근할 수 있도록 `QgsExpressionContext` 클래스 객체를 생성해서 평가 함수로 전달해야 합니다.

다음은 “Column”이라는 필드를 가진 피처를 생성하는 방법과 표현식 맥락에서 해당 피처를 추가하는 방법을 보여주는 예시입니다:

```
1 fields = QgsFields()
2 field = QgsField('Column')
3 fields.append(field)
4 feature = QgsFeature()
5 feature.setFields(fields)
6 feature.setAttribute(0, 99)
7
8 exp = QgsExpression('"Column"')
9 context = QgsExpressionContext()
10 context.setFeature(feature)
11 exp.evaluate(context)
12 # 99
```

다음은 새 필드 값을 계산하기 위해 벡터 레이어 맥락에서 표현식을 사용하는 방법을 보여주는 좀 더 복잡한 예시입니다:

```
1 from qgis.PyQt.QtCore import QMetaType
2
3 # create a vector layer
4 vl = QgsVectorLayer("Point", "Companies", "memory")
5 pr = vl.dataProvider()
6 pr.addAttributes([QgsField("Name", QMetaType.Type.QString),
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

7         QgsField("Employees", QMetaType.Type.Int),
8         QgsField("Revenue", QMetaType.Type.Double),
9         QgsField("Rev. per employee", QMetaType.Type.Double),
10        QgsField("Sum", QMetaType.Type.Double),
11        QgsField("Fun", QMetaType.Type.Double)])
12 vl.updateFields()
13
14 # add data to the first three fields
15 my_data = [
16     {'x': 0, 'y': 0, 'name': 'ABC', 'emp': 10, 'rev': 100.1},
17     {'x': 1, 'y': 1, 'name': 'DEF', 'emp': 2, 'rev': 50.5},
18     {'x': 5, 'y': 5, 'name': 'GHI', 'emp': 100, 'rev': 725.9}]
19
20 for rec in my_data:
21     f = QgsFeature()
22     pt = QgsPointXY(rec['x'], rec['y'])
23     f.setGeometry(QgsGeometry.fromPointXY(pt))
24     f.setAttributes([rec['name'], rec['emp'], rec['rev']])
25     pr.addFeature(f)
26
27 vl.updateExtents()
28 QgsProject.instance().addMapLayer(vl)
29
30 # The first expression computes the revenue per employee.
31 # The second one computes the sum of all revenue values in the layer.
32 # The final third expression doesn't really make sense but illustrates
33 # the fact that we can use a wide range of expression functions, such
34 # as area and buffer in our expressions:
35 expression1 = QgsExpression('"Revenue"/"Employees"')
36 expression2 = QgsExpression('sum("Revenue")')
37 expression3 = QgsExpression('area(buffer($geometry,"Employees"))')
38
39 # QgsExpressionContextUtils.globalProjectLayerScopes() is a convenience
40 # function that adds the global, project, and layer scopes all at once.
41 # Alternatively, those scopes can also be added manually. In any case,
42 # it is important to always go from "most generic" to "most specific"
43 # scope, i.e. from global to project to layer
44 context = QgsExpressionContext()
45 context.appendScopes(QgsExpressionContextUtils.globalProjectLayerScopes(vl))
46
47 with edit(vl):
48     for f in vl.getFeatures():
49         context.setFeature(f)
50         f['Rev. per employee'] = expression1.evaluate(context)
51         f['Sum'] = expression2.evaluate(context)
52         f['Fun'] = expression3.evaluate(context)
53         vl.updateFeature(f)
54
55 print(f['Sum'])

```

876.5

11.2.3 표현식으로 레이어 필터링하기

레이어를 필터링해서 술어와 일치하는 모든 피처를 반환하는 데 다음 예시를 사용할 수 있습니다:

```

1 layer = QgsVectorLayer("Point?field=Test:integer",
2                       "addfeat", "memory")
3
4 layer.startEditing()
5
6 for i in range(10):
7     feature = QgsFeature()
8     feature.setAttributes([i])
9     assert(layer.addFeature(feature))
10 layer.commitChanges()
11
12 expression = 'Test >= 3'
13 request = QgsFeatureRequest().setFilterExpression(expression)
14
15 matches = 0
16 for f in layer.getFeatures(request):
17     matches += 1
18
19 print(matches)

```

7

11.3 표현식 오류 처리하기

표현식을 파싱하거나 평가하는 동안 표현식 관련 오류가 발생할 수 있습니다:

```

1 exp = QgsExpression("1 + 1 = 2")
2 if exp.hasParserError():
3     raise Exception(exp.parserErrorString())
4
5 value = exp.evaluate()
6 if exp.hasEvalError():
7     raise ValueError(exp.evalErrorString())

```


CHAPTER 12

읽기 및 저장하기 설정

💡 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (  
2     QgsProject,  
3     QgsSettings,  
4     QgsVectorLayer  
5 )
```

플러그인에 몇몇 변수를 저장하는 것이 유용한 경우가 많습니다. 여러분이 다음에 해당 플러그인을 실행할 때 변수들을 다시 입력하거나 선택하지 않아도 되기 때문입니다.

Qt 및 QGIS API 덕분에 이런 변수들을 저장하고 가져올 수 있습니다. 각 변수에 대해, 해당 변수에 접근하는 데 사용할 키를 선택해야 합니다—예를 들면 여러분이 가장 선호하는 색상에 대해 “favourite_color” 또는 비슷한 의미를 가진 다른 어떤 문자열도 사용할 수 있습니다. 키를 명명할 때 어떤 기준을 따를 것을 권장합니다.

설정을 다음 몇 가지 서로 다른 유형으로 구분할 수 있습니다:

- **전체 수준 설정 (global settings)** —이 설정은 특정 머신의 사용자에게 종속됩니다. QGIS 자체도 수많은, 예를 들면 메인 창의 크기 또는 기본 스냅 작업 허용 오차와 같은 전체 수준 설정을 저장합니다. 예를 들어 `setValue()` 및 `value()` 메소드를 통해 `QgsSettings` 클래스를 사용해서 설정을 처리합니다.

다음은 이런 메소드들을 어떻게 사용하는지 보여주는 예시입니다.

```
1 def store():  
2     s = QgsSettings()  
3     s.setValue("myplugin/mytext", "hello world")  
4     s.setValue("myplugin/myint", 10)  
5     s.setValue("myplugin/myreal", 3.14)  
6  
7 def read():  
8     s = QgsSettings()
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

9  mytext = s.value("myplugin/mytext", "default text")
10 myint  = s.value("myplugin/myint", 123)
11 myreal = s.value("myplugin/myreal", 2.71)
12 nonexistent = s.value("myplugin/nonexistent", None)
13 print(mytext)
14 print(myint)
15 print(myreal)
16 print(nonexistent)

```

`value()` 메소드의 두 번째 파라미터는 선택적으로, 전달된 설정 이름에 대해 이전에 설정한 값이 없을 경우 반환하는 기본값을 지정합니다.

`qgis_global_settings.ini` 파일을 통해 전체 수준 설정들의 기본값을 사전 환경설정할 수 있는 메소드에 대해 알고 싶다면 `deploying_organization` 를 참조하십시오.

- **프로젝트 수준 설정**—프로젝트에 따라 달라지기 때문에 프로젝트 파일과 연동됩니다. 프로젝트 수준 설정의 예시로는 맵 캔버스 배경 색상 또는 대상 좌표계 등을 들 수 있습니다. 어떤 프로젝트에서는 하얀 배경과 WGS84 가 어울리는 반면 다른 프로젝트에서는 노란 배경과 UTM 투영 좌표계가 어울리는 경우가 있는 법이죠.

다음은 프로젝트 수준 설정 사용의 예시입니다.

```

1  proj = QgsProject.instance()
2
3  # store values
4  proj.writeEntry("myplugin", "mytext", "hello world")
5  proj.writeEntry("myplugin", "myint", 10)
6  proj.writeEntryDouble("myplugin", "mydouble", 0.01)
7  proj.writeEntryBool("myplugin", "mybool", True)
8
9  # read values (returns a tuple with the value, and a status boolean
10 # which communicates whether the value retrieved could be converted to
11 # its type, in these cases a string, an integer, a double and a boolean
12 # respectively)
13
14 mytext, type_conversion_ok = proj.readEntry("myplugin",
15                                             "mytext",
16                                             "default text")
17 myint, type_conversion_ok = proj.readNumEntry("myplugin",
18                                              "myint",
19                                              123)
20 mydouble, type_conversion_ok = proj.readDoubleEntry("myplugin",
21                                                     "mydouble",
22                                                     123)
23 mybool, type_conversion_ok = proj.readBoolEntry("myplugin",
24                                                  "mybool",
25                                                  123)

```

여러분도 알 수 있듯이, 많은(정수, 문자열, 목록) 데이터 유형에 대해 `writeEntry()` 메소드를 사용하지만, 설정값을 다시 읽어오기 위한 메소드가 몇 가지 존재하며 각 데이터 유형에 대응하는 메소드를 선택해야 합니다.

- **맵 레이어 수준 설정**—프로젝트의 맵 레이어 하나의 특정 인스턴스와 관련된 설정입니다. 이 설정은 레이어의 기저 데이터 소스와 연결되지 않기 때문에, 셰이프파일(shapefile) 1 개의 맵 레이어 인스턴스를 2 개 생성하는 경우 이 두 인스턴스는 설정을 공유하지 않을 것입니다. 맵 레이어 수준 설정은 프로젝트 파일 내부에 저장되기 때문에 여러분이 프로젝트를 다시 열면 레이어 관련 설정도 그대로 다시 불러올 것입니다. `customProperty()` 메소드를 사용해서 지정한 설정의 값을 검색하고, `setCustomProperty()` 메소드를 사용해서 설정할 수 있습니다.


```
1 vlayer = QgsVectorLayer()  
2 # save a value  
3 vlayer.setCustomProperty("mytext", "hello world")  
4  
5 # read the value again (returning "default text" if not found)  
6 mytext = vlayer.customProperty("mytext", "default text")
```


사용자에게 정보 전달하기

힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (  
2     QgsMessageLog,  
3     QgsGeometry,  
4 )  
5  
6 from qgis.gui import (  
7     QgsMessageBar,  
8 )  
9  
10 from qgis.PyQt.QtWidgets import (  
11     QSizePolicy,  
12     QPushButton,  
13     QDialog,  
14     QGridLayout,  
15     QDialogButtonBox,  
16 )
```

이 장에서는 사용자 인터페이스에서 일관성을 유지하기 위해 사용자에게 정보를 전달하는 데 사용해야 하는 몇몇 메소드 및 요소에 대해 설명합니다.

13.1 메시지 보이기: QgsMessageBar 클래스

메시지 상자를 사용하는 것은 사용자 경험이라는 관점에서 그리 좋은 생각이 아닙니다. 짧은 정보 또는 경고/오류 메시지를 표출하는 데에는 QGIS 메시지 바를 사용하는 편이 더 낫습니다.

QGIS 인터페이스 객체에 참조 (reference) 를 사용하면, 메시지 바에 메시지를 다음 코드로 표시할 수 있습니다:

```
from qgis.core import Qgs
iface.messageBar().pushMessage("Error", "I'm sorry Dave, I'm afraid I can't do that",
    ↳level=Qgis.Critical)
```

```
Messages(2): Error : I'm sorry Dave, I'm afraid I can't do that
```

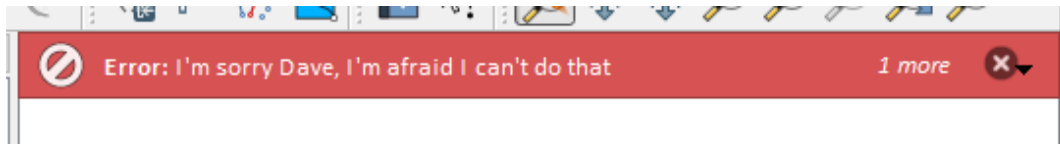


그림 13.1: QGIS 메시지 바

제한 시간 동안에만 메시지를 보이려면 지속 시간 (duration) 을 설정하면 됩니다.

```
iface.messageBar().pushMessage("Oops", "The plugin is not working as it should",
    ↳level=Qgis.Critical, duration=3)
```

```
Messages(2): Oops : The plugin is not working as it should
```

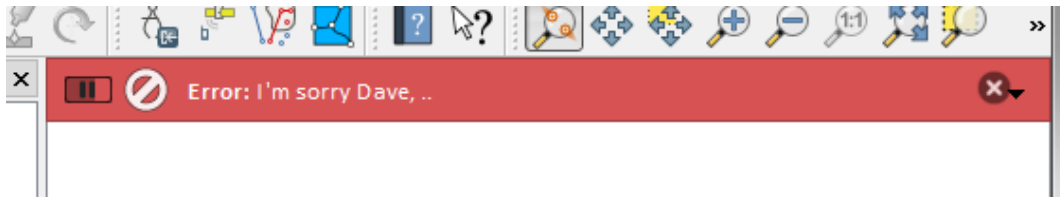


그림 13.2: 타이머를 적용한 QGIS 메시지 바

앞의 예시들은 오류 메시지를 보이지만, level 파라미터를 사용하면 Qgs.MessageLevel 클래스의 열거 목록 (enumeration) 을 통해 경고 또는 정보 메시지를 생성할 수 있습니다. 모두 4 개의 서로 다른 수준을 사용할 수 있습니다:

0. 정보 (Info)
1. 경고 (Warning)
2. 중요 (Critical)
3. 성공 (Success)

추가적인 정보를 표시하기 위한 버튼 등의 위젯을 메시지 바에 추가할 수도 있습니다:

```
1 def showError():
2     pass
3
4 widget = iface.messageBar().createMessage("Missing Layers", "Show Me")
5 button = QPushButton(widget)
```

(다음 페이지에 계속)

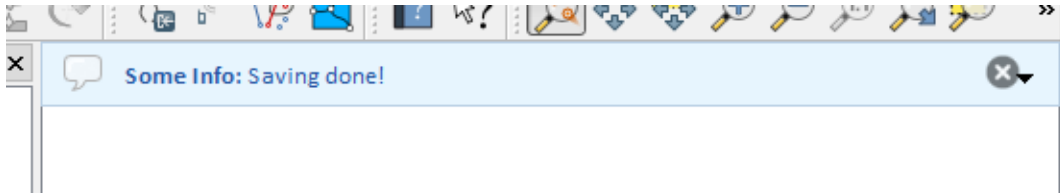


그림 13.3: QGIS 메시지 바 (정보)

(이전 페이지에서 계속)

```

6 button.setText("Show Me")
7 button.pressed.connect(showError)
8 widget.layout().addWidget(button)
9 iface.messageBar().pushWidget(widget, Qgis.Warning)

```

Messages(1): Missing Layers : Show Me

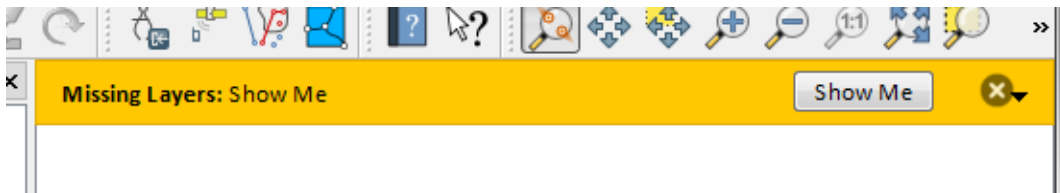


그림 13.4: 버튼이 들어간 QGIS 메시지 바

별도의 메시지 상자가 필요 없거나 QGIS 메인 창에 메시지를 표시할 이유가 없을 경우, 사용자의 대화창에 메시지 바를 표시할 수도 있습니다.

```

1 class MyDialog(QDialog):
2     def __init__(self):
3         QDialog.__init__(self)
4         self.bar = QgsMessageBar()
5         self.bar.setSizePolicy(QSizePolicy.Minimum, QSizePolicy.Fixed)
6         self.setLayout(QGridLayout())
7         self.layout().setContentsMargins(0, 0, 0, 0)
8         self.buttonbox = QDialogButtonBox(QDialogButtonBox.Ok)
9         self.buttonbox.accepted.connect(self.run)
10        self.layout().addWidget(self.buttonbox, 0, 0, 2, 1)
11        self.layout().addWidget(self.bar, 0, 0, 1, 1)
12    def run(self):
13        self.bar.pushMessage("Hello", "World", level=Qgis.Info)
14
15 myDlg = MyDialog()
16 myDlg.show()

```

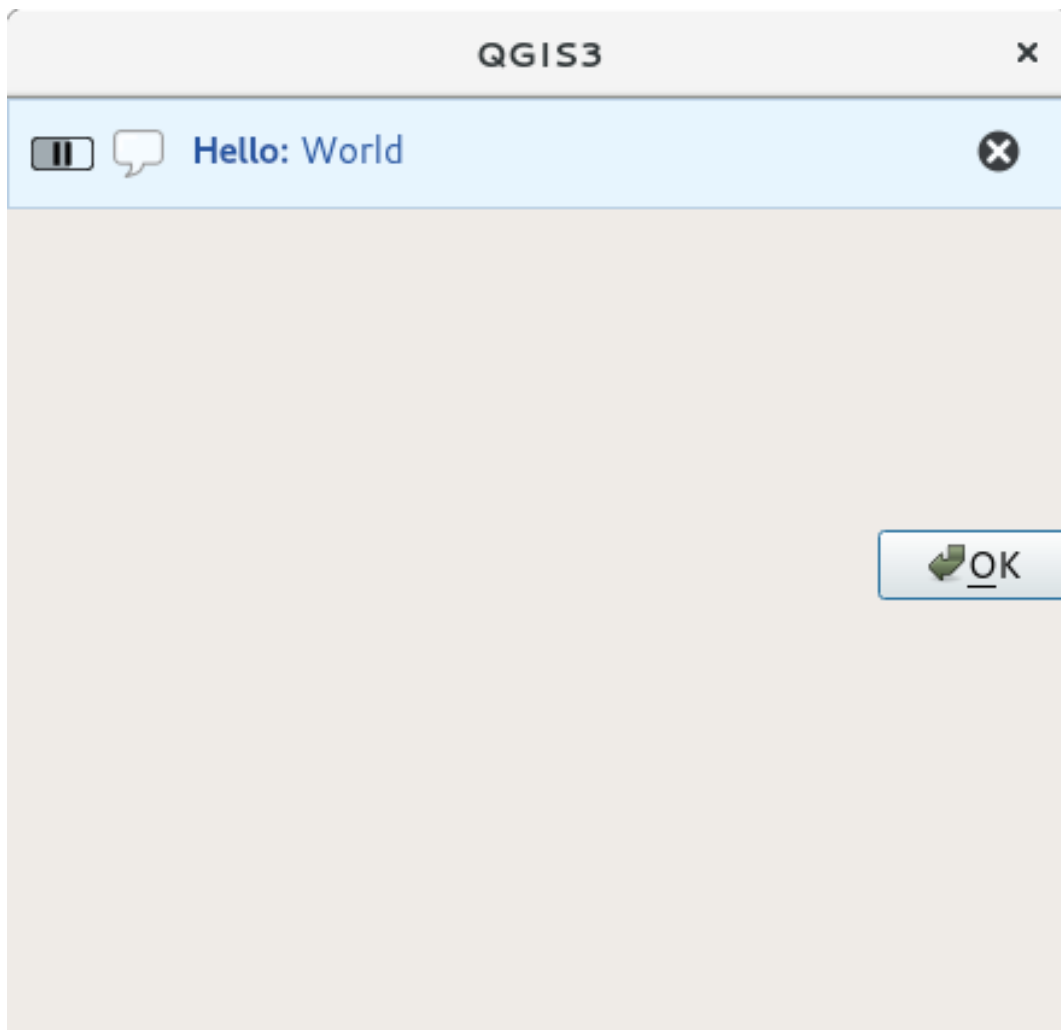


그림 13.5: 사용자 정의 대화창 내부의 QGIS 메시지 바

13.2 진행률 보이기

앞에서 배웠듯이 QGIS 메시지 바에 위젯을 추가할 수 있으므로, 진행률 (progress) 막대도 추가할 수 있습니다. 다음은 여러분이 콘솔에서 실행해볼 수 있는 예시 코드입니다.

```
1 import time
2 from qgis.PyQt.QtWidgets import QProgressBar
3 from qgis.PyQt.QtCore import *
4 progressMessageBar = iface.messageBar().createMessage("Doing something boring...")
5 progress = QProgressBar()
6 progress.setMaximum(10)
7 progress.setAlignment(Qt.AlignLeft|Qt.AlignVCenter)
8 progressMessageBar.layout().addWidget(progress)
9 iface.messageBar().pushWidget(progressMessageBar, Qgis.Info)
10
11 for i in range(10):
12     time.sleep(1)
13     progress.setValue(i + 1)
14
15 iface.messageBar().clearWidgets()
```

Messages(0): Doing something boring...

또한 다음 예시에서와 같이 진행률을 보고하는 내장 상태 바도 사용할 수 있습니다:

```
1 vlayer = iface.activeLayer()
2
3 count = vlayer.featureCount()
4 features = vlayer.getFeatures()
5
6 for i, feature in enumerate(features):
7     # do something time-consuming here
8     print('.') # printing should give enough time to present the progress
9
10    percent = i / float(count) * 100
11    # iface.mainWindow().statusBar().showMessage("Processed {} %".
    ↳ format(int(percent)))
12    iface.statusBarIface().showMessage("Processed {} %".format(int(percent)))
13
14 iface.statusBarIface().clearMessage()
```

13.3 로그 작성하기

여러분의 코드를 실행하는 것에 대한 모든 정보를 로그로 작성하고 저장할 수 있도록, QGIS 는 세 가지의 서로 다른 로그 작업 유형을 지원합니다. 각 유형은 전용 산출 위치를 가지고 있습니다. 여러분의 목적에 따라 올바른 로그 작업 방식을 사용해주십시오:

- `QgsMessageLog` 는 사용자에게 문제점을 전달하기 위한 메시지 용 클래스입니다. `QgsMessageLog` 클래스의 산출물은 로그 메시지 패널에 나타납니다.
- 파이썬 내장 **logging** 모듈은 QGIS 파이썬 API (PyQGIS) 수준에서 디버그 작업을 하기 위한 모듈입니다. 예를 들면 피쳐 ID 또는 도형 같은 파이썬 코드를 디버그해야 하는 파이썬 스크립트 개발자들에게 권장합니다.
- `QgsLogger` 는 (예를 들어 어떤 깨진 코드가 뭔가를 촉발한다고 의심하는 경우) QGIS 내부 디버그 작업 또는 개발자 용 클래스입니다. 이 메시지는 QGIS 개발자 버전에서만 보일 것입니다.

다음 절들에서 이 서로 다른 로그 작업 유형들의 예시를 볼 수 있습니다.

⚠ 경고

파이썬 `print` 선언문을 사용하면 멀티스레드일 수도 있는 모든 코드를 안전하지 못 하게 만들고 알고리즘을 극단적으로 느리게 합니다. 이때 모든 코드란 (다른 것들 가운데에서도) 표현식 함수, 렌더링 작업자, 심볼 레이어, 그리고 공간 처리 알고리즘 을 포함합니다. 이런 경우 항상 파이썬 로그 작업 모듈 또는 스레드 안전 클래스들 (`QgsLogger` 또는 `QgsMessageLog`) 을 대신 사용해야 합니다.

13.3.1 QgsMessageLog

```
# You can optionally pass a 'tag' and a 'level' parameters
QgsMessageLog.logMessage("Your plugin code has been executed correctly", 'MyPlugin', ↵
↵ level=Qgis.Info)
QgsMessageLog.logMessage("Your plugin code might have some problems", level=Qgis.
↵ Warning)
QgsMessageLog.logMessage("Your plugin code has crashed!", level=Qgis.Critical)
```

```
MyPlugin(0): Your plugin code has been executed correctly
(1): Your plugin code might have some problems
(2): Your plugin code has crashed!
```

i 참고

`log_message_panel` 에서 `QgsMessageLog` 클래스의 산출물을 볼 수 있습니다.

13.3.2 파이썬 내장 로그 작업 모듈

```
1 import logging
2 formatter = '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
3 logfilename=r'c:\temp\example.log'
4 logging.basicConfig(filename=logfilename, level=logging.DEBUG, format=formatter)
5 logging.info("This logging info text goes into the file")
6 logging.debug("This logging debug text goes into the file as well")
```

`basicConfig` 메소드가 로그 작업의 기본 설정을 환경설정합니다. 앞의 코드에서 파일 이름, 로그 작업 수준 및 서식을 정의했습니다. 파일 이름은 로그 파일을 작성할 위치를 알려주며, 로그 작업 수준은 산출할 수준을 정의하고, 서식은 각 메시지를 산출할 서식을 정의합니다.

```
2020-10-08 13:14:42,998 - root - INFO - This logging text goes into the file
2020-10-08 13:14:42,998 - root - DEBUG - This logging debug text goes into the file.↵
↵ as well
```

여러분이 여러분의 스크립트를 실행할 때마다 로그 파일을 지우길 바라는 경우, 다음과 같은 코드를 작성하면 됩니다:

```
if os.path.isfile(logfilename):
    with open(logfilename, 'w') as file:
        pass
```


다음 링크에서 파이썬 로그 작업 기능을 사용하는 방법에 대한 심화 자료들을 찾아볼 수 있습니다:

- <https://docs.python.org/3/library/logging.html>
- <https://docs.python.org/3/howto/logging.html>
- <https://docs.python.org/3/howto/logging-cookbook.html>

 경고

파일 이름을 설정해서 파일에 로그를 작성하지 않는다면 로그 작업이 산출 속도를 크게 떨어뜨리는 멀티스레드가 될 수도 있다는 사실을 기억하십시오.

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (  
2     QgsApplication,  
3     QgsRasterLayer,  
4     QgsAuthMethodConfig,  
5     QgsDataSourceUri,  
6     QgsPkiBundle,  
7     QgsMessageLog,  
8 )  
9  
10 from qgis.gui import (  
11     QgsAuthAuthoritiesEditor,  
12     QgsAuthConfigEditor,  
13     QgsAuthConfigSelect,  
14     QgsAuthSettingsWidget,  
15 )  
16  
17 from qgis.PyQt.QtWidgets import (  
18     QWidget,  
19     QTabWidget,  
20 )  
21  
22 from qgis.PyQt.QtNetwork import QSslCertificate
```

14.1 소개

사용자 지침서의 `authentication_overview` 문단에서 인증 인프라스트럭처의 사용자 참조 사항들을 읽어볼 수 있습니다.

이 장에서는 개발자 입장에서 이 인증 시스템을 가장 잘 활용할 수 있는 방법을 설명할 것입니다.

어떤 특정 리소스에 접근하려면 인증 정보가 필요할 때마다, 예를 들어 PostgreSQL 데이터베이스에 어떤 레이어를 연결할 때마다, 데이터 제공자는 QGIS 데스크탑에서 인증 시스템을 광범위하게 사용합니다.

플러그인 개발자가 자신의 코드에 인증 인프라스트럭처를 손쉽게 통합할 수 있도록 해주는 위젯 몇 개가 QGIS GUI 라이브러리에 존재합니다:

- `QgsAuthConfigEditor`
- `QgsAuthConfigSelect`
- `QgsAuthSettingsWidget`

인증 인프라스트럭처 테스트 코드에서 괜찮은 코드 참조 사례를 볼 수 있습니다.

⚠ 경고

인증 인프라스트럭처 설계 시 도입된 보안 제약조건들 때문에, 파이썬에서는 내부 메소드들 가운데 일부 하위 집합만을 사용할 수 있습니다.

14.2 용어 해설

이 장에서 가장 자주 취급하는 객체들을 몇 개 정의하겠습니다.

마스터 비밀번호

QGIS 인증 데이터베이스에 저장되어 있는 인증 정보 (credential) 에 접근해서 복호화할 수 있게 해주는 비밀번호입니다.

인증 데이터베이스

A *Master Password* encrypted SQLite db `qgis-auth.db` where *Authentication Configuration* are stored. e.g user/password, personal certificates and keys, Certificate Authorities

인증 DB

Authentication Database

인증 환경설정 (Authentication Configuration)

Authentication Method 에 따라 달라지는 인증 데이터 집합입니다. 예를 들면 기본 인증 방법은 사용자 이름/비밀번호 쌍을 저장합니다.

인증 환경설정 (Authentication Config)

Authentication Configuration

인증 방법

인증을 받기 위해 쓰이는 특정 방법입니다. 각 방법은 인증 수준을 얻는 데 사용되는 고유 프로토콜을 가지고 있습니다. 각 방법은 QGIS 인증 인프라스트럭처 초기 설정 시 동적으로 불러오는 공유 라이브러리로 구현됩니다.

14.3 QgsAuthManager: 입구점

`QgsAuthManager` 클래스 단독 개체 (singleton) 는 예를 들어 활성 사용자 프로파일 폴더 아래 있는 `qgis-auth.db` 파일 같은 QGIS 암호화 *Authentication DB* 에 저장되어 있는 인증 정보를 사용하기 위한 입구점 (entry point) 입니다.

이 클래스는 저장된 암호화 정보에 접근하기 위해 마스터 비밀번호를 설정하라고 요청하거나 또는 마스터 비밀번호를 투명하게 사용하는 방식으로 사용자의 쌍방향 작업을 처리합니다.

14.3.1 관리자 초기 설정 및 마스터 비밀번호 설정하기

다음 코드 조각은 인증 설정에 접근하기 위한 마스터 비밀번호를 설정하는 예시입니다. 이 코드 조각을 이해하려면 코드 주석을 잘 읽어보십시오.

```

1 authMgr = QgsApplication.authManager()
2
3 # check if QgsAuthManager has already been initialized... a side effect
4 # of the QgsAuthManager.init() is that AuthDbPath is set.
5 # QgsAuthManager.init() is executed during QGIS application init and hence
6 # you do not normally need to call it directly.
7 if authMgr.authenticationDatabasePath():
8     # already initialized => we are inside a QGIS app.
9     if authMgr.masterPasswordIsSet():
10         msg = 'Authentication master password not recognized'
11         assert authMgr.masterPasswordSame("your master password"), msg
12     else:
13         msg = 'Master password could not be set'
14         # The verify parameter checks if the hash of the password was
15         # already saved in the authentication db
16         assert authMgr.setMasterPassword("your master password",
17                                         verify=True), msg
18 else:
19     # outside qgis, e.g. in a testing environment => setup env var before
20     # db init
21     os.environ['QGIS_AUTH_DB_DIR_PATH'] = "/path/where/located/qgis-auth.db"
22     msg = 'Master password could not be set'
23     assert authMgr.setMasterPassword("your master password", True), msg
24     authMgr.init("/path/where/located/qgis-auth.db")

```

14.3.2 새 인증 환경설정 항목으로 인증 DB 채우기

저장된 모든 인증 정보는 다음과 같은 유일한 문자열을 사용해서 접근하는 `QgsAuthMethodConfig` 클래스의 *Authentication Configuration* 인스턴스입니다:

```
authcfg = 'fm1s770'
```

QGIS API 또는 GUI 를 사용해서 환경설정 항목을 생성할 때 이런 문자열을 자동으로 생성하지만, 어떤 조직 내부에서 사용자 여러 명 사이에서 인증 환경설정을 (서로 다른 인증 정보를 사용해서) 공유해야만 하는 경우 이미 알고 있는 값으로 직접 설정하는 편이 유용할 수도 있습니다.

`QgsAuthMethodConfig` 클래스는 모든 *Authentication Method* 의 기저 클래스입니다. 모든 인증 방법은 인증 정보를 저장할 환경설정 해시 맵 (configuration hash map) 을 설정합니다. 다음은 `alice` 라는 가상의 사용자에 대한 PKI 경로 인증 정보를 저장할 수 있는 유용한 코드 조각입니다:

```

1 authMgr = QgsApplication.authManager()
2 # set alice PKI data
3 config = QgsAuthMethodConfig()
4 config.setName("alice")
5 config.setMethod("PKI-Paths")
6 config.setUri("https://example.com")
7 config.setConfig("certpath", "path/to/alice-cert.pem" )
8 config.setConfig("keypath", "path/to/alice-key.pem" )
9 # check if method parameters are correctly set
10 assert config.isValid()
11
12 # register alice data in authdb returning the ``authcfg`` of the stored
13 # configuration
14 authMgr.storeAuthenticationConfig(config)
15 newAuthCfgId = config.id()
16 assert newAuthCfgId

```

사용할 수 있는 인증 방법들

인증 관리자 초기 설정 시 *Authentication Method* 라이브러리를 동적으로 불러옵니다. 다음 인증 방법을 사용할 수 있습니다:

Method	설명	Target providers
Basic	User and password authentication	postgres, oracle, ows, wfs, wcs, wms, ogr, g
APIHeader	HTTP headers	ows, wfs, wcs, wms
AWSS3	AWS Simple Storage Service	awss3
EsriToken	ESRI token based authentication for ArcGIS REST servers	arcgismapserver, arcgisfeatureserver
Identity-Cert	신원 인증서 인증	ows, wfs, wcs, wms, postgres
MapTilerHmacSha256	MapTiler HMAC-SHA256	wms, vectortile, xyzvectortiles
OAuth2	OAuth2 authentication	ows, wfs, wcs, wms
PKI-Paths	PKI 경로 인증	ows, wfs, wcs, wms, postgres
PKI-PKCS#12	PKI PKCS#12 인증	ows, wfs, wcs, wms, postgres

기관 채우기

```

1 authMgr = QgsApplication.authManager()
2 # add authorities
3 cacerts = QSslCertificate.fromPath( "/path/to/ca_chains.pem" )
4 assert cacerts is not None
5 # store CA
6 authMgr.storeCertAuthorities(cacerts)
7 # and rebuild CA caches
8 authMgr.rebuildCaCertsCache()
9 authMgr.rebuildTrustedCaCertsCache()

```

QgsPkiBundle 을 사용해서 PKI 번들 관리하기

`QgsPkiBundle` 클래스는 `SSLCert`, `SSLKey` 및 `CA` 체인에 대해 구성된 PKI 번들을 패키징하는 편리한 클래스입니다. 다음은 비밀번호를 보호하기 위한 코드 조각입니다:

```
1 # add alice cert in case of key with pwd
2 caBundlesList = [] # List of CA bundles
3 bundle = QgsPkiBundle.fromPemPaths( "/path/to/alice-cert.pem",
4                                     "/path/to/alice-key_w-pass.pem",
5                                     "unlock_pwd",
6                                     caBundlesList )
7
8 assert bundle is not None
9 # You can check bundle validity by calling:
10 # bundle.isValid()
```

이 번들에서 인증서/키/인증 기관을 추출하려면 `QgsPkiBundle` 클래스 문서를 참조하세요.

14.3.3 인증 DB 에서 항목 제거하기

Authentication Database 의 `authcfg` 식별자를 다음 코드 조각처럼 사용하면 인증 데이터베이스에서 항목 하나를 제거할 수 있습니다:

```
authMgr = QgsApplication.authManager()
authMgr.removeAuthenticationConfig( "authCfg_Id_to_remove" )
```

14.3.4 QgsAuthManager 에 인증 환경설정 확장을 맡기기

Authentication DB 에 저장되어 있는 *Authentication Config* 을 사용하는 최선의 방법은 `authcfg` 식별자를 사용해서 인증 환경설정을 참조하는 것입니다. 확장이란 이 식별자 하나를 완전한 인증 정보 집합으로 변환하는 것을 말합니다. 저장되어 있는 *Authentication Config* 들을 사용하는 최선의 방법은 인증 관리자가 자동으로 관리하도록 내버려두는 것입니다. `WMS` 또는 `WFS` 처럼 인증이 활성화된 서비스 또는 데이터베이스에 연결하는 것이 저장된 환경설정을 사용하는 가장 흔한 방법입니다.

참고

모든 QGIS 데이터 제공자가 인증 인프라스트럭처와 통합되어 있지는 않다는 사실을 기억하십시오. `QgsAuthMethod` 기저 클래스에서 파생된 각 인증 방법은 서로 다른 제공자들을 지원합니다. 예를 들어 `certIdentity()` 메소드는 다음 제공자들을 지원합니다:

```
authM = QgsApplication.authManager()
print(authM.authMethod("Identity-Cert").supportedDataProviders())
```

이 예시의 산출물:

```
['ows', 'wfs', 'wcs', 'wms', 'postgres']
```

예를 들면, `authcfg = 'fm1s770'` 문자열로 식별되는 저장된 인증 정보를 사용해서 `WMS` 서비스에 접근하려는 경우 다음 코드 조각에서처럼 데이터 소스 URL 에 `authcfg` 를 사용하지만 하면 됩니다:

```
1 authCfg = 'fm1s770'
2 quri = QgsDataSourceUri()
3 quri.setParam("layers", 'usa:states')
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

4 quri.setParam("styles", '')
5 quri.setParam("format", 'image/png')
6 quri.setParam("crs", 'EPSG:4326')
7 quri.setParam("dpiMode", '7')
8 quri.setParam("featureCount", '10')
9 quri.setParam("authcfg", authCfg) # <---- here my authCfg url parameter
10 quri.setParam("contextualWMSLegend", '0')
11 quri.setParam("url", 'https://my_auth_enabled_server_ip/wms')
12 rlayer = QgsRasterLayer(str(quri.encodedUri(), "utf-8"), 'states', 'wms')

```

앞의 예시에서, wms 제공자가 HTTP 연결을 설정하기 직전에 authcfg URI 파라미터를 인증 정보를 사용해서 확장시킬 것입니다.

⚠ 경고

개발자는 authcfg 확장을 `QgsAuthManager` 클래스에 맡겨야 할 것입니다. 이렇게 하면 너무 일찍 확장되지 않을 것을 믿을 수 있기 때문입니다.

`QgsDataSourceURI` 클래스를 사용해서 작성된 URI 문자열은 보통 다음 방식으로 데이터 소스를 설정하는 데 쓰입니다:

```

authCfg = 'fmls770'
quri = QgsDataSourceUri("my WMS uri here")
quri.setParam("authcfg", authCfg)
rlayer = QgsRasterLayer( quri.uri(False), 'states', 'wms')

```

i 참고

`False` 파라미터가 URI에 존재하는 authcfg ID의 URI 완전 확장을 방지하는 데 중요한 역할을 합니다.

기타 데이터 제공자들의 PKI 예시

QGIS 테스트 업스트림에 있는 `test_authmanager_pki_ows` 또는 `test_authmanager_pki_postgres`에서 기타 예시를 직접 읽어볼 수 있습니다.

14.4 플러그인이 인증 인프라스트럭처를 사용하도록 조정하기

제 3자 플러그인 가운데 다수가 HTTP 연결을 관리하기 위해 `QgsNetworkAccessManager` 클래스 및 관련 통합 인증 인프라스트럭처와 통합하는 대신 `httplib2` 또는 다른 파이썬 네트워크 작업 라이브러리를 사용하고 있습니다.

이런 통합을 용이하게 하기 위해 `NetworkAccessManager`라는 파이썬 도우미 함수가 생성되었습니다. 여기에서 그 코드를 찾아볼 수 있습니다.

이 도우미 클래스를 다음 코드 조각에서처럼 사용할 수 있습니다:

```

1 http = NetworkAccessManager(authid="my_authCfg", exception_class=My_
  ↳ FailedRequestError)
2 try:
3     response, content = http.request( "my_rest_url" )

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

4 except My_FailedRequestError, e:
5     # Handle exception
6     pass

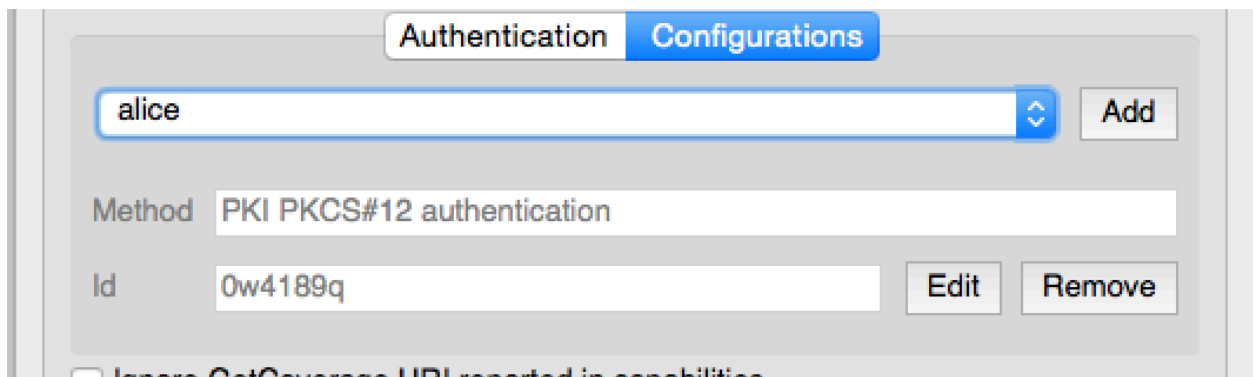
```

14.5 인증 GUI

이 절에서는 사용자 정의 인터페이스에 인증 인프라스트럭처를 통합하는 데 사용할 수 있는 유용한 GUI 들을 소개합니다.

14.5.1 인증 정보를 선택할 수 있는 GUI

Authentication DB 에 저장된 집합에서 *Authentication Configuration* 하나를 선택해야 하는 경우 *QgsAuthConfigSelect* GUI 클래스에서 선택할 수 있습니다.



다음 코드 조각에서와 같이 사용하면 됩니다:

```

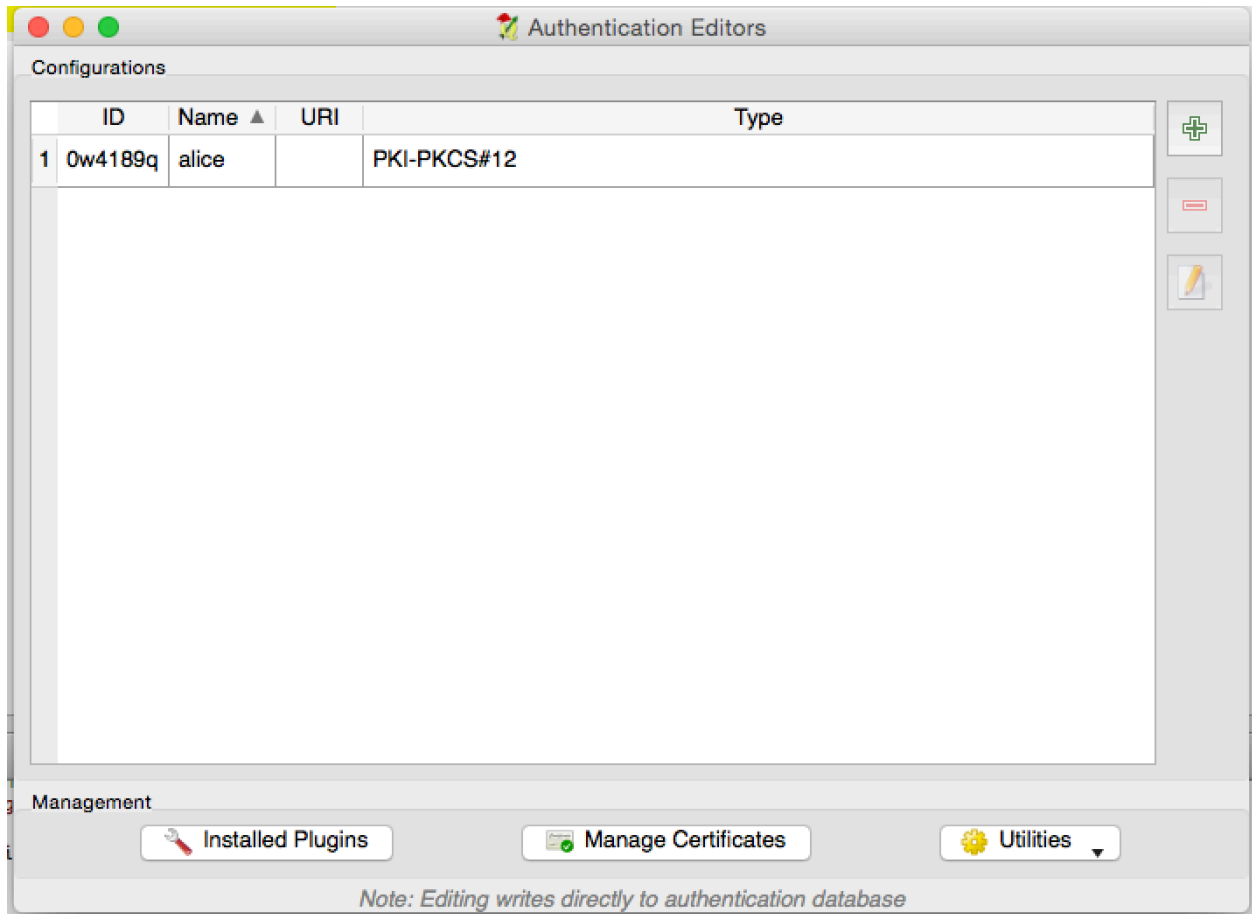
1 # create the instance of the QgsAuthConfigSelect GUI hierarchically linked to
2 # the widget referred with 'parent'
3 parent = QWidget() # Your GUI parent widget
4 gui = QgsAuthConfigSelect( parent, "postgres" )
5 # add the above created gui in a new tab of the interface where the
6 # GUI has to be integrated
7 tabGui = QTabWidget()
8 tabGui.insertTab( 1, gui, "Configurations" )

```

앞의 예시는 QGIS 소스 코드 에서 발췌한 것입니다. GUI 작성자의 두 번째 파라미터는 데이터 제공자 유형을 알려줍니다. 이 파라미터는 지정한 제공자와 호환되는 *Authentication Method* 들을 제한하기 위해 쓰입니다.

14.5.2 인증 편집기 GUI

`QgsAuthEditorWidgets` 클래스가 인증 정보, 기관을 관리하고 인증 유틸리티들에 접근하는 데 사용되는 완전한 GUI 를 관리합니다.



다음 코드 조각에서와 같이 사용하면 됩니다:

```

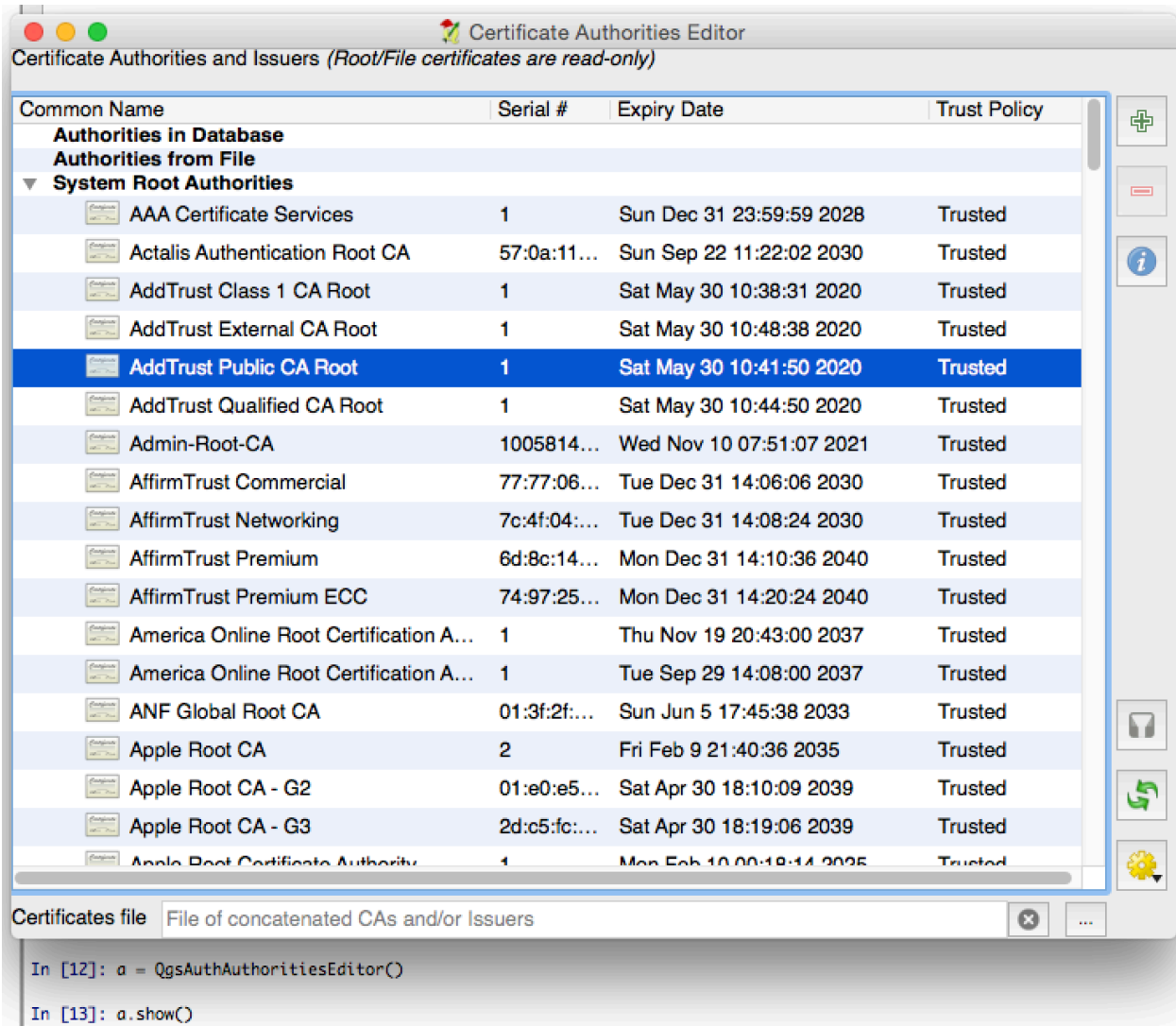
1 # create the instance of the QgsAuthEditorWidgets GUI hierarchically linked to
2 # the widget referred with `parent`
3 parent = QWidget() # Your GUI parent widget
4 gui = QgsAuthConfigSelect( parent )
5 gui.show()

```

관련 테스트 에서 통합된 예시를 찾아볼 수 있습니다.

14.5.3 기관 편집기 GUI

`QgsAuthAuthoritiesEditor` 클래스가 기관만을 관리하는 데 사용되는 GUI 를 관리합니다.



다음 코드 조각에서와 같이 사용하면 됩니다:

```

1 # create the instance of the QgsAuthAuthoritiesEditor GUI hierarchically
2 # linked to the widget referred with `parent`
3 parent = QWidget() # Your GUI parent widget
4 gui = QgsAuthAuthoritiesEditor( parent )
5 gui.show()

```


태스크 - 배경에서 무거운 작업 하기

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```

1 from qgis.core import (
2     Qgis,
3     QgsApplication,
4     QgsMessageLog,
5     QgsProcessingAlgRunnerTask,
6     QgsProcessingContext,
7     QgsProcessingFeedback,
8     QgsProject,
9     QgsTask,
10    QgsTaskManager,
11 )

```

15.1 소개

무거운 공간 처리 작업이 진행 중일 때, 스레드들을 사용한 배경 프로세스가 즉각 반응하는 사용자 인터페이스를 유지하는 한 방법일 수 있습니다. QGIS 에서는 스레드 작업을 태스크(task) 를 사용해서 달성할 수 있습니다.

태스크(`QgsTask` 클래스) 란 배경에서 수행될 코드를 위한 컨테이너이며, 태스크 관리자(`QgsTaskManager` 클래스) 는 태스크의 실행을 제어하는 데 쓰입니다. 이 클래스들은 신호 전달, 진행률 보고 및 배경 프로세스의 상태에 접근하기 위한 메커니즘을 제공해서 QGIS 에서의 배경 프로세스 작업을 단순화시켜줍니다. 태스크는 하위 태스크를 사용해서 그룹화할 수 있습니다.

일반적으로(`QgsApplication.taskManager()` 메소드로 호출할 수 있는) 전체 수준 태스크 관리자를 사용합니다. 다시 말해 태스크 관리자가 여러분의 태스크만 제어하는 것이 아닐 수도 있다는 뜻입니다.

몇 가지 방법으로 QGIS 태스크를 생성할 수 있습니다:

- `QgsTask` 클래스를 확장시켜 사용자의 태스크를 생성합니다:

```
class SpecialisedTask(QgsTask):
    pass
```

- 함수로부터 태스크를 생성합니다:

```
1 def heavyFunction():
2     # Some CPU intensive processing ...
3     pass
4
5 def workdone():
6     # ... do something useful with the results
7     pass
8
9 task = QgsTask.fromFunction('heavy function', heavyFunction,
10                             on_finished=workdone)
```

- 공간 처리 알고리즘으로부터 태스크를 생성합니다:

```
1 params = dict()
2 context = QgsProcessingContext()
3 context.setProject(QgsProject.instance())
4 feedback = QgsProcessingFeedback()
5
6 buffer_alg = QgsApplication.instance().processingRegistry().algorithmById(
7     ↪ 'native:buffer')
8 task = QgsProcessingAlgRunnerTask(buffer_alg, params, context,
9                                   feedback)
```

⚠ 경고

어떤 배경 태스크도 (생성 방법에 상관없이) 주 스레드에 살아 있는, 예를 들면 `QgsVectorLayer`, `QgsProject` 에 접근하거나 또는 새 위젯을 생성하거나 기존 위젯과 쌍방향 작업을 하는 것과 같은 GUI 기반 작업을 수행하는 `QObject` 를 절대로 사용해서는 안 됩니다. Qt 위젯은 주 스레드에서만 접근 또는 수정해야만 합니다. 태스크에 쓰이는 데이터는 태스크를 시작하기 전에 반드시 복사해야만 합니다. 배경 스레드에서 이들을 사용하려 시도하는 경우 충돌이 발생할 것입니다.

또한 `context` 와 `feedback` 클래스들이 적어도 이들을 사용하는 태스크만큼은 살아 있도록 항상 확인하십시오. 태스크 완료 시 `QgsTaskManager` 가 태스크가 예약되었던 `context` 와 `feedback` 에 접근하지 못하는 경우 QGIS 가 중단될 것입니다.

i 참고

`QgsProcessingContext` 클래스를 호출한 직후 `setProject()` 메소드를 호출하는 것이 흔한 패턴입니다. 이렇게 하면 태스크는 물론 태스크의 콜백 함수가 프로젝트 전체 설정 가운데 대부분을 사용할 수 있습니다. 콜백 함수에서 공간 레이어를 작업하는 경우 특히 유용합니다.

`QgsTask` 클래스의 `addSubTask()` 함수를 사용하면 태스크들 사이의 의존성을 설명할 수 있습니다. 의존성을 선언하는 경우, 태스크 관리자가 이런 의존성들을 실행하는 방법을 자동으로 결정할 것입니다. 의존성들을 가능한 한 빨리 만족시키기 위해 의존성들을 병렬로 실행할 것입니다. 또다른 태스크가 의존하고 있는 태스크를 취소하는 경우, 의존하는 태스크도 취소될 것입니다. 순환 의존성은 교착 상태 (deadlock) 를 발생시킬 수 있기 때문에 조심해야 합니다.

태스크가 사용할 수 있는 레이어에 의존하는 경우, `QgsTask` 클래스의 `setDependentLayers()` 함수를

사용해서 이를 선언할 수 있습니다. 태스크가 의존하는 레이어를 사용할 수 없는 경우 태스크가 취소될 것입니다. 태스크를 생성하고 나면 태스크 관리자의 `addTask()` 함수를 사용해서 실행을 예약할 수 있습니다. 관리자에 태스크를 추가하면 자동적으로 관리자가 해당 태스크를 소유하게 되며, 태스크를 실행한 후 관리자가 태스크를 정리하고 삭제할 것입니다. 태스크의 예약은 태스크 우선 순위에 영향을 받는데, `addTask()` 메소드에서 이 우선 순위를 설정합니다.

`QgsTask` 및 `QgsTaskManager` 클래스의 신호와 함수를 통해 태스크의 상태를 모니터링할 수 있습니다.

15.2 예제

15.2.1 `QgsTask` 클래스 확장하기

다음 예시에서는 `RandomIntegerSumTask` 가 `QgsTask` 클래스를 확장시키고 지정한 시간 동안 0 에서 500 사이의 정수 100 개를 랜덤하게 생성할 것입니다. 이때 랜덤한 숫자가 42 일 경우, 태스크를 중단하고 예외를 발생시킵니다. 두 가지 유형의 의존성을 보여주는 (하위 태스크를 가진) `RandomIntegerSumTask` 의 인스턴스를 몇 개 생성해서 태스크 관리자에 추가합니다.

```

1  import random
2  from time import sleep
3
4  from qgis.core import (
5      QgsApplication, QgsTask, QgsMessageLog, Qgis
6  )
7
8  MESSAGE_CATEGORY = 'RandomIntegerSumTask'
9
10 class RandomIntegerSumTask(QgsTask):
11     """This shows how to subclass QgsTask"""
12
13     def __init__(self, description, duration):
14         super().__init__(description, QgsTask.CanCancel)
15         self.duration = duration
16         self.total = 0
17         self.iterations = 0
18         self.exception = None
19
20     def run(self):
21         """Here you implement your heavy lifting.
22         Should periodically test for isCanceled() to gracefully
23         abort.
24         This method MUST return True or False.
25         Raising exceptions will crash QGIS, so we handle them
26         internally and raise them in self.finished
27         """
28         QgsMessageLog.logMessage('Started task {}'.format(
29             self.description()),
30             MESSAGE_CATEGORY, Qgis.Info)
31         wait_time = self.duration / 100
32         for i in range(100):
33             sleep(wait_time)
34             # use setProgress to report progress
35             self.setProgress(i)
36             arandominteger = random.randint(0, 500)
37             self.total += arandominteger

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

38         self.iterations += 1
39         # check isCanceled() to handle cancellation
40         if self.isCanceled():
41             return False
42         # simulate exceptions to show how to abort task
43         if arandominteger == 42:
44             # DO NOT raise Exception('bad value!')
45             # this would crash QGIS
46             self.exception = Exception('bad value!')
47             return False
48         return True
49
50     def finished(self, result):
51         """
52         This function is automatically called when the task has
53         completed (successfully or not).
54         You implement finished() to do whatever follow-up stuff
55         should happen after the task is complete.
56         finished is always called from the main thread, so it's safe
57         to do GUI operations and raise Python exceptions here.
58         result is the return value from self.run.
59         """
60         if result:
61             QgsMessageLog.logMessage(
62                 'RandomTask "{name}" completed\n' \
63                 'RandomTotal: {total} (with {iterations} '\
64                 'iterations)'.format(
65                     name=self.description(),
66                     total=self.total,
67                     iterations=self.iterations),
68                 MESSAGE_CATEGORY, Qgis.Success)
69         else:
70             if self.exception is None:
71                 QgsMessageLog.logMessage(
72                     'RandomTask "{name}" not successful but without '\
73                     'exception (probably the task was manually '\
74                     'canceled by the user)'.format(
75                         name=self.description()),
76                     MESSAGE_CATEGORY, Qgis.Warning)
77             else:
78                 QgsMessageLog.logMessage(
79                     'RandomTask "{name}" Exception: {exception}'.format(
80                         name=self.description(),
81                         exception=self.exception),
82                     MESSAGE_CATEGORY, Qgis.Critical)
83                 raise self.exception
84
85     def cancel(self):
86         QgsMessageLog.logMessage(
87             'RandomTask "{name}" was canceled'.format(
88                 name=self.description()),
89             MESSAGE_CATEGORY, Qgis.Info)
90         super().cancel()
91
92 longtask = RandomIntegerSumTask('waste cpu long', 20)

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

94 shorttask = RandomIntegerSumTask('waste cpu short', 10)
95 minitask = RandomIntegerSumTask('waste cpu mini', 5)
96 shortsubtask = RandomIntegerSumTask('waste cpu subtask short', 5)
97 longsubtask = RandomIntegerSumTask('waste cpu subtask long', 10)
98 shortestsubtask = RandomIntegerSumTask('waste cpu subtask shortest', 4)
99
100 # Add a subtask (shortsubtask) to shorttask that must run after
101 # minitask and longtask has finished
102 shorttask.addSubTask(shortsubtask, [minitask, longtask])
103 # Add a subtask (longsubtask) to longtask that must be run
104 # before the parent task
105 longtask.addSubTask(longsubtask, [], QgsTask.ParentDependsOnSubTask)
106 # Add a subtask (shortestsubtask) to longtask
107 longtask.addSubTask(shortestsubtask)
108
109 QgsApplication.taskManager().addTask(longtask)
110 QgsApplication.taskManager().addTask(shorttask)
111 QgsApplication.taskManager().addTask(minitask)

```

```

1 RandomIntegerSumTask(0): Started task "waste cpu subtask shortest"
2 RandomIntegerSumTask(0): Started task "waste cpu short"
3 RandomIntegerSumTask(0): Started task "waste cpu mini"
4 RandomIntegerSumTask(0): Started task "waste cpu subtask long"
5 RandomIntegerSumTask(3): Task "waste cpu subtask shortest" completed
6 RandomTotal: 25452 (with 100 iterations)
7 RandomIntegerSumTask(3): Task "waste cpu mini" completed
8 RandomTotal: 23810 (with 100 iterations)
9 RandomIntegerSumTask(3): Task "waste cpu subtask long" completed
10 RandomTotal: 26308 (with 100 iterations)
11 RandomIntegerSumTask(0): Started task "waste cpu long"
12 RandomIntegerSumTask(3): Task "waste cpu long" completed
13 RandomTotal: 22534 (with 100 iterations)

```

15.2.2 함수로부터 나온 태스크

함수로부터 (이 예시에서는 `doSomething` 으로부터) 태스크를 생성합니다. 이 함수의 첫 번째 파라미터가 함수를 위한 `QgsTask` 클래스를 담을 것입니다. `on_finished` 는 태스크 완료 시 호출할 함수를 지정하는 중요한 (이름이 있는) 파라미터입니다. 이 예시의 `doSomething` 함수는 추가적인 이름이 있는 파라미터 `wait_time` 을 가지고 있습니다.

```

1 import random
2 from time import sleep
3
4 MESSAGE_CATEGORY = 'TaskFromFunction'
5
6 def doSomething(task, wait_time):
7     """
8     Raises an exception to abort the task.
9     Returns a result if success.
10    The result will be passed, together with the exception (None in
11    the case of success), to the on_finished method.
12    If there is an exception, there will be no result.
13    """
14    QgsMessageLog.logMessage('Started task {}'.format(task.description()),

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

15             MESSAGE_CATEGORY, Qgis.Info)
16     wait_time = wait_time / 100
17     total = 0
18     iterations = 0
19     for i in range(100):
20         sleep(wait_time)
21         # use task.setProgress to report progress
22         task.setProgress(i)
23         arandominteger = random.randint(0, 500)
24         total += arandominteger
25         iterations += 1
26         # check task.isCanceled() to handle cancellation
27         if task.isCanceled():
28             stopped(task)
29             return None
30         # raise an exception to abort the task
31         if arandominteger == 42:
32             raise Exception('bad value!')
33     return {'total': total, 'iterations': iterations,
34           'task': task.description()}
35
36 def stopped(task):
37     QgsMessageLog.logMessage(
38         'Task "{name}" was canceled'.format(
39             name=task.description()),
40         MESSAGE_CATEGORY, Qgis.Info)
41
42 def completed(exception, result=None):
43     """This is called when doSomething is finished.
44     Exception is not None if doSomething raises an exception.
45     result is the return value of doSomething."""
46     if exception is None:
47         if result is None:
48             QgsMessageLog.logMessage(
49                 'Completed with no exception and no result '\
50                 '(probably manually canceled by the user)',
51                 MESSAGE_CATEGORY, Qgis.Warning)
52         else:
53             QgsMessageLog.logMessage(
54                 'Task {name} completed\n'
55                 'Total: {total} ( with {iterations} '\
56                 'iterations)'.format(
57                     name=result['task'],
58                     total=result['total'],
59                     iterations=result['iterations']),
60                 MESSAGE_CATEGORY, Qgis.Info)
61     else:
62         QgsMessageLog.logMessage("Exception: {}".format(exception),
63                                 MESSAGE_CATEGORY, Qgis.Critical)
64     raise exception
65
66 # Create a few tasks
67 task1 = QgsTask.fromFunction('Waste cpu 1', doSomething,
68                             on_finished=completed, wait_time=4)
69 task2 = QgsTask.fromFunction('Waste cpu 2', doSomething,
70                             on_finished=completed, wait_time=3)

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

71 QgsApplication.taskManager().addTask(task1)
72 QgsApplication.taskManager().addTask(task2)

```

```

1 RandomIntegerSumTask(0): Started task "waste cpu subtask short"
2 RandomTaskFromFunction(0): Started task Waste cpu 1
3 RandomTaskFromFunction(0): Started task Waste cpu 2
4 RandomTaskFromFunction(0): Task Waste cpu 2 completed
5 RandomTotal: 23263 ( with 100 iterations)
6 RandomTaskFromFunction(0): Task Waste cpu 1 completed
7 RandomTotal: 25044 ( with 100 iterations)

```

15.2.3 공간 처리 알고리즘으로부터 나온 태스크

지정한 범위 안에 포인트 50,000 개를 `qgis:randompointsinextent` 알고리즘을 사용해서 랜덤하게 생성하는 태스크를 생성합니다. 결과물은 프로젝트에 안전한 방식으로 추가됩니다.

```

1 from functools import partial
2 from qgis.core import (QgsTaskManager, QgsMessageLog,
3                        QgsProcessingAlgRunnerTask, QgsApplication,
4                        QgsProcessingContext, QgsProcessingFeedback,
5                        QgsProject)
6
7 MESSAGE_CATEGORY = 'AlgRunnerTask'
8
9 def task_finished(context, successful, results):
10     if not successful:
11         QgsMessageLog.logMessage('Task finished unsuccessfully',
12                                 MESSAGE_CATEGORY, Qgs.Warning)
13     output_layer = context.getMapLayer(results['OUTPUT'])
14     # because getMapLayer doesn't transfer ownership, the layer will
15     # be deleted when context goes out of scope and you'll get a
16     # crash.
17     # takeMapLayer transfers ownership so it's then safe to add it
18     # to the project and give the project ownership.
19     if output_layer and output_layer.isValid():
20         QgsProject.instance().addMapLayer(
21             context.takeResultLayer(output_layer.id()))
22
23 alg = QgsApplication.processingRegistry().algorithmById(
24     'qgis:randompointsinextent')
25 # `context` and `feedback` need to
26 # live for as least as long as `task`,
27 # otherwise the program will crash.
28 # Initializing them globally is a sure way
29 # of avoiding this unfortunate situation.
30 context = QgsProcessingContext()
31 feedback = QgsProcessingFeedback()
32 params = {
33     'EXTENT': '0.0,10.0,40,50 [EPSG:4326]',
34     'MIN_DISTANCE': 0.0,
35     'POINTS_NUMBER': 50000,
36     'TARGET_CRS': 'EPSG:4326',
37     'OUTPUT': 'memory:My random points'
38 }

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
39 task = QgsProcessingAlgRunnerTask(alg, params, context, feedback)
40 task.executed.connect(partial(task_finished, context))
41 QgsApplication.taskManager().addTask(task)
```

<https://www.opengis.ch/2018/06/22/threads-in-pyqgis3/> 도 참조하세요.

파이썬 플러그인 개발하기

16.1 파이썬 플러그인 구성하기

플러그인을 생성하기 위한 주요 단계는 다음과 같습니다:

1. 아이디어: 새로운 QGIS 플러그인으로 무엇을 하고 싶은지에 대한 아이디어를 떠올립니다.
2. 준비 (*setup*): 여러분의 플러그인을 위한 파일들을 생성합니다. 플러그인 유형에 따라, 몇몇 파일은 필수인 반면 몇몇 파일은 선택적입니다.
3. 개발: 알맞은 파일에 코드를 작성합니다.
4. 문서화: 플러그인 문서를 작성합니다.
5. 선택적으로 번역: 여러분의 플러그인을 다른 언어로 번역합니다.
6. 테스트: 모든 것이 괜찮은지 확인하기 위해 여러분의 플러그인을 다시 불러옵니다.
7. 공개 (*publish*): QGIS 저장소에 여러분의 플러그인을 공개하거나, 또는 개인적인 “GIS 무기”의 “병기고”로써 여러분 자신의 저장소를 만드십시오.

16.1.1 시작하기

새 플러그인을 작성하기 전에, 공식 파이썬 플러그인 저장소를 살펴보십시오. 기존 플러그인들의 소스 코드를 연구하면 프로그래밍에 대해 더 많은 것을 배울 수 있습니다. 비슷한 플러그인이 이미 존재하는지 찾아볼 수도 있고, 존재한다면 기존 플러그인을 확장하거나 또는 적어도 기존 플러그인을 바탕으로 여러분의 자신의 플러그인을 작성할 수도 있습니다.

플러그인 파일 구조 준비하기

새 플러그인을 시작하려면, 필요한 플러그인 파일들을 준비해야 합니다.

플러그인 준비를 도와줄 수 있는 플러그인 템플릿 리소스가 2 개 있습니다:

- 교육 목적인 경우 또는 미니멀리스트 접근법이 바람직할 때마다, **최소 플러그인** 템플릿 이 무결한 QGIS 파이썬 플러그인을 생성하는 데 필요한 기본 (최소한의) 파일들을 제공합니다.
- 좀 더 완전한 기능 플러그인 템플릿이 필요한 경우, **플러그인 작성자** 가 현지화 (번역) 및 테스트 작업 같은 기능들을 포함하는 서로 다른 여러 플러그인 유형들을 위한 템플릿들을 생성할 수 있습니다.

전형적인 플러그인 디렉터리는 다음 파일들을 포함합니다:

- `metadata.txt` - 필수 - 플러그인 웹사이트와 플러그인 인프라스트럭처에서 사용하는 일반 정보, 버전, 이름, 그리고 몇몇 기타 메타데이터를 담고 있습니다.
- `__init__.py` - 필수 - 플러그인의 시작점입니다. 이 파일은 `classFactory()` 메소드를 가지고 있어야 하며, 다른 초기 설정 코드를 가지고 있을 수도 있습니다.
- `mainPlugin.py` - 핵심 코드 - 플러그인의 주요 작업 코드입니다. 플러그인과 주요 코드의 액션에 대한 모든 정보를 담고 있습니다.
- `form.ui` - 사용자 정의 *GUI* 를 가진 플러그인 용 - Qt 설계자가 생성한 GUI 입니다.
- `form.py` - 컴파일된 *GUI* - 앞에서 설명한 `form.ui` 를 파이썬으로 번역한 파일입니다.
- `resources.qrc` - 선택적 - Qt 설계자가 생성한 .xml 문서입니다. GUI 양식에 쓰이는 리소스들을 가리키는 상대 경로를 담고 있습니다.
- `resources.py` - 컴파일된 리소스, 선택적 - 앞에서 설명한 `resources.qrc` 를 파이썬으로 번역한 파일입니다.
- `LICENSE` - *required* if plugin is to be published or updated in the QGIS Plugins Directory, otherwise *optional*. File should be a plain text file with no file extension in the filename.

⚠ 경고

공식 파이썬 플러그인 저장소 에 플러그인을 업로드할 계획이 있다면 여러분의 플러그인이 플러그인무결성 검증 에 필수적인 몇몇 추가 규칙들을 따르는지 반드시 확인해야만 합니다.

16.1.2 플러그인 코드 작성하기

다음은 앞에서 소개한 파일들 각각에 어떤 내용을 추가해야 하는지를 설명합니다.

metadata.txt

먼저, 플러그인 관리자는 플러그인의 이름, 설명 등등과 같은 플러그인에 대한 몇몇 기본 정보를 검색해야 합니다. `metadata.txt` 파일에 이 정보들을 저장합니다.

i 참고

모든 메타데이터는 반드시 UTF-8 으로 인코딩되어 있어야만 합니다.

메타데이터 이름	필수 여부	설명
name	O	플러그인의 이름을 담고 있는 짧은 문자열
qgisMinimumVersion	O	최저 QGIS 버전의 닷 데시멀 노테이션
qgisMaximumVersion	X	최고 QGIS 버전의 닷 데시멀 노테이션
description	O	플러그인을 설명하는 짧은 텍스트, HTML 은 허용되지 않습니다
about	O	플러그인을 자세히 설명하는 좀 더 긴 텍스트, HTML 은 허용되지 않습니다
version	O	버전의 닷 데시멀 노테이션을 담고 있는 짧은 문자열
author	O	개발자 이름
email	O	개발자의 이메일 주소, 웹사이트에서는 로그인한 사용자에게만 보이지만 플러그인을 설치했다면 플러그인 관리자에서 볼 수 있습니다
changelog	X	문자열, 여러 줄일 수 있지만 HTML 은 허용되지 않습니다
experimental	X	불 (boolean) 플래그, True 또는 False - 이 버전이 실험적 버전인 경우 True
deprecated	X	불 (boolean) 플래그, True 또는 False - 업로드된 버전만이 아니라 전체 플러그인에 적용됩니다
tags	X	선표로 구분된 목록, 개별 태그 안에 공백이 허용됩니다
homepage	X	사용자 플러그인의 홈페이지를 가리키는 무결한 URL
repository	O	소스 코드 저장소를 가리키는 무결한 URL
tracker	X	티켓 및 버그 보고서를 가리키는 무결한 URL
icon	X	웹 친화적인 이미지 (PNG, JPEG) 의 파일 이름 또는 (플러그인 압축 패키지의 기반 폴더를 기준으로 하는) 상대 경로
category	X	Raster, Vector, Database, Mesh 및 Web 가운데 하나
plugin_dependencies	X	PIP 처럼 설치할 다른 플러그인들을 선표로 구분한 목록, 플러그인들의 메타데이터의 이름 필드에서 나오는 플러그인 이름들을 사용합니다
server	X	불 (boolean) 플래그, True 또는 False - 플러그인이 서버 인터페이스를 가지고 있는지 여부를 결정합니다
hasProcessingProvider	X	불 (boolean) 플래그, True 또는 False - 플러그인이 공간 처리 알고리즘을 제공하는지 여부를 결정합니다

플러그인은 기본적으로 *Plugins* 메뉴에 배치되지만 (다음 부분에서 여러분의 플러그인에 대한 메뉴 항목을 추가하는 방법을 배울 것입니다) *Raster*, *Vector*, *Database*, *Mesh*, 그리고 *Web* 메뉴에도 배치할 수 있습니다.

이를 지정하기 위한 해당 “category” 메타데이터 항목이 존재하기 때문에, 이에 맞춰 플러그인을 분류할 수 있습니다. 이 메타데이터 항목을 사용자를 위한 도움말처럼 사용해서 어디에서 (어떤 메뉴에서) 플러그인을 찾을 수 있는지 알려줄 수 있습니다. “category”에 사용할 수 있는 값은 *Vector*, *Raster*, *Database*, *Mesh* 또는 *Web* 가운데 하나입니다. 예를 들어 *Raster* 메뉴에서 여러분의 플러그인을 사용할 수 있게 하려면, `metadata.txt` 에 이를 추가하십시오.

```
category=Raster
```

참고

“qgisMaximumVersion”이 비어 있는 경우, 공식 파이썬 플러그인 저장소에 업로드할 때 자동적으로 주요 버전에 .99 를 더한 값으로 설정될 것입니다.

이 `metadata.txt` 의 예시:

```
; the next section is mandatory
[general]
```

(다음 페이지에 계속)

```

name=HelloWorld
email=me@example.com
author=Just Me
qgisMinimumVersion=3.0
description=This is an example plugin for greeting the world.
    Multiline is allowed:
    lines starting with spaces belong to the same
    field, in this case to the "description" field.
    HTML formatting is not allowed.
about=This paragraph can contain a detailed description
    of the plugin. Multiline is allowed, HTML is not.
version=version 1.2
tracker=http://bugs.itopen.it
repository=http://www.itopen.it/repo
; end of mandatory metadata

; start of optional metadata
category=Raster
changelog=The changelog lists the plugin versions
    and their changes as in the example below:
    1.0 - First stable release
    0.9 - All features implemented
    0.8 - First testing release

; Tags are in comma separated value format, spaces are allowed within the
; tag name.
; Tags should be in English language. Please also check for existing tags and
; synonyms before creating a new one.
tags=wkt,raster,hello world

; these metadata can be empty, they will eventually become mandatory.
homepage=https://www.itopen.it
icon=icon.png

; experimental flag (applies to the single version)
experimental=True

; deprecated flag (applies to the whole plugin and not only to the uploaded version)
deprecated=False

; if empty, it will be automatically set to major version + .99
qgisMaximumVersion=3.99

; Since QGIS 3.8, a comma separated list of plugins to be installed
; (or upgraded) can be specified.
; The example below will try to install (or upgrade) "MyOtherPlugin" version 1.12
; and any version of "YetAnotherPlugin".
; Both "MyOtherPlugin" and "YetAnotherPlugin" names come from their own metadata's
; name field
plugin_dependencies=MyOtherPlugin==1.12,YetAnotherPlugin

```


__init__.py

파이썬의 가져오기 시스템이 이 파일을 필요로 합니다. 또 QGIS 도 이 파일이 `classFactory()` 함수를 담고 있을 것을 요구합니다. 이 함수는 QGIS 에 플러그인을 불러왔을 때 호출되어, `QgisInterface` 클래스의 인스턴스를 받아서 `mainplugin.py` 파일로부터 플러그인의 클래스 객체를 반환해야만 합니다—이 경우 `TestPlugin` 이라는 플러그인입니다. (다음은 참조하세요.) `__init__.py` 의 내용은 다음과 같이 보여야 합니다:

```
def classFactory(iface):
    from .mainPlugin import TestPlugin
    return TestPlugin(iface)

# any other initialisation needed
```

mainPlugin.py

이 파일에서 마법이 벌어지는데, 이 마법은 다음과 같이 보일 것입니다: (`mainPlugin.py` 파일의 예시)

```
from qgis.PyQt.QtGui import *
from qgis.PyQt.QtWidgets import *

# initialize Qt resources from file resources.py
from . import resources

class TestPlugin:

    def __init__(self, iface):
        # save reference to the QGIS interface
        self.iface = iface

    def initGui(self):
        # create action that will start plugin configuration
        self.action = QAction(QIcon("testplug:icon.png"),
                               "Test plugin",
                               self.iface.mainWindow())
        self.action.setObjectName("testAction")
        self.action.setWhatsThis("Configuration for test plugin")
        self.action.setStatusTip("This is status tip")
        self.action.triggered.connect(self.run)

        # add toolbar button and menu item
        self.iface.addToolBarIcon(self.action)
        self.iface.addPluginToMenu("&Test plugins", self.action)

        # connect to signal renderComplete which is emitted when canvas
        # rendering is done
        self.iface.mapCanvas().renderComplete.connect(self.renderTest)

    def unload(self):
        # remove the plugin menu item and icon
        self.iface.removePluginMenu("&Test plugins", self.action)
        self.iface.removeToolBarIcon(self.action)

        # disconnect form signal of the canvas
        self.iface.mapCanvas().renderComplete.disconnect(self.renderTest)

    def run(self):
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
# create and show a configuration dialog or something similar
print("TestPlugin: run called!")

def renderTest(self, painter):
    # use painter for drawing to map canvas
    print("TestPlugin: renderTest called!")
```

주 플러그인 소스 파일 (예: mainPlugin.py) 에 반드시 존재해야만 하는 플러그인 함수는 다음과 같습니다:

- `__init__`: QGIS 인터페이스에 접근할 수 있게 해줍니다.
- `initGui()`: 플러그인을 불러왔을 때 호출됩니다.
- `unload()`: 플러그인 불러오기를 해제했을 때 호출됩니다.

앞의 예시에서 `addPluginToMenu()` 메소드를 사용했습니다. 이 메소드는 *Plugins* 메뉴에 해당 메뉴 액션을 추가할 것입니다. 다른 메뉴에 액션을 추가하는 대체 메소드들이 존재합니다. 다음은 대체 메소드 목록입니다:

- `addPluginToRasterMenu()`
- `addPluginToVectorMenu()`
- `addPluginToDatabaseMenu()`
- `addPluginToWebMenu()`

이 대체 메소드들은 모두 `addPluginToMenu()` 메소드와 동일한 문법 (syntax) 을 사용합니다.

플러그인 항목들을 정리하는 방식의 일관성을 유지하기 위해 이 사전 정의 메소드들 가운데 하나에 여러분의 플러그인 메뉴를 추가할 것을 권장합니다. 하지만, 다음 예시에서처럼 여러분이 메뉴 바에 사용자 정의 메뉴 그룹을 직접 추가할 수도 있습니다:

```
def initGui(self):
    self.menu = QMenu(self.iface.mainWindow())
    self.menu.setObjectName("testMenu")
    self.menu.setTitle("MyMenu")

    self.action = QAction(QIcon("testplug:icon.png"),
                          "Test plugin",
                          self.iface.mainWindow())
    self.action.setObjectName("testAction")
    self.action.setWhatsThis("Configuration for test plugin")
    self.action.setStatusTip("This is status tip")
    self.action.triggered.connect(self.run)
    self.menu.addAction(self.action)

    menuBar = self.iface.mainWindow().menuBar()
    menuBar.insertMenu(self.iface.firstRightStandardMenu().menuAction(),
                      self.menu)

def unload(self):
    self.menu.deleteLater()
```

여러분의 플러그인에 `QAction` 및 `QMenu` 의 `objectName` 을 특정 이름으로 설정해서 사용자 정의할 수 있도록 만드는 일을 잊지 마십시오.

사용자 정의 메뉴에 도움말 (help) 및 정보 (about) 액션도 추가할 수 있지만, 이 액션들을 사용할 수 있게 해주는 편리한 장소는 QGIS 의 *Help* > *Plugins* 메뉴입니다. `pluginHelpMenu()` 메소드를 사용하면 됩니다.

```
def initGui(self):

    self.help_action = QAction(
        QIcon("testplug:icon.png"),
        self.tr("Test Plugin..."),
        self.iface.mainWindow()
    )
    # Add the action to the Help menu
    self.iface.pluginHelpMenu().addAction(self.help_action)

    self.help_action.triggered.connect(self.show_help)

    @staticmethod
    def show_help():
        """ Open the online help. """
        QDesktopServices.openUrl(QUrl('https://docs.qgis.org'))

    def unload(self):

        self.iface.pluginHelpMenu().removeAction(self.help_action)
        del self.help_action
```

실제 플러그인을 작업하는 경우 또다른 (작업) 디렉터리에 플러그인을 작성하고, UI 와 리소스 파일을 생성할 makefile 을 생성해서 QGIS 설치본에 플러그인을 설치하는 편이 좋습니다.

16.1.3 플러그인 문서화하기

플러그인에 대한 문서는 HTML 도움말 파일로 작성할 수 있습니다. qgis.utils 모듈이 다른 QGIS 도움말과 동일한 방식으로 도움말 파일 탐색기를 여는 showPluginHelp() 함수를 제공합니다.

showPluginHelp() 함수는 호출한 모듈과 동일한 디렉터리에서 도움말 파일을 검색합니다. 이 함수는 index-ll_cc.html, index-ll.html, index-en.html, index-en_us.html, 그리고 index.html 순서대로 파일을 검색해서 첫 번째로 찾은 파일을 표시합니다. 이때 ll_cc 는 QGIS 로케일로, 플러그인에 문서를 여러 언어로 번역한 파일들을 포함시킬 수 있게 해줍니다.

또 showPluginHelp() 함수는 도움말이 표시될 특정 플러그인을 식별하는 packageName 파라미터, 검색 중인 파일들의 이름에서 "index"를 대체할 수 있는 filename 파라미터, 그리고 문서 안에서 탐색기가 배치될 위치를 결정하는 HTML 앵커 태그의 이름인 section 파라미터도 받을 수 있습니다.

16.1.4 플러그인 번역하기

몇 단계만 거치면 여러분의 컴퓨터의 로케일 설정에 따라 플러그인을 서로 다른 언어들로 불러오게 만들 플러그인 현지화를 위한 환경을 준비할 수 있습니다.

소프트웨어 요구 사항

The easiest way to create and manage all the translation files is to install [Qt Linguist](#). In a Debian-based GNU/Linux environment you can install it typing:

```
sudo apt install qttools5-dev-tools
```

파일과 디렉터리

플러그인을 생성했을 때 주 플러그인 디렉터리 안에 `i18n` 폴더가 있을 것입니다.

모든 번역 파일은 이 디렉터리 안에 있어야 합니다.

.pro 파일

먼저 [Qt Linguist](#) 가 관리할 수 있는 프로젝트 파일인 `.pro` 파일을 생성해야 합니다.

이 `.pro` 파일에 번역하고자 하는 모든 파일과 양식을 지정해야 합니다. 이 파일은 현지화 파일들과 변수들을 준비하는 데 쓰입니다. 다음은 [예제 플러그인](#)의 구성과 일치하는 프로젝트 파일의 예시입니다:

```
FORMS = ../form.ui
SOURCES = ../your_plugin.py
TRANSLATIONS = your_plugin_it.ts
```

여러분의 플러그인이 좀 더 복잡한 구성을 따를 수도 있고, 파일 여러 개로 배포될 수도 있습니다. 이런 경우, `.pro` 파일을 읽고 번역할 수 있는 문자열을 업데이트하는 데 사용하는 `pylupdate5` 프로그램이 와일드카드 문자들을 지원하지 않기 때문에 `.pro` 파일에 모든 파일을 명확하게 배치해야 한다는 사실을 기억하십시오. 이때 프로젝트 파일은 다음과 같이 보일 수도 있습니다:

```
FORMS = ../ui/about.ui ../ui/feedback.ui \
        ../ui/main_dialog.ui
SOURCES = ../your_plugin.py ../computation.py \
        ../utils.py
```

이에 더해, `your_plugin.py` 파일이 QGIS 툴바에서 플러그인의 모든 메뉴와 하위 메뉴를 호출하는 파일이기 때문에, 이들도 전부 번역해야 합니다.

마지막으로 `TRANSLATIONS` 변수를 사용해서 원하는 번역 언어를 지정할 수 있습니다.

⚠ 경고

`.ts` 파일의 이름을 `your_plugin_ + language + .ts` 형식으로 붙여야 한다는 사실을 기억하십시오. 그러지 않으면 언어를 불러오지 못할 것입니다! 언어의 2 문자 단축 이름을 (이탈리아어의 경우 `it`, 독일어의 경우 `de`, ...) 사용하십시오.

.ts 파일

.pro 파일을 생성하고 나면 플러그인의 언어(들)를 위한 .ts 파일(들)을 생성할 준비가 된 것입니다.

터미널을 열고, `your_plugin/i18n` 디렉터리로 가서 다음을 입력하십시오:

```
pylupdate5 your_plugin.pro
```

`your_plugin_language.ts` 파일(들)을 볼 수 있을 것입니다.

Qt Linguist로 .ts 파일을 열고 번역을 시작하십시오.

.qm 파일

플러그인 번역을 끝내고 나면 (번역이 완료되지 않은 일부 문자열의 경우 해당 문자열의 소스 언어를 사용할 것입니다) .qm 파일을 생성해야 합니다. (QGIS가 사용하게 될, 컴파일된 .ts 파일입니다.)

터미널을 열고, `your_plugin/i18n` 디렉터리로 가서 다음을 입력하십시오:

```
lrelease your_plugin.ts
```

이제 `i18n` 디렉터리 안에서 `your_plugin.qm` 파일(들)을 볼 수 있을 것입니다.

Makefile 을 사용해서 번역하기

다른 방법으로는 여러분이 플러그인 작성자를 사용해서 플러그인을 생성한 경우, 파이썬 코드와 Qt 대화창으로부터 메시지를 `makefile`을 사용해서 추출할 수 있습니다. `Makefile`의 첫머리에 `LOCALES` 변수가 있습니다:

```
LOCALES = en
```

이 변수에 언어의 단축 이름을 추가하십시오. 예를 들어 헝가리어를 추가한다면:

```
LOCALES = en hu
```

이제 다음 명령어를 실행하면 소스로부터 (`en.ts` 파일은 물론) `hu.ts` 파일을 생성하거나 업데이트할 수 있습니다:

```
make transup
```

이 명령어를 실행하고 나면 `LOCALES` 변수에 설정된 모든 언어에 대한 .ts 파일이 업데이트됩니다. **Qt Linguist**를 사용해서 프로그램 메시지를 번역하십시오. 번역을 완료하면 `transcompile`로 .qm 파일을 생성할 수 있습니다:

```
make transcompile
```

.ts 파일을 여러분의 플러그인과 함께 배포해야 합니다.

플러그인 불러오기

플러그인의 번역을 보려면, QGIS 를 열어서 언어를 변경한 다음 (*Settings ▸ Options ▸ General*) QGIS 를 다시 시작하십시오.

여러분의 플러그인을 정확한 언어로 보게 될 것입니다.

⚠ 경고

플러그인에서 무언가를 (새로운 UI, 새로운 메뉴 등등) 변경하는 경우 .ts 와 .qm 파일 둘 다 업데이트 버전을 다시 생성 해야 하기 때문에, 앞의 명령어를 다시 실행하십시오.

16.1.5 사용자 플러그인 공유하기

QGIS 는 플러그인 저장소에 수백 개의 플러그인을 호스팅하고 있습니다. 여러분의 플러그인을 공유해보십시오! 그러면 QGIS 의 가능성이 확장되는 것은 물론 사람들이 여러분의 코드로부터 배울 수 있을 것입니다. QGIS 에서 호스팅하고 있는 모든 플러그인을 플러그인 관리자를 통해 검색하고 설치할 수 있습니다.

plugins.qgis.org 에서 관련 정보와 요구 사항을 찾아볼 수 있습니다.

16.1.6 도움말 및 꼼수

플러그인 다시 불러오기

여러분이 플러그인을 개발하는 동안 테스트를 위해 QGIS 에 플러그인을 자주 다시 불러와야 할 것입니다. 이때 플러그인 다시 불러오기 (**Plugin Reloader**) 플러그인을 사용하면 플러그인을 매우 쉽게 다시 불러올 수 있습니다. 이 플러그인은 플러그인 관리자 를 통해 찾을 수 있습니다.

qgis-plugin-ci 를 사용한 자동 패키지 작업, 배포 및 번역

[qgis-plugin-ci](#) provides a command line interface to perform automated packaging and deployment for QGIS plugins on your computer, or using continuous integration like [GitHub workflows](#) or [Gitlab-CI](#) as well as [Transifex](#) for translation.

이 파이썬 프로그램은 CLI(명령 줄 인터페이스) 를 통해 또는 CI(지속적 통합) 에서의 액션으로 XML 플러그인 저장소 파일을 배포하고, 번역하고, 공개하고, 또는 생성할 수 있게 해줍니다.

플러그인에 접근하기

QGIS 안에서, 설치된 플러그인의 모든 클래스에 접근할 수 있습니다. 디버그 작업을 수행하는 경우에 편리합니다.

```
my_plugin = qgis.utils.plugins['My Plugin']
```

로그 메시지

log_message_panel 안에 플러그인 전용 탭이 있습니다.

리소스 파일

일부 플러그인은 리소스 파일을 사용합니다. 예를 들어 resources.qrc 파일은 아이콘 같은 GUI 용 리소스를 정의합니다:

```
<RCC>
<qresource prefix="/plugins/testplug" >
  <file>icon.png</file>
</qresource>
</RCC>
```

다른 플러그인들 또는 QGIS 의 어떤 부분과도 충돌하지 않을 접두어를 사용하는 편이 좋습니다. 그러지 않으면 원하지 않는 리소스를 얻게 될 수도 있습니다. 이제 리소스를 담게 될 파이썬 파일을 **pyrcc5** 명령어를 사용해서 생성해야 합니다:

```
pyrcc5 -o resources.py resources.qrc
```

참고

In Windows environments, attempting to run the **pyrcc5** from Command Prompt or Powershell will probably result in the error “Windows cannot access the specified device, path, or file [...]”. The easiest solution is probably to use the OSGeo4W Shell but if you are comfortable modifying the PATH environment variable or specifying the path to the executable explicitly you should be able to find it at <Your QGIS Install Directory>\bin\pyrcc5.exe.

16.2 코드 조각

힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (
2     QgsProject,
3     QgsApplication,
4     QgsMapLayer,
5 )
6
7 from qgis.gui import (
8     QgsGui,
9     QgsOptionsWidgetFactory,
10    QgsOptionsPageWidget,
11    QgsLayerTreeEmbeddedWidgetProvider,
12    QgsLayerTreeEmbeddedWidgetRegistry,
13 )
14
15 from qgis.PyQt.QtCore import Qt
```

```

16 from qgis.PyQt.QtWidgets import (
17     QMessageBox,
18     QAction,
19     QHBoxLayout,
20     QComboBox,
21 )
22 from qgis.PyQt.QtGui import QIcon

```

이 장에서는 플러그인을 쉽게 개발할 수 있도록 도와주는 코드 조각 (snippet) 에 대해 설명합니다.

16.2.1 단축키로 메소드를 호출하는 방법

플러그인의 `initGui()` 함수에 다음 코드를 추가하십시오:

```

self.key_action = QAction("Test Plugin", self.iface.mainWindow())
self.iface.registerMainWindowAction(self.key_action, "Ctrl+I") # action triggered by
↳ Ctrl+I
self.iface.addPluginToMenu("&Test plugins", self.key_action)
self.key_action.triggered.connect(self.key_action_triggered)

```

`unload()` 함수에 다음 코드를 추가하십시오:

```

self.iface.unregisterMainWindowAction(self.key_action)

```

CTRL+I 단축키를 누르면 해당 메소드를 호출합니다:

```

def key_action_triggered(self):
    QMessageBox.information(self.iface.mainWindow(), "Ok", "You pressed Ctrl+I")

```

사용자가 해당 액션에 단축키를 사용자 정의할 수 있게 할 수도 있습니다. 다음 코드를 추가하면 됩니다:

```

1 # in the initGui() function
2 QgsGui.shortcutsManager().registerAction(self.key_action)
3
4 # and in the unload() function
5 QgsGui.shortcutsManager().unregisterAction(self.key_action)

```

16.2.2 QGIS 아이콘을 재사용하는 방법

QGIS 아이콘이 잘 알려져 있고 사용자에게 명확한 메시지를 전달하기 때문에, 여러분의 플러그인에 새 아이콘을 그려서 설정하는 대신 QGIS 아이콘을 재사용하고 싶을 경우가 있을 수도 있습니다. 이런 경우 `getThemeIcon()` 메소드를 사용하십시오.

예를 들어 QGIS 코드 저장소에서 사용할 수 있는  `mActionFileOpen.svg` 아이콘을 재사용하려면:

```

1 # e.g. somewhere in the initGui
2 self.file_open_action = QAction(
3     QgsApplication.getThemeIcon("/mActionFileOpen.svg"),
4     self.tr("Select a File..."),
5     self.iface.mainWindow()
6 )
7 self.iface.addPluginToMenu("MyPlugin", self.file_open_action)

```


QGIS 아이콘을 호출할 수 있는 또다른 메소드로는 `iconPath()` 가 있습니다. QGIS 내장 이미지 - 치트시트 에서 테마 아이콘을 호출하는 예시를 찾아보십시오.

16.2.3 옵션 대화창의 플러그인 용 인터페이스

Settings ▸ Options 에 사용자 정의 플러그인 옵션 탭을 추가할 수 있습니다. 사용자 플러그인의 옵션을 위해 특정 메인 메뉴 항목을 추가하는 것보다는 이 편이 좋은데, 모든 QGIS 응용 프로그램 설정과 플러그인 설정을 단일 위치에 모아두기 때문에 사용자가 설정을 찾고 탐색하기에 더 편하기 때문입니다.

다음 코드 조각은 플러그인의 설정에 새 비어 있는 탭만 추가할 것입니다. 여러분이 이 탭에 사용자 플러그인에 특화된 모든 옵션 및 설정을 채워 넣을 수 있습니다. 다음 클래스들을 서로 다른 파일들로 분할할 수 있습니다. 이 예제에서 우리는 주 `mainPlugin.py` 파일 안에 클래스 2 개를 추가할 것입니다.

```

1 class MyPluginOptionsFactory(QgsOptionsWidgetFactory):
2
3     def __init__(self):
4         super().__init__()
5
6     def icon(self):
7         return QIcon('icons/my_plugin_icon.svg')
8
9     def createWidget(self, parent):
10        return ConfigOptionsPage(parent)
11
12
13 class ConfigOptionsPage(QgsOptionsPageWidget):
14
15     def __init__(self, parent):
16         super().__init__(parent)
17         layout = QHBoxLayout()
18         layout.setContentsMargins(0, 0, 0, 0)
19         self.setLayout(layout)

```

마지막으로 가져온 것들을 추가하고 `__init__` 함수를 수정합니다:

```

1 from qgis.PyQt.QtWidgets import QHBoxLayout
2 from qgis.gui import QgsOptionsWidgetFactory, QgsOptionsPageWidget
3
4
5 class MyPlugin:
6     """QGIS Plugin Implementation."""
7
8     def __init__(self, iface):
9         """Constructor.
10
11         :param iface: An interface instance that will be passed to this class
12                       which provides the hook by which you can manipulate the QGIS
13                       application at run time.
14         :type iface: QgsInterface
15         """
16         # Save reference to the QGIS interface
17         self.iface = iface
18
19
20     def initGui(self):
21         self.options_factory = MyPluginOptionsFactory()

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

22     self.options_factory.setTitle(self.tr('My Plugin'))
23     iface.registerOptionsWidgetFactory(self.options_factory)
24
25     def unload(self):
26         iface.unregisterOptionsWidgetFactory(self.options_factory)

```

💡 팁

레이어 속성 대화창에 사용자 정의 탭 추가하기

비슷한 논리를 적용해서 `QgsMapLayerConfigWidgetFactory` 및 `QgsMapLayerConfigWidget` 클래스를 사용하면 레이어 속성 대화창에 플러그인 사용자 정의 옵션을 추가할 수 있습니다.

16.2.4 레이어 트리에 있는 레이어에 사용자 정의 위젯 내장시키기

Layers 패널에 있는 레이어 항목 옆 또는 아래에 표시되는 일반 레이어 심볼 요소 이외에도, 레이어에 자주 쓰이는 (필터링 설정, 선택, 스타일, 버튼 위젯으로 레이어 새로고침, 시간 슬라이드 바를 바탕으로 레이어 생성, 또는 라벨에 추가 레이어 정보만 보이게 하기, ...) 몇몇 액션에 빠르게 접근할 수 있게 해주는 여러분의 위젯을 추가할 수 있습니다. 이 소위 레이어 트리 내장 위젯은 개별 레이어에 대한 레이어 속성 대화창의 *Legend* 탭을 통해 사용할 수 있습니다.

다음 코드 조각은 범례 안에 레이어에 사용할 수 있는 레이어 스타일을 보여줘서 서로 다른 레이어 스타일들을 빠르게 전환할 수 있는 드롭 다운 메뉴를 생성합니다.

```

1 class LayerStyleComboBox(QComboBox):
2     def __init__(self, layer):
3         QComboBox.__init__(self)
4         self.layer = layer
5         for style_name in layer.styleManager().styles():
6             self.addItem(style_name)
7
8         idx = self.findText(layer.styleManager().currentStyle())
9         if idx != -1:
10             self.setCurrentIndex(idx)
11
12         self.currentIndexChanged.connect(self.on_current_changed)
13
14     def on_current_changed(self, index):
15         self.layer.styleManager().setCurrentStyle(self.itemText(index))
16
17 class LayerStyleWidgetProvider(QgsLayerTreeEmbeddedWidgetProvider):
18     def __init__(self):
19         QgsLayerTreeEmbeddedWidgetProvider.__init__(self)
20
21     def id(self):
22         return "style"
23
24     def name(self):
25         return "Layer style chooser"
26
27     def createWidget(self, layer, widgetIndex):
28         return LayerStyleComboBox(layer)
29

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

30 def supportsLayer(self, layer):
31     return True    # any layer is fine
32
33 provider = LayerStyleWidgetProvider()
34 QgsGui.layerTreeEmbeddedWidgetRegistry().addProvider(provider)

```

그러면 지정한 레이어의 *Legend* 속성 탭에서, *Available widgets*로부터 *Layer style chooser*를 *Used widgets*로 드래그하면 해당 위젯이 레이어 트리에 활성화됩니다. 내장 위젯은 항상 관련 레이어 노드 하위 항목의 최상단에 표시됩니다.

예를 들어 플러그인 내부에서 위젯을 사용하고 싶은 경우, 다음과 같이 추가하면 됩니다:

```

1 layer = iface.activeLayer()
2 counter = int(layer.customProperty("embeddedWidgets/count", 0))
3 layer.setCustomProperty("embeddedWidgets/count", counter+1)
4 layer.setCustomProperty("embeddedWidgets/{}".format(counter), "style")
5 view = self.iface.layerTreeView()
6 view.layerTreeModel().refreshLayerLegend(view.currentLegendNode())
7 view.currentNode().setExpanded(True)

```

16.3 플러그인 작성 및 디버그 작업을 위한 IDE 설정

프로그래머마다 선호하는 IDE/텍스트 편집기가 있을 테지만, 이 장에서는 QGIS 파이썬 플러그인을 작성하고 디버그하기 위한 유명한 IDE 몇 개를 추천합니다.

16.3.1 파이썬 플러그인 작성에 유용한 플러그인들

파이썬 플러그인 작성을 편리하게 해주는 플러그인이 몇 개 있습니다. *Plugins ▢ Manage and Install plugins...* 메뉴에서 다음을 설치하십시오:

- 플러그인 다시 불러오기 (*Plugin reloader*): QGIS를 다시 시작하지 않고서도 플러그인을 다시 불러와 새로운 변경 사항을 적용할 수 있게 해줍니다.
- 응급 처치 (*First Aid*): 플러그인에서 예외가 발생한 경우 변수들을 조사할 수 있는 파이썬 콘솔과 로컬 디버그 작업자를 추가합니다.

경고

지속적으로 애를 쓰고 있지만, 이 줄 다음에 나오는 정보들이 QGIS 3 버전에 대해 업데이트되지 않았을 수도 있습니다.

16.3.2 리눅스 및 윈도우 상에서 사용자의 IDE 환경설정에 대한 첨언

리눅스 상에서, 일반적으로 해야 하는 일은 사용자의 PYTHONPATH 환경 변수에 QGIS 라이브러리 위치를 추가하는 것이 전부입니다. 대부분의 배포판들에서는 ~/.bashrc 또는 ~/.bash-profile 파일을 다음 줄을 사용해서 편집하면 됩니다. (오픈수세 텀블위드 (openSUSE Tumbleweed) 버전에서 테스트했습니다.):

```
export PYTHONPATH="$PYTHONPATH:/usr/share/qgis/python/plugins:/usr/share/qgis/python"
```

파일을 저장하고, 다음 셸 명령어를 사용해서 환경 설정을 구현하십시오:

```
source ~/.bashrc
```

윈도우 상에서는, 여러분이 QGIS 와 동일한 환경을 설정했는지 그리고 동일한 라이브러리와 해석기를 사용하고 있는지 확인해야 합니다. QGIS 의 구동 배치 (batch) 파일을 수정하는 것이 가장 빠른 방법입니다.

OSGeo4W 설치자를 사용한 경우, 여러분의 OSGeo4W 설치 위치의 bin 폴더에서 이 파일을 찾을 수 있습니다. C:\OSGeo4W\bin\qgis-unstable.bat 같은 경로를 검색해보십시오.

16.3.3 파이스크립터 IDE 를 사용한 디버그 작업 (윈도우)

파이스크립터 (Pyscripter) IDE 를 사용하려면, 다음 작업을 해야 합니다:

1. qgis-unstable.bat 파일을 복사한 다음 복사본의 이름을 pyscripter.bat 으로 바꾸십시오.
2. 편집기에서 이 파일을 연 다음, QGIS 를 구동하는 마지막 줄을 제거하십시오.
3. 파이스크립터 실행 파일을 가리키는 줄을 추가한 다음 사용할 파이썬 버전을 설정하는 명령줄 인자를 추가하십시오.
4. 또 파이스크립터가 QGIS 가 사용하는 파이썬 DLL 파일을 찾을 수 있는 폴더를 가리키는 인자를 추가하십시오. 이 폴더는 OSGeo4W 설치 위치의 bin 폴더에서 찾을 수 있습니다.

```
@echo off
SET OSGEO4W_ROOT=C:\OSGeo4W
call "%OSGEO4W_ROOT%"\bin\o4w_env.bat
call "%OSGEO4W_ROOT%"\bin\gdal16.bat
@echo off
path %PATH%;%GISBASE%\bin
Start C:\pyscripter\pyscripter.exe --python25 --pythondllpath=C:\OSGeo4W\bin
```

5. 이제 이 배치 파일을 더블 클릭하면 정확한 경로를 가진 파이스크립터를 시작할 것입니다.

파이스크립터보다 더 유명한 이클립스 (Eclipse) 는 개발자들이 흔히 선택하곤 하는 IDE 입니다. 다음 항에서는 플러그인을 개발하고 테스트하기 위해 이클립스를 어떻게 환경설정해야 하는지를 설명할 것입니다.

16.3.4 이클립스 및 파이데브를 사용한 디버그 작업

설치

이클립스를 사용하기 위해서는 다음을 설치했는지 확인해야 합니다:

- Eclipse
- Aptana Studio 3 Plugin or PyDev
- QGIS 3.x

- QGIS 플러그인 원격 디버그 (**Remote Debug**) 를 설치하려 할 수도 있습니다. 현 시점에서 이 플러그인은 아직 실험적이기 때문에, 설치하기 전에 *Plugins ▸ Manage and Install plugins...* ▸ *Options* 메뉴에서  *Experimental plugins* 옵션을 활성화시키십시오.

윈도우에서 이클립스를 사용하기 위한 환경을 준비하려면, 이클립스를 구동하는 배치 파일도 생성해서 사용해야 합니다:

1. qgis_core.dll 파일이 있는 폴더를 찾으십시오. 일반적으로 C:\OSGeo4W\apps\qgis\bin 폴더이지만, 여러분이 직접 QGIS 응용 프로그램을 컴파일한 경우 이 폴더는 빌드 폴더 아래 있는 output/bin/RelWithDebInfo 폴더입니다.
2. eclipse.exe 실행 파일의 위치를 찾으십시오.
3. 다음 스크립트를 생성한 다음 QGIS 플러그인 개발 시 이 스크립트를 사용해서 이클립스를 구동하십시오.

```
call "C:\OSGeo4W\bin\o4w_env.bat"
set PATH=%PATH%;C:\path\to\your\qgis_core.dll\parent\folder
start /B C:\path\to\your\eclipse.exe
```

이클립스 설정하기

1. 이클립스에서 새 프로젝트를 생성하십시오. 일반 프로젝트 (*General Project*) 를 선택한 다음 나중에 실제 리소스를 링크시킬 수 있습니다. 즉 여러분이 이 프로젝트를 어디에 배치하느냐는 그렇게 중요하지 않습니다.
2. 새 프로젝트를 오른쪽 클릭한 다음 *New ▸ Folder* 메뉴를 선택하십시오.
3. *Advanced* 를 클릭하고 *Link to alternate location (Linked Folder)* 를 선택하십시오. 여러분이 이미 디버그하고자 하는 소스를 가지고 있는 경우 해당 리소스를 선택하십시오. 그렇지 않다면, 앞에서 설명한 것처럼 폴더를 하나 생성하십시오.

이제 *Project Explorer* 뷰에 여러분의 소스 트리가 뜨고 여러분이 코드 작업을 시작할 수 있습니다. 처음부터 문법 강조 및 다른 모든 강력한 IDE 도구들을 사용할 수 있습니다.

디버그 작업자 환경설정하기

디버그 작업자를 작동하게 하려면:

1. 이클립스에서 디버그 관점으로 전환합니다. (*Window ▸ Open Perspective ▸ Other ▸ Debug*)
2. *PyDev ▸ Start Debug Server* 메뉴를 선택해서 파이테브 디버그 서버를 시작합니다.
3. 이클립스는 이제 QGIS 가 디버그 서버로 연결하기를 기다립니다. QGIS 가 디버그 서버에 연결되면 QGIS 는 이클립스가 파이썬 스크립트를 제어할 수 있게 해줄 것입니다. 이것이 바로 우리가 원격 디버그 플러그인을 설치한 이유입니다. 따라서 QGIS 를 아직 시작하지 않았다면 QGIS 를 시작한 다음 버그 심볼을 클릭하십시오.

이제 중단점 (breakpoint) 을 설정할 수 있기 때문에, 코드가 중단점에 도달하는 순간 실행을 멈추고 여러분의 플러그인의 현재 상태를 조사할 수 있습니다. (중단점은 다음 그림에 있는 녹색 점으로, 중단점을 설정하려는 줄 왼쪽의 하얀 공간을 더블 클릭해서 설정합니다.)

이제 디버그 콘솔이라는 매우 재미있는 물건을 사용할 수 있게 되었습니다. 디버그 콘솔로 진행하기 전에 먼저 실행이 현재 중단점에서 중단된 상태인지 확인하십시오.

1. 콘솔 뷰를 (*Window ▸ Show view*) 여십시오. 그러면 *Debug Server* 콘솔이 열리는데, 그렇게 재미있어 보이지만 아닙니다. 하지만 *Open Console* 버튼을 누르면 좀 더 재미있는 파이테브 디버그 콘솔로 전환시킬 수 있습니다.
2. *Open Console* 버튼 옆에 있는 화살표를 클릭한 다음 파이테브 콘솔 (*PyDev Console*) 을 선택하십시오. 여러분에게 어떤 콘솔을 시작하고 싶은지 묻는 창이 열립니다.

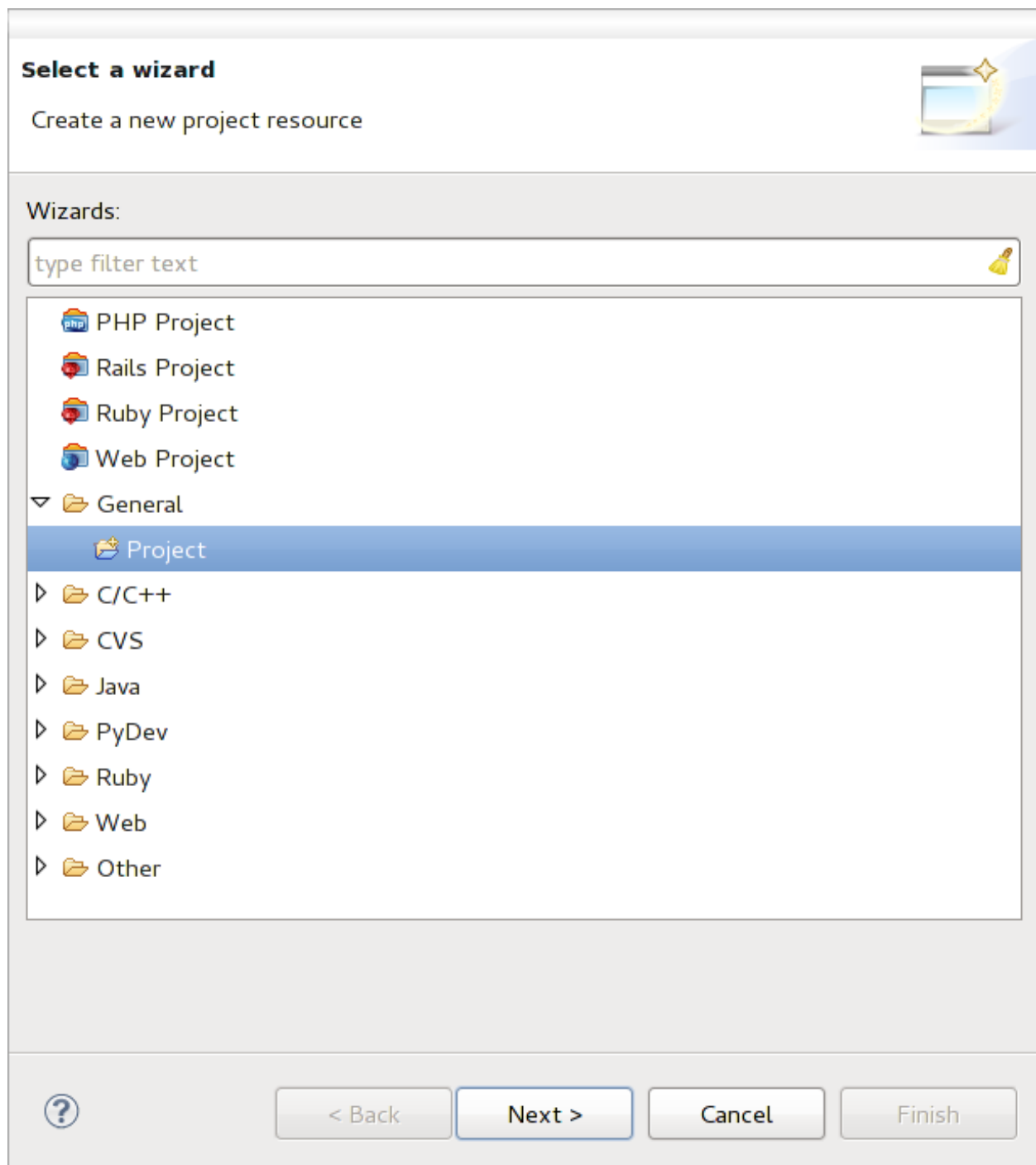


그림 16.1: 이클립스 프로젝트

```

87         self.verticalExaggerationChanged.emit(val)
88
89     def printProfile(self):
90         printer = QPrinter( QPrinter.HighResolution )
91         printer.setOutputFormat( QPrinter.PdfFormat )
92         printer.setPaperSize( QPrinter.A4 )
93         printer.setOrientation( QPrinter.Landscape )
94
95         printPreviewDlg = QPrintPreviewDialog( )
96         printPreviewDlg.paintRequested.connect( self.printRequested )
97
98         printPreviewDlg.exec_()
99
100     @pyqtSlot( QPrinter )
101     def printRequested( self, printer ):
102         self.webView.print_( printer )

```

그림 16.2: 중단점

3. 파이데브 디버그 콘솔 (*PyDev Debug Console*) 을 선택하십시오. 이 항목이 회색조로 비활성화되어 있고 디버그 작업을 시작해서 무결한 프레임을 선택하라는 메시지를 보이는 경우, 원격 디버그 플러그인을 활성화했는지 그리고 현재 중단점에서 중단된 상태인지 확인하십시오.

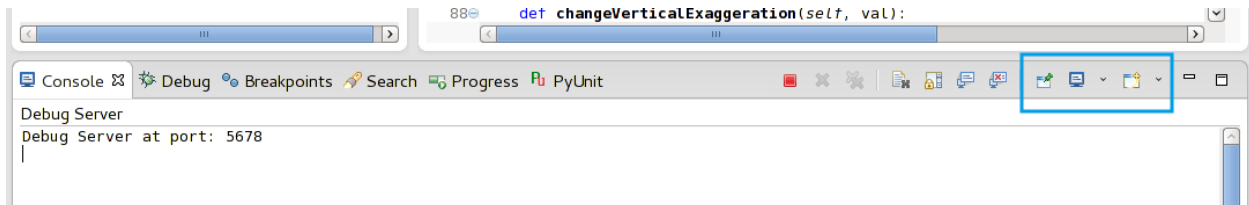


그림 16.3: 파이데브 디버그 콘솔

이제 여러분이 현재 맥락 안에서 어떤 명령어든 테스트할 수 있게 해주는 쌍방향 작업 콘솔이 열렸습니다. 변수를 조작하거나 API 를 호출하거나 원하는 것은 무엇이든 할 수 있습니다.

💡 팁

조금 짜증스러운 사실은, 여러분이 명령어를 입력할 때마다 콘솔이 디버그 서버로 다시 전환된다는 점입니다. 이 습성을 중단시키려면, 디버그 서버 페이지에 있을 때 콘솔 고정 (*Pin Console*) 버튼을 클릭하면 됩니다. 적어도 현재 디버그 세션 동안에는 이 결정을 기억할 것입니다.

이클립스에 API 를 이해시키기

이클립스가 실제로 QGIS API 에 대해 알도록 해주는 매우 편리한 기능이 있습니다. 이 기능은 이클립스가 코드의 오타를 점검할 수 있게 해줍니다. 이뿐만이 아니라, 이클립스가 가져오기로부터 API 호출에 자동 완성 기능을 사용할 수 있게 해줍니다.

이클립스는 이를 위해 QGIS 라이브러리 파일들을 파싱해서 모든 정보를 가져옵니다. 여러분이 해야 할 일은 이클립스에 라이브러리의 위치를 알려주는 것뿐입니다.

1. Window ▸ Preferences ▸ PyDev ▸ Interpreter ▸ Python 메뉴를 클릭하십시오.

창 윗부분에 사용자가 환경설정한 (현 시점에서는 QGIS 용 파이썬 2.7) 파이썬 해석기가 나타날 것입니다. 아랫부분에는 탭 몇 개가 보일 것입니다. 여기서 흥미로운 탭은 라이브러리 (*Libraries*) 와 강제 내장 (*Forced Builtins*) 탭입니다.

2. 먼저 라이브러리 탭을 여십시오.

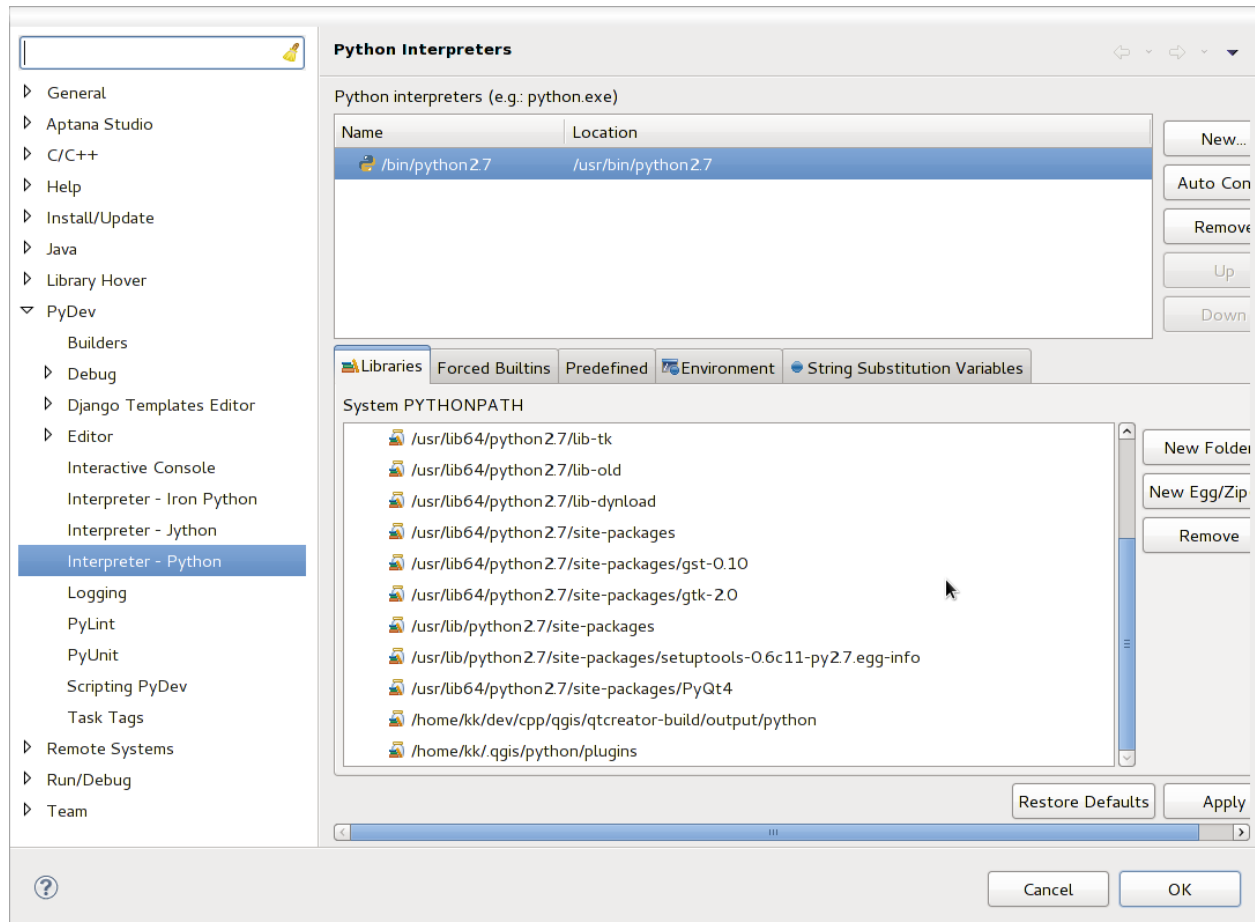


그림 16.4: 파이데브 디버그 콘솔

3. *New Folder* 를 클릭하고 QGIS 설치본의 파이썬 폴더를 선택하십시오. 이 폴더의 위치를 모르는 경우 (플러그인 폴더가 아닙니다):
 1. QGIS 를 여십시오.
 2. 파이썬 콘솔을 시작하십시오.
 3. `qgis` 를 입력하십시오.
 4. 그리고 엔터 키를 누르십시오. 파이썬이 사용하는 QGIS 모듈과 그 경로를 출력할 것입니다.
 5. 이 경로 맨 뒤에 있는 `/qgis/___init___.pyc` 문자열을 제거한 경로가 파이썬 폴더를 가리키는 경로입니다.
4. 여기에 플러그인 폴더도 추가해야 합니다. (사용자 프로파일 폴더 아래 있는 `python/plugins` 폴더입니다.)
5. 그 다음 강제 내장 탭으로 가서 *New...*를 클릭한 다음 `qgis` 를 입력하십시오. 그러면 이클립스가 QGIS API 를 파싱할 것입니다. 이클립스가 PyQt API 에 대해서도 알고 있기를 원할 수도 있습니다. 따라서 PyQt 도 강제 내장으로 추가하십시오. PyQt 는 이미 라이브러리 탭에 있을 것입니다.
6. *OK* 를 클릭하면 끝입니다.

참고

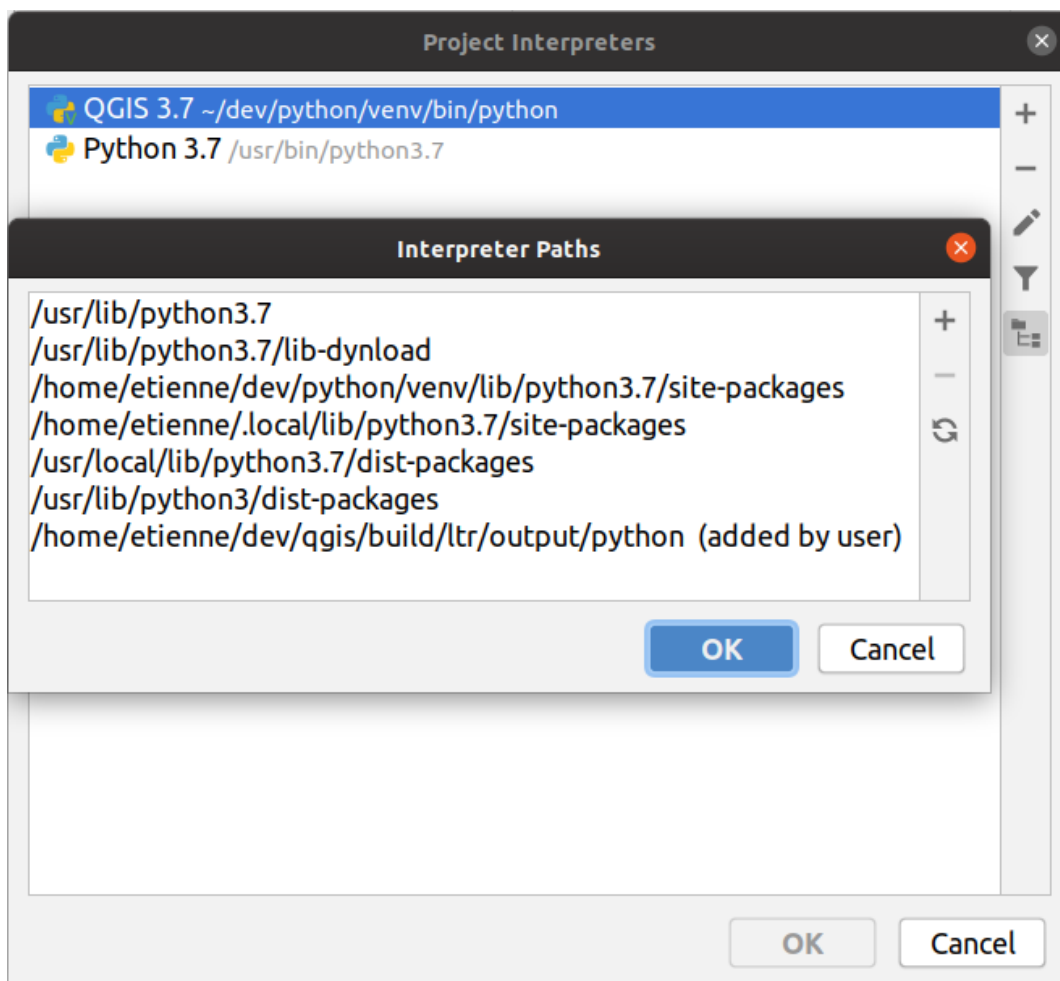
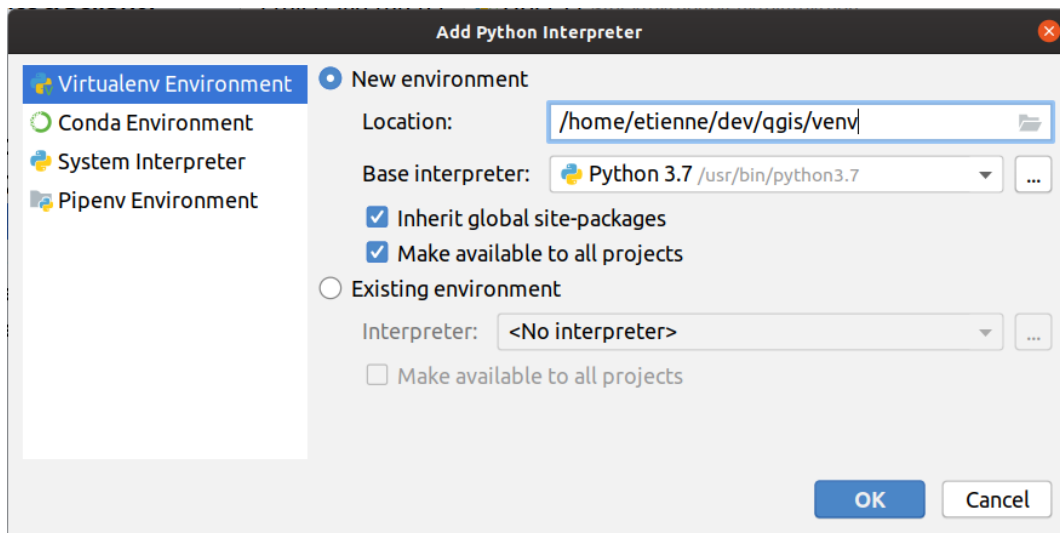
QGIS API 가 변경될 때마다 (예를 들면 QGIS 마스터 분기를 컴파일하는 도중 SIP 파일이 변경되는 경우) 다시 이 페이지로 돌아가서 *Apply* 를 클릭하기만 하십시오. 이클립스가 모든 라이브러리를 다시 파싱할 것입니다.

16.3.5 QGIS 가 컴파일된 우분투에서 파이참을 사용한 디버그 작업

파이참 (PyCharm) 은 젯브레인스 (JetBrains) 에서 개발한 파이썬 용 IDE 입니다. 커뮤니티 에디션이라는 무료 버전과 프로페셔널 에디션이라는 유료 버전이 있습니다. <https://www.jetbrains.com/pycharm/download> 웹사이트에서 파이참을 다운로드할 수 있습니다.

여러분이 우분투 상에 지정한 `~/dev/qgis/build/master` 빌드 디렉터리를 사용해서 QGIS 를 컴파일했다고 가정합니다. 여러분이 직접 QGIS 를 컴파일해야만 하는 것은 아니지만, 컴파일된 QGIS 에 대해서만 테스트되었습니다. QGIS 를 직접 컴파일하지 않은 경우, 경로들을 조정해야만 합니다.

1. 파이참의 *Project Properties* 및 *Project Interpreter* 에서 QGIS 라는 파이썬 가상 환경을 생성할 것입니다.
2. 작은 톱니바퀴 아이콘을 클릭한 다음 *Add* 를 클릭하십시오.
3. *Virtualenv environment* 를 선택하십시오.
4. `~/dev/qgis/venv` 같은 여러분의 모든 파이썬 프로젝트의 일반 위치를 선택하십시오. 모든 플러그인에 대해 이 파이썬 해석기를 사용할 것이기 때문입니다.
5. 여러분의 시스템 상에서 사용할 수 있는 파이썬 3 버전 기반 해석기를 선택하고 다음 *Inherit global site-packages* 및 *Make available to all projects* 옵션 2 개를 체크하십시오.
1. *OK* 를 클릭한 다음, 다시 작은 톱니바퀴 아이콘으로 돌아와 *Show all* 을 클릭하십시오.
2. 새 창이 열리면, 새 QGIS 해석기를 선택한 다음 *Show paths for the selected interpreter* 수직 메뉴의 마지막 아이콘을 클릭하십시오.
3. 마지막으로, 목록에 다음 `~/dev/qgis/build/master/output/python` 절대 경로를 추가하십시오.
1. 파이참을 다시 시작하면 여러분의 모든 플러그인에 대해 이 새로운 파이썬 가상 환경을 사용할 수 있습니다.



파이썬은 QGIS API 는 물론 `from qgis.PyQt.QtCore import QDir` 같이 QGIS 가 제공하는 Qt 를 사용하는 경우 PyQt API 도 인식하게 될 것입니다.

파이썬의 프로페셔널 에디션에서는 원격 디버그 작업도 잘 작동합니다. 커뮤니티 에디션의 경우, 원격 디버거를 사용할 수 없습니다. 로컬 디버거 작업자에만 접근할 수 있는데, QGIS 자체에서가 아니라 파이썬 내부에서 (스크립트로써 또는 `unittest` 로) 코드를 실행해야만 한다는 뜻입니다. QGIS 안에서 실행되는 파이썬 코드의 경우, 앞에서 설명한 응용 처치 플러그인을 사용할 수도 있습니다.

16.3.6 PDB 를 사용한 디버그 작업

이클립스 또는 파이썬 같은 IDE 를 사용하지 않는 경우, 다음 단계를 따라 PDB(Python Debugger) 를 사용해서 디버그할 수 있습니다.

1. 먼저 디버그하고자 하는 위치에 다음 코드를 추가하십시오:

```
# Use pdb for debugging
import pdb
# also import PyQtRemoveInputHook
from qgis.PyQt.QtCore import PyQtRemoveInputHook
# These lines allow you to set a breakpoint in the app
PyQtRemoveInputHook()
pdb.set_trace()
```

2. 그리고 명령줄에서 QGIS 를 실행하십시오.

리눅스에서:

```
$ ./Qgis
```

맥 OS 에서:

```
$ /Applications/Qgis.app/Contents/MacOS/Qgis
```

3. 그러면 응용 프로그램이 중단점에 도달했을 때 콘솔에 입력할 수 있게 됩니다!

할 일:

테스트 정보 추가하기

16.4 사용자 플러그인 배포하기

여러분의 플러그인이 준비되었고 이 플러그인이 몇몇 사람에게 도움이 될 거라 생각한다면, 주지 말고 공식 파이썬 플러그인 저장소 에 업로드하십시오. 해당 페이지에서 플러그인 설치자와 잘 작동하도록 플러그인을 준비하는 방법에 대한 패키지 작업 지침도 찾아볼 수 있습니다. 또는 사용자 자신의 플러그인 저장소를 준비하고 싶은 경우, 플러그인과 그 메타데이터를 목록화할 단순 XML 파일을 생성하십시오.

다음 권고 사항들에 특히 신경을 써주십시오:

16.4.1 메타데이터와 이름

- 기존 플러그인과 너무 비슷한 이름을 사용하지 말아주십시오.
- 여러분의 플러그인이 기존 플러그인과 유사한 기능을 가지고 있는 경우, 사용자가 플러그인을 설치해서 테스트할 필요 없이 어떤 플러그인을 사용해야 할지 알 수 있도록 'About' 필드에 차이점을 설명해주십시오.
- 플러그인 이름 자체에 "plugin"을 반복하지 말아주십시오.
- 메타데이터의 'Description' 필드에는 한 줄 설명을 그리고 'About' 필드에는 좀 더 상세한 지침을 작성해주십시오.
- 코드 저장소, 버그 추적기, 그리고 홈페이지를 포함시켜주십시오. 그렇게 하면 공동 작업의 가능성을 크게 높여주며, 사용할 수 있는 웹 인프라스트럭처 (깃허브 (Github), 깃랩 (Gitlab), 비트버킷 (Bitbucket) 등등) 가운데 하나를 사용해서 공동 작업을 매우 쉽게 할 수 있습니다.
- 주의를 기울여 태그를 선택하십시오: 쓸데없는 정보를 주는 (예: 벡터) 태그를 피하고 다른 사람들이 이미 사용하고 있는 태그를 쓰는 편이 좋습니다. (플러그인 웹사이트를 참조하세요.)
- 기본 아이콘을 내버려두지 말고 제대로 된 아이콘을 추가해주십시오. 사용할 모범적인 스타일을 알고 싶다면 QGIS 인터페이스를 참조하세요.

16.4.2 코드와 도움말

- 저장소에 생성된 파일 (ui_*.py, resources_rc.py, 생성된 도움말 파일, ...) 및 쓸모없는 것 (예: .gitignore) 들을 포함시키지 말아주십시오.
- 알맞은 메뉴 (Vector, Raster, Web, Database) 에 플러그인을 추가하십시오.
- 적절한 경우 (플러그인이 분석을 수행하는 경우) 플러그인을 공간 처리 프레임워크의 하위 플러그인으로 추가하는 것을 고려해주십시오. 그러면 사용자가 플러그인을 배치 (batch) 모드로 실행하거나 좀 더 복잡한 워크플로에 통합시킬 수 있고, 여러분이 인터페이스를 설계할 필요가 없어집니다.
- 적어도 최소한의 문서를 그리고 테스트 및 사용자 이해에 도움이 된다면 샘플 데이터를 포함시켜주십시오.

16.4.3 공식 파이썬 플러그인 저장소

공식 파이썬 플러그인 저장소는 <https://plugins.qgis.org/> 입니다.

공식 저장소를 사용하려면 [OSGEO 웹 포털](#) 에서 OSGEO ID 를 만들어야만 합니다.

여러분의 플러그인을 업로드하고 나면 스태프 멤버가 이를 승인하고 여러분에게 알려줄 것입니다.

할 일:

관리 문서 (governance document) 를 가리키는 링크 삽입하기

권한

공식 플러그인 저장소에는 다음 규칙들이 구현되어 있습니다:

- 등록된 사용자 모두가 새 플러그인을 추가할 수 있습니다.
- 스태프 사용자는 모든 플러그인 버전들을 승인하거나 승인 취소할 수 있습니다.
- `plugins.can_approve` 특수 권한을 가진 사용자가 업로드한 버전은 자동으로 승인됩니다.
- `plugins.can_approve` 특수 권한을 가진 사용자는 플러그인 소유자 (owner) 목록에 들어가 있는 한 다른 사람이 업로드한 버전을 승인할 수 있습니다.

- 스택 멤버와 해당 플러그인 소유자 만이 플러그인을 삭제하고 편집할 수 있습니다.
- `plugins.can_approve` 특수 권한이 없는 사용자가 새 버전을 업로드한 경우, 해당 플러그인 버전은 자동으로 승인 취소됩니다.

신뢰 관리

스택 멤버들은 프론트엔드 응용 프로그램을 통해 선택된 플러그인 개발자들에게 `plugins.can_approve` 권한을 설정해서 신뢰 (*trust*) 를 부여할 수 있습니다.

플러그인 상세 정보 뷰는 플러그인 개발자 또는 플러그인 소유자 에게 신뢰를 부여할 수 있는 직접 링크를 제공합니다.

무결성 검증

플러그인이 업로드되면 압축 패키지에서 플러그인의 메타데이터를 자동으로 가져와서 무결성을 검증합니다.

공식 저장소에 플러그인을 업로드하려 할 때 알고 있어야 할 무결성 검증 규칙 몇 개를 소개합니다:

1. 여러분의 플러그인을 담고 있는 주 폴더의 이름은 오직 ASCII 문자 (A-Z 및 a-z), 숫자, 그리고 밑줄 (`_`) 과 빼기 (`-`) 기호로만 이루어져야 하며, 숫자로 시작되어서는 안 됩니다.
2. `metadata.txt` 파일은 필수입니다.
3. 메타데이터 표 목록에 있는 필수 메타데이터가 전부 있어야만 합니다.
4. `version` 메타데이터 필드가 유일해야만 합니다.
5. 사용 허가서 파일을 반드시 포함시켜야만 합니다. 확장자 없이 (예를 들어 `LICENSE.txt` 가 아닌) `LICENSE` 라는 이름으로 저장하십시오.

플러그인 구조

무결성 검증 규칙에 따라 플러그인의 압축 (`.zip`) 패키지는 기능하는 플러그인으로써 승인받기 위한 특수 구조를 가져야만 합니다. 사용자의 플러그인 폴더 안에 플러그인을 압축 해제할 것이기 때문에, 다른 플러그인과 섞이지 않도록 `.zip` 파일 안에 자체 디렉터리를 가지고 있어야만 합니다. 필수 파일은 `metadata.txt`, `__init__.py` 그리고 `LICENSE` 입니다. 하지만 `README` 파일은 물론 플러그인을 표현할 아이콘도 있는 편이 좋습니다. 다음은 `plugin.zip` 파일이 어떻게 보이면 되는지에 대한 예시입니다.

```
plugin.zip
pluginfolder/
|-- i18n
|   |-- translation_file_de.ts
|-- img
|   |-- icon.png
|   |-- iconsource.svg
|-- __init__.py
|-- LICENSE
|-- Makefile
|-- metadata.txt
|-- more_code.py
|-- main_code.py
|-- README
|-- ui_Qt_user_interface_file.ui
```

파이썬 프로그래밍 언어로 플러그인을 생성할 수 있습니다. C++ 로 작성된 전통적인 플러그인에 비해 더 쉽게 플러그인을 작성하고, 이해하고, 유지하고 배포할 수 있습니다. 이는 파이썬 언어의 동적인 특성 덕분입니다.

파이썬 플러그인 목록은 QGIS 플러그인 관리자에서 C++ 플러그인과 함께 볼 수 있습니다. ~/ (UserProfile) / python/plugins 및 다음 경로에서 플러그인을 검색합니다:

- **UNIX/Mac:** (qgis_prefix) / share / qgis / python / plugins
- **Windows:** (qgis_prefix) / python / plugins

~ 및 (UserProfile) 이 어떻게 정의되는지 알고 싶다면 `core_and_external_plugins` 을 참조하세요.

i 참고

기존 디렉터리 경로에 `QGIS_PLUGINPATH` 를 설정하면, 플러그인 검색 경로 목록에 이 경로를 추가할 수 있습니다.

공간 처리 플러그인 작성하기

여러분이 개발하려는 플러그인의 유형에 따라, 플러그인의 기능을 공간 처리 알고리즘으로 (또는 공간 처리 알고리즘 집합으로) 추가하는 편이 더 나을 수도 있습니다. QGIS 안에 더 잘 통합시킬 수 있고, (모델 작성자 또는 배치 (batch) 공간 처리 인터페이스 같은 공간 처리 프레임워크 구성 요소에서 실행할 수 있기 때문에) 추가 기능을 제공할 수 있으며, (공간 처리 프레임워크가 작업의 상당 부분을 처리해줄 것이기 때문에) 개발 시간도 절약될 것입니다.

이런 알고리즘을 배포하려면, 공간 처리 툴박스에 알고리즘을 추가하는 새 플러그인을 생성해야 합니다. 이 플러그인은 알고리즘 제공자를 담고 있어야 하는데, 플러그인이 인스턴스화될 때 이 알고리즘 제공자를 등록해야 합니다.

17.1 플러그인을 업데이트하기

공간 처리 프레임워크에 여러분의 기존 플러그인을 추가하고 싶은 경우, 몇몇 코드를 추가해야 합니다.

1. metadata.txt 파일에 다음 변수를 추가해야 합니다:

```
hasProcessingProvider=yes
```

2. initGui 메소드를 사용해서 플러그인을 초기 설정하는 파이썬 파일에서 몇 줄을 다음과 같이 조정해줘야 합니다:

```
1 from qgis.core import QgsApplication
2 from .processing_provider.provider import Provider
3
4 class YourPluginName:
5
6     def __init__(self):
7         self.provider = None
8
9     def initProcessing(self):
10         self.provider = Provider()
11         QgsApplication.processingRegistry().addProvider(self.provider)
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

12
13     def initGui(self):
14         self.initProcessing()
15
16     def unload(self):
17         QgsApplication.processingRegistry().removeProvider(self.provider)

```

3. processing_provider 폴더를 생성하고 다음 파일 3 개를 생성할 수 있습니다:

- 아무 내용도 없는 __init__.py 파일—이 파일은 무결한 파이썬 패키지를 만들기 위해 필요합니다.
- provider.py 파일—이 파일이 공간 처리 제공자를 생성하고 여러분의 알고리즘을 알릴 것입니다.

```

1  from qgis.core import QgsProcessingProvider
2  from qgis.PyQt.QtGui import QIcon
3
4  from .example_processing_algorithm import ExampleProcessingAlgorithm
5
6  class Provider(QgsProcessingProvider):
7
8      """ The provider of our plugin. """
9
10     def loadAlgorithms(self):
11         """ Load each algorithm into the current provider. """
12         self.addAlgorithm(ExampleProcessingAlgorithm())
13         # add additional algorithms here
14         # self.addAlgorithm(MyOtherAlgorithm())
15
16     def id(self) -> str:
17         """The ID of your plugin, used for identifying the provider.
18
19         This string should be a unique, short, character only string,
20         eg "qgis" or "gdal". This string should not be localised.
21         """
22         return 'yourplugin'
23
24     def name(self) -> str:
25         """The human friendly name of your plugin in Processing.
26
27         This string should be as short as possible (e.g. "Lastools", not
28         "Lastools version 1.0.1 64-bit") and localised.
29         """
30         return self.tr('Your plugin')
31
32     def icon(self) -> QIcon:
33         """Should return a QIcon which is used for your provider inside
34         the Processing toolbox.
35         """
36         return QgsProcessingProvider.icon(self)

```

- example_processing_algorithm.py 파일—이 파일은 예시 알고리즘 파일을 담습니다. 이 파일에 스크립트 템플릿 파일의 내용을 복사/붙여넣기한 다음 여러분의 필요에 따라 업데이트하십시오.

You should have a tree similar to this:

```

1  └─ your_plugin_root_folder

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

2  |— __init__.py
3  |— LICENSE
4  |— metadata.txt
5  |— processing_provider
6      |— example_processing_algorithm.py
7      |— __init__.py
8      |— provider.py

```

4. 이제 QGIS 에 플러그인을 다시 불러오면 공간 처리 툴박스 와 모델 작성자 에서 여러분의 예시 스크립트를 볼 수 있을 것입니다.

17.1.1 Creating a custom algorithm

Here's a simple example of a custom buffer algorithm:

```

1  from qgis.core import (
2      QgsProcessingAlgorithm,
3      QgsProcessingParameterFeatureSource,
4      QgsProcessingParameterNumber,
5      QgsProcessingParameterFeatureSink,
6      QgsFeatureSink,
7  )
8
9  class BufferAlgorithm(QgsProcessingAlgorithm):
10
11      INPUT = 'INPUT'
12      DISTANCE = 'DISTANCE'
13      OUTPUT = 'OUTPUT'
14
15      def initAlgorithm(self, config=None):
16          self.addParameter(QgsProcessingParameterFeatureSource(self.INPUT, 'Input layer
17          ↪'))
18          self.addParameter(QgsProcessingParameterNumber(self.DISTANCE, 'Buffer distance
19          ↪', defaultValue=100.0))
20          self.addParameter(QgsProcessingParameterFeatureSink(self.OUTPUT, 'Output layer
21          ↪'))
22
23      def processAlgorithm(self, parameters, context, feedback):
24          source = self.parameterAsSource(parameters, self.INPUT, context)
25          distance = self.parameterAsDouble(parameters, self.DISTANCE, context)
26          (sink, dest_id) = self.parameterAsSink(parameters, self.OUTPUT, context,
27          ↪source.fields(), source.wkbType(),
28          ↪source.sourceCrs())
29
30          for f in source.getFeatures():
31              f.setGeometry(f.geometry().buffer(distance, 5))
32              sink.addFeature(f, QgsFeatureSink.FastInsert)
33
34          return {self.OUTPUT: dest_id}
35
36      def name(self):
37          return 'buffer'
38
39      def displayName(self):
40          return 'Buffer Features'

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
37
38     def group(self):
39         return 'Examples'
40
41     def groupId(self):
42         return 'examples'
43
44     def createInstance(self):
45         return BufferAlgorithm()
```

플러그인 레이어 사용하기

💡 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
1 from qgis.core import (
2     QgsPluginLayer,
3     QgsPluginLayerType,
4     QgsMapLayerRenderer,
5     QgsApplication,
6     QgsProject,
7 )
8
9 from qgis.PyQt.QtGui import QImage
```

여러분의 플러그인이 맵 레이어를 렌더링하기 위해 자신만의 메소드를 이용하는 경우, 이를 구현하는 데 `QgsPluginLayer` 클래스에 기반한 여러분만의 레이어 유형을 작성하는 것이 가장 좋을 수도 있습니다.

18.1 QgsPluginLayer 하위 클래스 만들기

다음은 최소한의 `QgsPluginLayer` 클래스를 구현하는 예시입니다. 워터마크 예제 플러그인 의 원본 코드를 기반으로 하고 있습니다.

이 구현의 일부로써, 사용자 정의 렌더링 작업자가 캔버스에 실제로 그려지는 내용을 정의합니다.

```
1 class WatermarkLayerRenderer(QgsMapLayerRenderer):
2
3     def __init__(self, layerId, rendererContext):
4         super().__init__(layerId, rendererContext)
5
6     def render(self):
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

7         image = QImage("/usr/share/icons/hicolor/128x128/apps/qgis.png")
8         painter = self.renderContext().painter()
9         painter.save()
10        painter.drawImage(10, 10, image)
11        painter.restore()
12        return True
13
14    class WatermarkPluginLayer(QgsPluginLayer):
15
16        LAYER_TYPE="watermark"
17
18        def __init__(self):
19            super().__init__(WatermarkPluginLayer.LAYER_TYPE, "Watermark plugin layer")
20            self.setValid(True)
21
22        def createMapRenderer(self, renderContext):
23            return WatermarkLayerRenderer(self.id(), renderContext)
24
25        def setTransformContext(self, ct):
26            pass
27
28        # Methods for reading and writing specific information to the project file can
29        # also be added:
30
31        def readXml(self, node, context):
32            pass
33
34        def writeXml(self, node, doc, context):
35            pass

```

다른 모든 맵 레이어와 마찬가지로, 프로젝트 및 캔버스에 플러그인 레이어를 추가할 수 있습니다.

```

plugin_layer = WatermarkPluginLayer()
QgsProject.instance().addMapLayer(plugin_layer)

```

이런 레이어를 담고 있는 프로젝트를 불러오는 경우, 팩토리 클래스가 필요합니다:

```

1    class WatermarkPluginLayerType(QgsPluginLayerType):
2
3        def __init__(self):
4            super().__init__(WatermarkPluginLayer.LAYER_TYPE)
5
6        def createLayer(self):
7            return WatermarkPluginLayer()
8
9        # You can also add GUI code for displaying custom information
10       # in the layer properties
11       def showLayerProperties(self, layer):
12           pass
13
14
15       # Keep a reference to the instance in Python so it won't
16       # be garbage collected
17       plt = WatermarkPluginLayerType()
18
19       assert QgsApplication.pluginLayerRegistry().addPluginLayerType(plt)

```


네트워크 분석 라이브러리

💡 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```
from qgis.core import (  
    QgsVectorLayer,  
    QgsPointXY,  
)
```

네트워크 분석 라이브러리는 다음에 사용할 수 있습니다:

- 지리 데이터 (폴리라인 벡터 레이어)로부터 수학적인 그래프를 생성할 수 있습니다.
- 그래프 이론으로부터 기본 메소드들을 (현재로서는 데이크스트라 (Dijkstra) 알고리즘만) 구현할 수 있습니다.

이 라이브러리의 전형적인 용도를 간단히 설명하면 다음과 같습니다:

1. 지리 데이터 (일반적으로 폴리라인 벡터 레이어)로부터 그래프 생성
2. 그래프 분석 실행
3. 분석 결과 이용 (예를 들어 분석 결과의 시각화 등)

19.1 그래프 작성하기

가장 먼저 해야 할 일은 입력 데이터를 준비하는, 다시 말해 벡터 레이어를 그래프로 변환하는 것입니다. 이후의 모든 작업은 레이어가 아니라 이 그래프를 사용할 것입니다.

As a source we can use any polyline vector layer. Nodes of the polylines become graph vertices, and segments of the polylines are graph edges. If several nodes have the same coordinates then they are the same graph vertex. So two lines that have a common node become connected to each other.

또, 그래프를 생성하는 도중에 입력 벡터 레이어에 추가적인 포인트를 몇 개라도 “고정 (fix)” (다른 용어로는 “결속 (tie)”) 시킬 수 있습니다. 각 추가 포인트에 대응하는, 가장 가까운 그래프 꼭짓점 또는 가장 가까운 그래프 변을 찾을 것입니다. 후자의 경우 변을 분할해서 새 꼭짓점을 추가합니다.

벡터 레이어의 속성과 변 길이를 그래프 변의 속성으로 쓸 수 있습니다.

빌더 프로그래밍 패턴을 사용해서 벡터 레이어를 그래프로 변환합니다. 소위 말하는 디렉터 (Director) 를 사용해서 그래프를 작성합니다. 현재 디렉터는 `QgsVectorLayerDirector` 클래스 하나뿐입니다. 디렉터는 라인 벡터 레이어로부터 그래프를 작성하는 데 쓰일 기본 설정을 설정하는데, 빌더가 이를 사용해서 그래프를 생성합니다. 현재 존재하는 빌더는 디렉터의 경우와 마찬가지로 `QgsGraph` 클래스 객체를 생성하는 `QgsGraphBuilder` 클래스 하나뿐입니다. 여러분은 `BGL` 또는 `NetworkX` 같은 라이브러리와 호환되는 그래프를 작성하는 사용자 정의 빌더를 구현하기를 원할 수도 있습니다.

To calculate edge properties the programming pattern `strategy` is used. For now only `QgsNetworkDistanceStrategy` strategy (that takes into account the length of the route) and `QgsNetworkSpeedStrategy` (that also considers the speed) are available. You can implement your own strategy that will use all necessary parameters.

이제 실제로 해봅시다.

1. First of all, to use this library we should import the analysis module:

```
from qgis.analysis import *
```

2. Then some examples for creating a director:

```
1 # Don't use information about road direction from layer attributes,
2 # all roads are treated as two-way
3 director = QgsVectorLayerDirector(
4     vectorLayer, -1, "", "", "", QgsVectorLayerDirector.DirectionBoth
5 )
```

```
1 # Use field with index 5 as source of information about road direction.
2 # one-way roads with direct direction have attribute value "yes",
3 # one-way roads with reverse direction have the value "1", and accordingly
4 # bidirectional roads have "no". By default roads are treated as two-way.
5 # This scheme can be used with OpenStreetMap data
6 director = QgsVectorLayerDirector(
7     vectorLayer, 5, "yes", "1", "no", QgsVectorLayerDirector.DirectionBoth
8 )
```

To construct a director, we should pass a vector layer that will be used as the source for the graph structure and information about allowed movement on each road segment (one-way or bidirectional movement, direct or reverse direction). The call looks like this (find more details on the parameters at `qgis.analysis.QgsVectorLayerDirector`):

```
1 director = QgsVectorLayerDirector(
2     vectorLayer,
3     directionFieldId,
4     directDirectionValue,
5     reverseDirectionValue,
6     bothDirectionValue,
7     defaultDirection,
8 )
```

3. 그 다음 변의 속성을 계산하기 위한 전략을 생성해야 합니다:

```
1 # The index of the field that contains information about the edge speed
2 attributeId = 1
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

3 # Default speed value
4 defaultValue = 50
5 # Conversion from speed to metric units ('1' means no conversion)
6 toMetricFactor = 1
7 strategy = QgsNetworkSpeedStrategy(attributeId, defaultValue, toMetricFactor)

```

4. 그리고 디렉터에 이 전략을 알려줍니다:

```

director = QgsVectorLayerDirector(vectorLayer, -1, "", "", "", 3)
director.addStrategy(strategy)

```

5. Now we can use the builder, which will create the graph, using the `QgsGraphBuilder` class constructor.

```

# only CRS is set, all other values are defaults
builder = QgsGraphBuilder(vectorLayer.crs())

```

6. Also we can define several points, which will be used in the analysis. For example:

```

startPoint = QgsPointXY(1179720.1871, 5419067.3507)
endPoint = QgsPointXY(1180616.0205, 5419745.7839)

```

7. Now all is in place so we can build the graph and “tie” these points to it:

```

tiedPoints = director.makeGraph(builder, [startPoint, endPoint])

```

Building the graph can take some time (which depends on the number of features in a layer and layer size). `tiedPoints` is a list with coordinates of “tied” points.

8. When the build operation is finished we can get the graph and use it for the analysis:

```

graph = builder.graph()

```

9. With the next code we can get the vertex indexes of our points:

```

startId = graph.findVertex(tiedPoints[0])
endId = graph.findVertex(tiedPoints[1])

```

19.2 그래프 분석

Networks analysis is used to find answers to two questions: which vertices are connected and how to find a shortest path. To solve these problems the network analysis library provides Dijkstra’s algorithm.

Dijkstra’s algorithm finds the shortest route from one of the vertices of the graph to all the others and the values of the optimization parameters. The results can be represented as a shortest path tree.

최단 경로 트리는 다음 속성들을 가진 방향성 가중치 그래프 (더 정확하게는 트리) 입니다:

- 들어오는 변이 없는 꼭짓점은 단 하나, 트리의 루트 (root) 뿐입니다.
- all other vertices have only one incoming edge
- 꼭짓점 A 에서 꼭짓점 B 에 도달할 수 있다면, A 에서 B 로의 경로는 사용할 수 있는 단 하나의 경로이며 이 그래프 상에서 최적 (최단) 경로입니다.

최단 경로 트리를 생성하려면 `QgsGraphAnalyzer` 클래스의 `shortestTree()` 와 `dijkstra()` 메소드를 사용하십시오. `dijkstra()` 메소드를 사용할 것을 권장합니다. 더 빠르고 메모리를 더 효율적으로 사용하기 때문입니다.

The `shortestTree()` method is useful when you want to walk around the shortest path tree. It always creates a new graph object (`QgsGraph`) and accepts three variables:

- `source` —입력 그래프
- `startVertexIdx` —트리 상에 있는 포인트의 인덱스 (트리의 루트)
- `criterionNum` —사용할 변 속성의 번호 (0 에서 시작)

```
tree = QgsGraphAnalyzer.shortestTree(graph, startId, 0)
```

The `dijkstra()` method has the same arguments, but returns a tuple of arrays:

- In the first array, element n contains index of the incoming edge or -1 if there are no incoming edges.
- In the second array, element n contains the distance from the root of the tree to vertex n or `DOUBLE_MAX` if vertex n is unreachable from the root.

```
(tree, cost) = QgsGraphAnalyzer.dijkstra(graph, startId, 0)
```

Here is some very simple code to display the shortest path tree using the graph created with the `shortestTree()` or the `dijkstra()` method (select linestring layer in *Layers* panel and replace coordinates with your own).

⚠ 경고

이 코드를 예제로써만 사용하십시오. 이 코드는 `QgsRubberBand` 클래스 객체를 많이 생성하기 때문에 대용량 데이터셋의 경우 느려질 수도 있습니다.

shortestTree()

```
1 from qgis.core import *
2 from qgis.gui import *
3 from qgis.analysis import *
4 from qgis.PyQt.QtCore import *
5 from qgis.PyQt.QtGui import *
6
7 vectorLayer = QgsVectorLayer(
8     "testdata/network.gpkg|layername=network_lines", "lines"
9 )
10 director = QgsVectorLayerDirector(
11     vectorLayer, -1, "", "", "", QgsVectorLayerDirector.DirectionBoth
12 )
13 strategy = QgsNetworkDistanceStrategy()
14 director.addStrategy(strategy)
15 builder = QgsGraphBuilder(vectorLayer.crs())
16
17 pStart = QgsPointXY(1179661.925139, 5419188.074362)
18 tiedPoint = director.makeGraph(builder, [pStart])
19 pStart = tiedPoint[0]
20
21 graph = builder.graph()
22
23 idStart = graph.findVertex(pStart)
24
25 tree = QgsGraphAnalyzer.shortestTree(graph, idStart, 0)
26
27 i = 0
28 while i < tree.edgeCount():
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

29     rb = QgsRubberBand(iface.mapCanvas())
30     rb.setColor(Qt.red)
31     rb.addPoint(tree.vertex(tree.edge(i).fromVertex()).point())
32     rb.addPoint(tree.vertex(tree.edge(i).toVertex()).point())
33     i = i + 1

```

dijkstra()

```

1  from qgis.core import *
2  from qgis.gui import *
3  from qgis.analysis import *
4  from qgis.PyQt.QtCore import *
5  from qgis.PyQt.QtGui import *
6
7  vectorLayer = QgsVectorLayer(
8      "testdata/network.gpkg|layername=network_lines", "lines"
9  )
10 director = QgsVectorLayerDirector(
11     vectorLayer, -1, "", "", "", QgsVectorLayerDirector.DirectionBoth
12 )
13 strategy = QgsNetworkDistanceStrategy()
14 director.addStrategy(strategy)
15 builder = QgsGraphBuilder(vectorLayer.crs())
16
17 pStart = QgsPointXY(1179661.925139, 5419188.074362)
18 tiedPoint = director.makeGraph(builder, [pStart])
19 pStart = tiedPoint[0]
20
21 graph = builder.graph()
22
23 idStart = graph.findVertex(pStart)
24 (tree, costs) = QgsGraphAnalyzer.dijkstra(graph, idStart, 0)
25
26 for edgeId in tree:
27     if edgeId == -1:
28         continue
29     rb = QgsRubberBand(iface.mapCanvas())
30     rb.setColor(Qt.red)
31     rb.addPoint(graph.vertex(graph.edge(edgeId).fromVertex()).point())
32     rb.addPoint(graph.vertex(graph.edge(edgeId).toVertex()).point())

```

19.2.1 최단 경로 찾기

두 포인트 사이의 최적 경로를 찾기 위해 다음 접근법을 사용합니다. 두 포인트 (시작 포인트 A 와 종단 포인트 B) 모두 작성 시 그래프에 “결속” 됩니다. 그 다음 `shortestTree()` 또는 `dijkstra()` 메소드를 사용해서 트리의 루트가 시작 포인트 A 인 최단 경로 트리를 생성합니다. 이 트리에서 종단 포인트 B 를 찾아, 트리 안에서 B 에서 A 로 가는 경로를 탐색합니다. 이 전체 알고리즘을 다음처럼 작성할 수 있습니다:

```

1  assign T = B
2  while T != B
3      add point T to path
4      get incoming edge for point T
5      look for point TT, that is start point of this edge
6      assign T = TT
7  add point A to path

```

At this point we have the path, in the form of the inverted list of vertices (vertices are listed in reversed order from end point to start point) that will be visited during traveling by this path.

Here is the sample code for QGIS Python Console (you may need to load and select a linestring layer in TOC and replace coordinates in the code with yours) that uses the `shortestTree()` or `dijkstra()` method:

`shortestTree()`

```

1  from qgis.core import *
2  from qgis.gui import *
3  from qgis.analysis import *
4
5  from qgis.PyQt.QtCore import *
6  from qgis.PyQt.QtGui import *
7
8  vectorLayer = QgsVectorLayer(
9      "testdata/network.gpkg|layername=network_lines", "lines"
10 )
11 director = QgsVectorLayerDirector(
12     vectorLayer, -1, "", "", "", QgsVectorLayerDirector.DirectionBoth
13 )
14 strategy = QgsNetworkDistanceStrategy()
15 director.addStrategy(strategy)
16
17 builder = QgsGraphBuilder(vectorLayer.sourceCrs())
18
19 startPoint = QgsPointXY(1179661.925139, 5419188.074362)
20 endPoint = QgsPointXY(1180942.970617, 5420040.097560)
21
22 tiedPoints = director.makeGraph(builder, [startPoint, endPoint])
23 tStart, tStop = tiedPoints
24
25 graph = builder.graph()
26 idxStart = graph.findVertex(tStart)
27
28 tree = QgsGraphAnalyzer.shortestTree(graph, idxStart, 0)
29
30 idxStart = tree.findVertex(tStart)
31 idxEnd = tree.findVertex(tStop)
32
33 if idxEnd == -1:
34     raise Exception("No route!")
35
36 # Add last point
37 route = [tree.vertex(idxEnd).point()]
38
39 # Iterate the graph
40 while idxEnd != idxStart:
41     edgeIds = tree.vertex(idxEnd).incomingEdges()
42     if len(edgeIds) == 0:
43         break
44     edge = tree.edge(edgeIds[0])
45     route.insert(0, tree.vertex(edge.fromVertex()).point())
46     idxEnd = edge.fromVertex()
47
48 # Display
49 rb = QgsRubberBand(iface.mapCanvas())
50 rb.setColor(Qt.green)
51

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

52 # This may require coordinate transformation if project's CRS
53 # is different from layer's CRS
54 for p in route:
55     rb.addPoint(p)

```

dijkstra()

```

1  from qgis.core import *
2  from qgis.gui import *
3  from qgis.analysis import *
4
5  from qgis.PyQt.QtCore import *
6  from qgis.PyQt.QtGui import *
7
8  vectorLayer = QgsVectorLayer(
9      "testdata/network.gpkg|layername=network_lines", "lines"
10 )
11 director = QgsVectorLayerDirector(
12     vectorLayer, -1, "", "", "", QgsVectorLayerDirector.DirectionBoth
13 )
14 strategy = QgsNetworkDistanceStrategy()
15 director.addStrategy(strategy)
16
17 builder = QgsGraphBuilder(vectorLayer.sourceCrs())
18
19 startPoint = QgsPointXY(1179661.925139, 5419188.074362)
20 endPoint = QgsPointXY(1180942.970617, 5420040.097560)
21
22 tiedPoints = director.makeGraph(builder, [startPoint, endPoint])
23 tStart, tStop = tiedPoints
24
25 graph = builder.graph()
26 idxStart = graph.findVertex(tStart)
27 idxEnd = graph.findVertex(tStop)
28
29 (tree, costs) = QgsGraphAnalyzer.dijkstra(graph, idxStart, 0)
30
31 if tree[idxEnd] == -1:
32     raise Exception('No route!')
33
34 # Total cost
35 cost = costs[idxEnd]
36
37 # Add last point
38 route = [graph.vertex(idxEnd).point()]
39
40 # Iterate the graph
41 while idxEnd != idxStart:
42     idxEnd = graph.edge(tree[idxEnd]).fromVertex()
43     route.insert(0, graph.vertex(idxEnd).point())
44
45 # Display
46 rb = QgsRubberBand(iface.mapCanvas())
47 rb.setColor(Qt.red)
48
49 # This may require coordinate transformation if project's CRS

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

50 # is different from layer's CRS
51 for p in route:
52     rb.addPoint(p)

```

19.2.2 도달 가능 영역

The area of availability for vertex A is the subset of graph vertices that are accessible from vertex A and the cost of the paths from A to these vertices are not greater than some value.

다음 질문을 통해 이를 더 명확히 알 수 있습니다. “소방서가 있다. 소방차가 5 분/10 분/15 분 안에 도착할 수 있는 도시의 구역은 어디인가?”이 질문에 대한 답이 바로 소방서의 도달 가능 영역입니다.

도달 가능 영역을 찾으려면 `QgsGraphAnalyzer` 클래스의 `dijkstra()` 메소드를 사용하면 됩니다. 비용 배열의 요소들을 사전 정의된 값과 비교하는 것으로 충분합니다. `cost[i]` 가 사전 정의 값 이하인 경우, 꼭짓점 `i` 는 도달 가능 영역 안에 있는 것이고, 초과하는 경우 밖에 있는 것입니다.

A more difficult problem is to get the borders of the area of availability. The bottom border is the set of vertices that are still accessible, and the top border is the set of vertices that are not accessible. In fact this is simple: it is the availability border based on the edges of the shortest path tree for which the source vertex of the edge is accessible and the target vertex of the edge is not.

Here is an example:

```

1  director = QgsVectorLayerDirector(
2      vectorLayer, -1, "", "", "", QgsVectorLayerDirector.DirectionBoth
3  )
4  strategy = QgsNetworkDistanceStrategy()
5  director.addStrategy(strategy)
6  builder = QgsGraphBuilder(vectorLayer.crs())
7
8
9  pStart = QgsPointXY(1179661.925139, 5419188.074362)
10 delta = iface.mapCanvas().getCoordinateTransform().mapUnitsPerPixel() * 1
11
12 rb = QgsRubberBand(iface.mapCanvas())
13 rb.setColor(Qt.green)
14 rb.addPoint(QgsPointXY(pStart.x() - delta, pStart.y() - delta))
15 rb.addPoint(QgsPointXY(pStart.x() + delta, pStart.y() - delta))
16 rb.addPoint(QgsPointXY(pStart.x() + delta, pStart.y() + delta))
17 rb.addPoint(QgsPointXY(pStart.x() - delta, pStart.y() + delta))
18
19 tiedPoints = director.makeGraph(builder, [pStart])
20 graph = builder.graph()
21 tStart = tiedPoints[0]
22
23 idStart = graph.findVertex(tStart)
24
25 (tree, cost) = QgsGraphAnalyzer.dijkstra(graph, idStart, 0)
26
27 upperBound = []
28 r = 1500.0
29 i = 0
30 tree.reverse()
31
32 while i < len(cost):
33     if cost[i] > r and tree[i] != -1:

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
34     outVertexId = graph.edge(tree [i]).toVertex()
35     if cost[outVertexId] < r:
36         upperBound.append(i)
37     i = i + 1
38
39 for i in upperBound:
40     centerPoint = graph.vertex(i).point()
41     rb = QgsRubberBand(iface.mapCanvas())
42     rb.setColor(Qt.red)
43     rb.addPoint(QgsPointXY(centerPoint.x() - delta, centerPoint.y() - delta))
44     rb.addPoint(QgsPointXY(centerPoint.x() + delta, centerPoint.y() - delta))
45     rb.addPoint(QgsPointXY(centerPoint.x() + delta, centerPoint.y() + delta))
46     rb.addPoint(QgsPointXY(centerPoint.x() - delta, centerPoint.y() + delta))
```


20.1 소개

QGIS 서버에 대해 더 알고 싶다면, *QGIS-Server-manual* 를 읽어보십시오.

QGIS 서버는 서로 다른 세 가지로 이루어져 있습니다:

1. QGIS 서버 라이브러리: OGC 웹 서비스를 생성하기 위한 API 를 제공하는 라이브러리입니다.
2. QGIS 서버 FCGI: 웹 서버와 함께 일련의 (WMS, WFS, WCS 등등의) OGC 서비스들 및 OGC API 들 (WFS3/OAPIF) 을 구현하는 FCGI 바이너리 응용 프로그램 `qgis_mapserv.fcgi` 파일
3. QGIS 개발 서버: 일련의 (WMS, WFS, WCS 등등의) OGC 서비스들 및 OGC API 들 (WFS3/OAPIF) 을 구현하는 개발 서버 바이너리 응용 프로그램 `qgis_mapserver` 파일

PyQGIS 쿡북의 이 장에서는 첫 번째 항목에 집중할 것입니다. QGIS 서버 API 의 사용 사례를 들어 파이썬을 사용해서 어떻게 서버의 습성을 확장하거나 향상시키거나 사용자 정의할 수 있는지를, 또는 QGIS 서버 API 를 사용해서 어떻게 또다른 응용 프로그램 안에 QGIS 서버를 내장시킬 수 있는지를 보여줄 것입니다.

QGIS 서버의 습성을 변경하거나, 새로운 사용자 정의 서비스 또는 API 를 제공하기 위해 QGIS 서버의 커스터마이징을 확장할 수 있는 서로 다른 방법들이 몇 가지 있습니다. 다음은 여러분이 마주할 수도 있는 주요 시나리오들입니다:

- EMBEDDING [\[7\]](#) Use QGIS Server API from another Python application
- STANDALONE [\[7\]](#) Run QGIS Server as a standalone WSGI/HTTP service
- FILTERS [\[7\]](#) Enhance/Customize QGIS Server with filter plugins
- SERVICES [\[7\]](#) Add a new *SERVICE*
- OGC APIs [\[7\]](#) Add a new *OGC API*

내장형 및 독립형 응용 프로그램들은 또다른 파이썬 스크립트 또는 응용 프로그램으로부터 QGIS 서버 파이썬 API 를 사용해야 합니다. 나머지 옵션들은 표준 QGIS 서버 바이너리 응용 프로그램 (FCGI 또는 개발 서버) 에 사용자 정의 기능들을 추가하고 싶은 경우 더 어울립니다. 이런 경우 서버 응용 프로그램 용 파이썬 플러그인을 작성한 다음 사용자 정의 필터, 서비스, 또는 API 를 등록해야 할 것입니다.

20.2 서버 API 기본 사항

전형적인 QGIS 서버 응용 프로그램과 관련된 기본 클래스는 다음과 같습니다:

- `QgsServer`: 서버 인스턴스 (일반적으로 응용 프로그램 전체 생애 주기에 대해 단일 인스턴스)
- `QgsServerRequest`: 요청 객체 (일반적으로 각 요청마다 다시 생성)
- `QgsServer.handleRequest(request, response)`: 이 메소드가 요청을 처리하고 응답을 채웁니다.

QGIS 서버 FCGI 또는 개발 서버 워크플로는 다음과 같이 요약할 수 있습니다:

```

1 initialize the QgsApplication
2 create the QgsServer
3 the main server loop waits forever for client requests:
4     for each incoming request:
5         create a QgsServerRequest request
6         create a QgsServerResponse response
7         call QgsServer.handleRequest(request, response)
8         filter plugins may be executed
9         send the output to the client

```

`QgsServer.handleRequest(request, response)` 메소드 안에서 필터 플러그인 콜백을 호출하고 `QgsServerInterface` 클래스를 통해 플러그인이 `QgsServerRequest` 및 `QgsServerResponse` 클래스들을 사용할 수 있게 됩니다.

⚠ 경고

QGIS 서버 클래스들은 스레드 안전 (thread safety) 하지 않기 때문에, QGIS 서버 API 를 기반으로 확장 가능한 (scalable) 응용 프로그램을 작성할 때 항상 다중 처리 (multiprocessing) 모델 또는 컨테이너를 사용해야 합니다.

20.3 독립형 또는 내장형

독립형 또는 내장형 서버 응용 프로그램의 경우, 앞에서 언급했던 서버 클래스들을 클라이언트와의 모든 HTTP 프로토콜 상호 작용들을 관리하는 웹 서버 구현으로 래핑 (wrapping) 해서 직접 사용해야 합니다.

QGIS 서버 API 사용 사례의 (HTTP 부분을 뺀) 최소한의 예시는 다음과 같습니다:

```

1 from qgis.core import QgsApplication
2 from qgis.server import *
3 app = QgsApplication([], False)
4
5 # Create the server instance, it may be a single one that
6 # is reused on multiple requests
7 server = QgsServer()
8
9 # Create the request by specifying the full URL and an optional body
10 # (for example for POST requests)
11 request = QgsBufferServerRequest(
12     'http://localhost:8081/?MAP=/qgis-server/projects/helloworld.qgs' +
13     '&SERVICE=WMS&REQUEST=GetCapabilities')
14
15 # Create a response objects

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

16 response = QgsBufferServerResponse()
17
18 # Handle the request
19 server.handleRequest(request, response)
20
21 print(response.headers())
22 print(response.body().data().decode('utf8'))
23
24 app.exitQgis()

```

다음은 QGIS 소스 코드 저장소 상에서 CI(continuous integration) 를 테스트하기 위해 개발된 완전한 독립형 응용 프로그램의 예시로써, 서로 다른 플러그인 필터들과 인증 스키마들의 광범위한 집합을 보여줍니다 (테스트 목적으로만 개발되었기 때문에 생산 용은 아니지만, 학습 용으로도 흥미롭습니다): [qgis_wrapped_server.py](#)

20.4 서버 플러그인

QGIS 서버 응용 프로그램을 구동하고 필터, 서비스, 또는 API 를 등록하기 위해 사용할 수 있게 되면, 서버 파이썬 플러그인을 불러옵니다.

서버 플러그인의 구조는 대응하는 데스크탑 플러그인과 매우 유사합니다. 플러그인이 `QgsServerInterface` 객체를 사용할 수 있게 해주고, 플러그인이 대응하는 레지스트리에 하나 이상의 사용자 정의 필터, 서비스, 또는 API 를 서버 인터페이스에서 볼 수 있는 메소드들 가운데 하나를 사용해서 등록할 수 있습니다.

20.4.1 서버 필터 플러그인

Filters come in three different flavors and they can be instantiated by subclassing one of the classes below and by calling the corresponding method of `QgsServerInterface`:

필터 유형	기반 클래스	QgsServerInterface 등록
I/O	<code>QgsServerFilter</code>	<code>registerFilter()</code>
접근 제어	<code>QgsAccessControlFilter</code>	<code>registerAccessControl()</code>
캐시	<code>QgsServerCacheFilter</code>	<code>registerServerCache()</code>

I/O 필터

I/O 필터는 (WMS, WFS 등등의) 핵심 서비스들의 서버 입력 및 출력을 (요청 및 응답을) 조정할 수 있기 때문에 서비스 워크플로를 어떤 유형으로든 조작할 수 있게 해줍니다. 예를 들면 선택한 레이어에의 접근을 제한할 수도 있고, XML 응답에 XSL 스타일시트를 주입 (inject) 할 수도 있으며, 생성한 WMS 이미지에 워터마크를 추가할 수도 있습니다.

이 시점에서 서버 플러그인 API 문서 를 한번 훑어보는 편이 유용할 수도 있습니다.

각 필터는 다음 콜백 3 개 가운데 적어도 하나를 구현해야 합니다:

- `onRequestReady()`
- `onResponseComplete()`
- `onSendResponse()`

모든 필터는 요청/응답 객체 (`QgsRequestHandler`)에 접근할 수 있고 이 객체의 모든 속성 (입력/출력)을 조작할 수 있으며 (다음에서 보게 될 아주 특수한 방식이긴 하지만) 예외를 발생시킬 수 있습니다.

이 메소드들은 모두 호출을 후속 필터들까지 전파해야 하는지 여부를 나타내는 불 (`boolean`) 값을 반환합니다. 이 메소드들 가운데 하나가 `False`를 반환하면 연쇄 전파가 중단되고, 그렇지 않으면 호출을 다음 필터로 전파할 것입니다.

다음은 서버가 전형적인 요청을 어떻게 처리하는지 그리고 언제 필터의 콜백을 호출하는지를 보여주는 의사 (`pseudo`) 코드입니다:

```

1 for each incoming request:
2     create GET/POST request handler
3     pass request to an instance of QgsServerInterface
4     call onRequestReady filters
5
6     if there is not a response:
7         if SERVICE is WMS/WFS/WCS:
8             create WMS/WFS/WCS service
9             call service's executeRequest
10            possibly call onSendResponse for each chunk of bytes
11            sent to the client by a streaming services (WFS)
12            call onResponseComplete
13            request handler sends the response to the client

```

다음 문단들에서 사용할 수 있는 콜백들을 자세히 설명합니다.

onRequestReady

요청이 준비됐을 때 이 콜백을 호출합니다. 들어오는 URL과 데이터를 파싱한 다음 핵심 서비스 (`WMS`, `WFS` 등등) 스위치를 입력하기 전, 이 때가 여러분이 입력을 조작하고 다음과 같은 액션들을 수행할 수 있는 시점입니다:

- 인증/승인
- 리다이렉트
- 특정 파라미터 (예를 들면 `typename`) 추가/제거
- 예외 발생

SERVICE 파라미터를 변경, 핵심 서비스를 완전히 우회 (`bypass`) 해서 핵심 서비스조차도 완전히 대체할 수 있습니다. (물론 큰 의미가 있는 방법은 아닙니다.)

onSendResponse

어떤 부분적인 출력이라도 응답 버퍼에서 (예를 들어 `FCGI` 서버를 사용하는 경우 `FCGI stdout`으로) 플러시 (`flush`) 될 때마다 이 콜백을 호출하고, 그 다음 클라이언트로 전달합니다. 대용량 콘텐츠가 스트리밍 될 때 (예: `WFS GetFeature`) 이런 일이 발생합니다. 이 경우 `onSendResponse()`를 여러 번 호출할 수도 있습니다.

응답을 스트리밍하지 않는 경우 `onSendResponse()` 메소드를 전혀 호출하지 않을 것이란 점을 기억하십시오.

모든 경우에, `onResponseComplete()` 메소드를 호출한 다음 클라이언트에 마지막 (또는 유일한) 덩어리 (`chunk`)를 전달할 것입니다.

`False`를 반환하면 클라이언트에 데이터를 플러시하지 않을 것입니다. 플러그인이 응답에서 나오는 모든 덩어리들을 모은 다음 검사하거나 또는 `onResponseComplete()`에서 응답을 변경하고자 하는 경우 유용합니다.

onResponseComplete

핵심 서비스가 (적중한 경우) 처리 과정을 완료하고 클라이언트에 응답을 전달할 준비가 됐을 때 이 콜백을 한 번 호출합니다. 앞에서 설명한 대로, 클라이언트에 마지막 (또는 유일한) 데이터 덩어리를 전달하기 전에 이 메소드를 호출할 것입니다. 스트리밍 서비스의 경우, `onSendResponse()` 메소드를 여러 번 호출할 수도 있습니다.

`onResponseComplete()` 가 새로운 서비스를 (WPS 또는 사용자 정의 서비스를) 구현하고 핵심 서비스에서 나오는 출력을 직접 조작할 수 있는 (예를 들면 WMS 이미지 위에 워터마크를 추가할 수 있는) 이상적인 위치입니다.

`False` 를 반환하면 다음 플러그인들이 `onResponseComplete()` 메소드를 실행하지 않을 것이지만, 어떤 경우여라도 클라이언트에 응답을 전달하지 않을 것이라는 사실을 기억하십시오.

플러그인에서 예외를 발생시키기

이 주제에 대해서는 아직 좀 더 작업을 해야 합니다. 현재 구현된 바로는 `QgsMapServiceException` 클래스의 인스턴스에 `QgsRequestHandler` 속성을 설정해서 처리된 예외와 처리되지 않은 예외를 구별할 수 있습니다. 이런 방식은 주 C++ 코드가 처리된 파이썬 예외를 캐치하고 처리되지 않은 예외는 무시 (또는 더 나은 방법으로는 로그를 작성) 할 수 있게 합니다.

이 접근법은 기본적으로 작동하지만 그다지 “파이썬스럽지” 않습니다. 더 나은 접근법은 파이썬 코드로부터 예외를 발생시킨 다음 처리를 위해 C++ 루프로 버블링 (bubbling) 시키는 것일 겁니다.

서버 플러그인 작성하기

서버 플러그인은 파이썬 플러그인 개발하기 에서 설명한 대로 추가적인 (또는 대체하는) 인터페이스만 제공하는 표준 QGIS 파이썬 플러그인입니다. 일반적인 QGIS 데스크탑 플러그인은 `QgisInterface` 인스턴스를 통해 QGIS 응용 프로그램에 접근할 수 있지만, 서버 플러그인은 QGIS 서버 응용 프로그램 맥락 안에서 실행될 때에만 `QgsServerInterface` 클래스에 접근할 수 있습니다.

QGIS 서버가 플러그인이 서버 인터페이스를 가지고 있다는 사실을 알게 하려면, (`metadata.txt` 파일에) 특별한 메타데이터 항목이 필요합니다:

```
server=True
```

중요

QGIS 서버는 `server=True` 메타데이터를 가진 플러그인들만 불러오고 실행할 것입니다.

여기에서 설명하는 (더 많은 플러그인을 가진) `qgis3-server-vagrant` 예제 플러그인은 깃허브에서 사용할 수 있으며, 몇몇 서버 플러그인은 공식 QGIS 플러그인 저장소 에도 공개되어 있습니다.

플러그인 파일들

다음은 예제 서버 플러그인의 디렉터리 구조입니다.

```
1 PYTHON_PLUGINS_PATH/
2   HelloServer/
3     __init__.py    --> *required*
4     HelloServer.py --> *required*
5     metadata.txt   --> *required*
```

__init__.py

파이썬의 가져오기 시스템이 이 파일을 필요로 합니다. 또 QGIS 서버도 이 파일이 `serverClassFactory()` 함수를 담고 있을 것을 요구합니다. 이 함수는 서버 구동 시 QGIS 서버에 플러그인을 불러왔을 때 호출되어, `Qgis-ServerInterface` 클래스의 인스턴스를 가리키는 참조를 받아서 플러그인 클래스의 인스턴스를 반환해야만 합니다. `__init__.py`의 내용은 다음과 같이 보여야 합니다:

```
def serverClassFactory(serverIface):
    from .HelloServer import HelloServerServer
    return HelloServerServer(serverIface)
```

HelloServer.py

이 파일에서 마법이 벌어지는데, 이 마법은 다음과 같이 보일 것입니다: (`HelloServer.py` 파일의 예시)

서버 플러그인은 일반적으로 `QgsServerFilter` 클래스의 인스턴스에 패키징된 하나 이상의 콜백으로 이루어져 있습니다.

각각의 `QgsServerFilter` 클래스는 다음과 같은 하나 이상의 콜백을 구현합니다:

- `onRequestReady()`
- `onResponseComplete()`
- `onSendResponse()`

다음은 **SERVICE** 파라미터가 "HELLO"와 동등한 경우 *HelloServer!* 를 출력하는 최소한의 필터를 구현하는 예시입니다:

```
1 class HelloFilter(QgsServerFilter):
2
3     def __init__(self, serverIface):
4         super().__init__(serverIface)
5
6     def onRequestReady(self) -> bool:
7         QgsMessageLog.logMessage("HelloFilter.onRequestReady")
8         return True
9
10    def onSendResponse(self) -> bool:
11        QgsMessageLog.logMessage("HelloFilter.onSendResponse")
12        return True
13
14    def onResponseComplete(self) -> bool:
15        QgsMessageLog.logMessage("HelloFilter.onResponseComplete")
16        request = self.serverInterface().requestHandler()
17        params = request.parameterMap()
18        if params.get('SERVICE', '').upper() == 'HELLO':
19            request.clear()
20            request.setResponseHeader('Content-type', 'text/plain')
21            # Note that the content is of type "bytes"
22            request.appendBody(b'HelloServer!')
23        return True
```

이 필터들은 다음 예시에서처럼 반드시 `serverIface`에 등록되어 있어야만 합니다:

```
class HelloServerServer:
    def __init__(self, serverIface):
        serverIface.registerFilter(HelloFilter(serverIface), 100)
```

`registerFilter()` 메소드의 두 번째 파라미터는 같은 이름을 가진 콜백들에 대한 순서를 정의하는 우선 순위를 설정합니다 (우선 순위 값이 낮을수록 먼저 호출됩니다):

플러그인은 이 3 개의 콜백을 사용해서 서버의 입력 그리고/또는 출력을 서로 다른 많은 방식으로 조작할 수 있습니다. 모든 경우에서, 플러그인 인스턴스는 `QgsServerInterface` 클래스를 통해 `QgsRequestHandler` 클래스에 접근합니다. `QgsRequestHandler` 클래스는 (`requestReady()` 함수를 사용해서) 서버의 핵심 처리 과정에 들어가기 전에 또는 (`sendResponse()` 함수를 사용해서) 핵심 서비스가 요청을 처리한 후에 입력 파라미터들을 변경하는 데 쓰일 수 있는 메소드들을 많이 가지고 있습니다.

다음은 흔한 사용 사례 몇 개를 설명하는 예시들입니다:

입력을 수정하기

다음 플러그인은 쿼리 문자열에서 나오는 입력 파라미터들을 변경하는 테스트 예제로, 이 예시에서는 (이미 파싱된) `parameterMap` 에 새 파라미터를 주입합니다. 그러면 (WMS 등등의) 핵심 서비스가 이 파라미터를 인식해서 핵심 서비스 처리 과정 마지막에 해당 파라미터가 그대로 있는지 확인합니다:

```
1 class ParamsFilter(QgsServerFilter):
2
3     def __init__(self, serverIface):
4         super(ParamsFilter, self).__init__(serverIface)
5
6     def onRequestReady(self) -> bool:
7         request = self.serverInterface().requestHandler()
8         params = request.parameterMap( )
9         request.setParameter('TEST_NEW_PARAM', 'ParamsFilter')
10        return True
11
12    def onResponseComplete(self) -> bool:
13        request = self.serverInterface().requestHandler()
14        params = request.parameterMap( )
15        if params.get('TEST_NEW_PARAM') == 'ParamsFilter':
16            QgsMessageLog.logMessage("SUCCESS - ParamsFilter.onResponseComplete")
17        else:
18            QgsMessageLog.logMessage("FAIL - ParamsFilter.onResponseComplete")
19        return True
```

다음은 로그 파일에 보이는 내용을 추출한 것입니다:

```
1 src/core/qgsmessagelog.cpp: 45: (logMessage) [0ms] 2014-12-12T12:39:29 plugin[0]
↳HelloServerServer - loading filter ParamsFilter
2 src/core/qgsmessagelog.cpp: 45: (logMessage) [1ms] 2014-12-12T12:39:29 Server[0]
↳Server plugin HelloServer loaded!
3 src/core/qgsmessagelog.cpp: 45: (logMessage) [0ms] 2014-12-12T12:39:29 Server[0]
↳Server python plugins loaded
4 src/mapserver/qgshttprequesthandler.cpp: 547: (requestStringToParameterMap) [1ms]
↳inserting pair SERVICE // HELLO into the parameter map
5 src/mapserver/qgsserverfilter.cpp: 42: (onRequestReady) [0ms] QgsServerFilter plugin
↳default onRequestReady called
6 src/core/qgsmessagelog.cpp: 45: (logMessage) [0ms] 2014-12-12T12:39:29 plugin[0]
↳SUCCESS - ParamsFilter.onResponseComplete
```

강조된 줄에 있는 “SUCCESS” 문자열이 플러그인이 테스트를 통과했다는 사실을 나타냅니다.

동일한 기술을 사용해서 핵심 서비스 대신 사용자 정의 서비스를 사용할 수 있습니다. 예를 들어 **SERVICE** 파라미터를 다른 무언가로 변경하는 것만으로도 **WFS SERVICE** 요청 또는 다른 모든 핵심 요청을 뛰어넘을 수 있으며, 그러면 핵심 서비스를 뛰어넘을 것입니다. 그 다음 출력에 사용자 정의 결과를 주입해서 클라이언트에 전달할 수 있습니다. (이에 대해서는 다음 부분에서 설명할 것입니다.)

💡 팁

여러분이 정말로 사용자 정의 서비스를 구현하고자 하는 경우, `QgsService` 클래스의 하위 클래스를 생성한 다음 `registerService(service)` 메소드를 호출해서 `registerFilter()` 메소드 상에서 사용자 정의 서비스를 등록하는 방식을 권장합니다.

출력을 수정 또는 대체하기

다음 워터마크 필터 예시는 WMS 출력을 WMS 핵심 서비스가 생성한 WMS 이미지 위에 워터마크 이미지를 추가해서 만든 새 이미지로 대체하는 방법을 보여줍니다:

```

1 from qgis.server import *
2 from qgis.PyQt.QtCore import *
3 from qgis.PyQt.QtGui import *
4
5 class WatermarkFilter(QgsServerFilter):
6
7     def __init__(self, serverIface):
8         super().__init__(serverIface)
9
10    def onResponseComplete(self) -> bool:
11        request = self.serverInterface().requestHandler()
12        params = request.parameterMap()
13        # Do some checks
14        if (params.get('SERVICE').upper() == 'WMS' \
15            and params.get('REQUEST').upper() == 'GETMAP' \
16            and not request.exceptionRaised()):
17            QgsMessageLog.logMessage("WatermarkFilter.onResponseComplete: image ready
↪ %s" % request.parameter("FORMAT"))
18            # Get the image
19            img = QImage()
20            img.loadFromData(request.body())
21            # Adds the watermark
22            watermark = QImage(os.path.join(os.path.dirname(__file__), 'media/
↪ watermark.png'))
23            p = QPainter(img)
24            p.drawImage(QRect( 20, 20, 40, 40), watermark)
25            p.end()
26            ba = QByteArray()
27            buffer = QBuffer(ba)
28            buffer.open(QIODevice.WriteOnly)
29            img.save(buffer, "PNG" if "png" in request.parameter("FORMAT") else "JPG")
30            # Set the body
31            request.clearBody()
32            request.appendBody(ba)
33            return True

```

이 예시에서는 **SERVICE** 파라미터 값을 확인해서 들어오는 요청이 **WMS GETMAP** 이며 이전에 실행된 플러그인이 또는 핵심 서비스가(이 경우 WMS 가) 어떤 예외도 설정하지 않은 경우, 출력 버퍼에서 WMS 가 생성한 이미지를 검색해서 워터마크 이미지를 추가합니다. 마지막 단계는 출력 버퍼를 지운 다음 새로 생성한 이미지로 대체하는 것입니다. 실제 상황에서는 PNG 또는 JPG 만 지원하는 것이 아니라 요청된 이미지 유형도 확인해야 한다는 사실을 기억하십시오.

접근 제어 필터

접근 제어 필터들은 개발자가 어떤 레이어, 필터, 그리고 속성에 접근할 수 있는지에 대해 세밀하게 제어할 수 있게 해줍니다. 접근 제어 필터에 다음 콜백들을 구현할 수 있습니다:

- `layerFilterExpression(layer)`
- `layerFilterSubsetString(layer)`
- `layerPermissions(layer)`
- `authorizedLayerAttributes(layer, attributes)`
- `allowToEdit(layer, feature)`
- `cacheKey()`

플러그인 파일들

다음은 예제 플러그인의 디렉터리 구조입니다.

```
1 PYTHON_PLUGINS_PATH/
2   MyAccessControl/
3     __init__.py    --> *required*
4     AccessControl.py --> *required*
5     metadata.txt   --> *required*
```

`__init__.py`

파이썬의 가져오기 시스템이 이 파일을 필요로 합니다. 모든 QGIS 서버 플러그인과 마찬가지로, 이 파일이 `serverClassFactory()` 함수를 담고 있습니다. 이 함수는 서버 구동 시 QGIS 서버에 플러그인을 불러왔을 때 호출되어, `QgisServerInterface` 클래스의 인스턴스를 가리키는 참조를 받아서 플러그인 클래스의 인스턴스를 반환해야만 합니다. `__init__.py`의 내용은 다음과 같이 보여야 합니다:

```
def serverClassFactory(serverIface):
    from MyAccessControl.AccessControl import AccessControlServer
    return AccessControlServer(serverIface)
```

`AccessControl.py`

```
1 class AccessControlFilter(QgsAccessControlFilter):
2
3     def __init__(self, server_iface):
4         super().__init__(server_iface)
5
6     def layerFilterExpression(self, layer):
7         """ Return an additional expression filter """
8         return super().layerFilterExpression(layer)
9
10    def layerFilterSubsetString(self, layer):
11        """ Return an additional subset string (typically SQL) filter """
12        return super().layerFilterSubsetString(layer)
13
14    def layerPermissions(self, layer):
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

15     """ Return the layer rights """
16     return super().layerPermissions(layer)
17
18     def authorizedLayerAttributes(self, layer, attributes):
19         """ Return the authorised layer attributes """
20         return super().authorizedLayerAttributes(layer, attributes)
21
22     def allowToEdit(self, layer, feature):
23         """ Are we authorised to modify the following geometry """
24         return super().allowToEdit(layer, feature)
25
26     def cacheKey(self):
27         return super().cacheKey()
28
29 class AccessControlServer:
30
31     def __init__(self, serverIface):
32         """ Register AccessControlFilter """
33         serverIface.registerAccessControl(AccessControlFilter(serverIface), 100)

```

이 예시는 모두에게 완전한 접근 권한을 부여합니다.

누가 로그인했는지 확인하는 것이 플러그인의 역할입니다.

이런 모든 메소드에는 제약 조건을 레이어 별로 사용자 정의할 수 있는 레이어 인자가 있습니다.

layerFilterExpression

결과를 제한하기 위한 표현식을 추가하는 데 쓰입니다.

예를 들면 role 속성이 user 인 경우로 피처를 제한할 수 있습니다.

```

def layerFilterExpression(self, layer):
    return "$role = 'user'"

```

layerFilterSubsetString

바로 앞의 함수와 동일하지만 (데이터베이스에서 실행되는) SubsetString 을 사용합니다.

예를 들면 role 속성이 user 인 경우로 피처를 제한할 수 있습니다.

```

def layerFilterSubsetString(self, layer):
    return "role = 'user'"

```

layerPermissions

레이어에의 접근을 제한합니다.

다음 속성을 가진 `LayerPermissions()` 유형의 객체를 반환합니다:

- `canRead`: `GetCapabilities` 에서 피처를 볼 수 있고 읽기 접근 권한을 가집니다.
- `canInsert`: 새 피처를 삽입할 수 있습니다.
- `canUpdate`: 피처를 업데이트할 수 있습니다.
- `canDelete`: 피처를 삭제할 수 있습니다.

예를 들어 읽기 전용 접근에서 모든 권한을 제한하려면:

```
1 def layerPermissions(self, layer):
2     rights = QgsAccessControlFilter.LayerPermissions()
3     rights.canRead = True
4     rights.canInsert = rights.canUpdate = rights.canDelete = False
5     return rights
```

authorizedLayerAttributes

속성의 특정 하위 집합의 가시성을 제한하는 데 쓰입니다.

이 인자 속성은 현재 가시 속성 집합을 반환합니다.

예를 들어 `role` 속성을 숨기려면:

```
def authorizedLayerAttributes(self, layer, attributes):
    return [a for a in attributes if a != "role"]
```

allowToEdit

피처들의 하위 집합에 대한 편집 작업을 제한하는 데 쓰입니다.

이 함수는 WFS-Transaction 프로토콜에 사용됩니다.

예를 들면 `role` 속성이 `user` 인 피처만 편집할 수 있게 하려면:

```
def allowToEdit(self, layer, feature):
    return feature.attribute('role') == 'user'
```

cacheKey

QGIS 서버가 캐시퍼빌리티의 캐시를 유지·관리하는 경우 역할 (`role`) 별로 캐시하려면 이 메소드에서 역할을 반환하면 됩니다. 또는 `None` 을 반환해서 캐시를 완전히 비활성화시킬 수도 있습니다.

20.4.2 사용자 정의 서비스

QGIS 서버에서 WMS, WFS 및 WCS 같은 핵심 서비스들은 `QgsService` 클래스의 하위 클래스로 구현됩니다.

SERVICE 쿼리 문자열 파라미터가 서비스 이름과 일치할 때 실행될 새 서비스를 구현하려면, 여러분 고유의 `QgsService` 클래스를 구현한 다음 `registerService(service)` 메소드를 호출해서 `serviceRegistry()` 메소드 상에서 여러분의 서비스를 등록하면 됩니다.

다음은 CUSTOM 이라는 이름을 가진 사용자 정의 서비스의 예시입니다:

```

1 from qgis.server import QgsService
2 from qgis.core import QgsMessageLog
3
4 class CustomServiceService(QgsService):
5
6     def __init__(self):
7         QgsService.__init__(self)
8
9     def name(self):
10        return "CUSTOM"
11
12    def version(self):
13        return "1.0.0"
14
15    def executeRequest(self, request, response, project):
16        response.setStatusCode(200)
17        QgsMessageLog.logMessage('Custom service executeRequest')
18        response.write("Custom service executeRequest")
19
20
21 class CustomService():
22
23     def __init__(self, serverIface):
24         serverIface.serviceRegistry().registerService(CustomServiceService())

```

20.4.3 사용자 정의 API

QGIS 서버에서, OAPIF (다른 이름으로는 WFS3) 같은 OGC API 들은 `QgsServerOgcApi` 클래스의 인스턴스에 등록된 `QgsServerOgcApiHandler` 하위 클래스들의 집합으로 (또는 그 상위 클래스인 `QgsServerApi` 로) 구현됩니다.

URL 경로가 특정 URL 과 일치할 때 실행될 새 API 를 구현하려면, 여러분 고유의 `QgsServerOgcApiHandler` 클래스 인스턴스들을 구현한 다음 `QgsServerOgcApi` 클래스에 추가하고 `registerApi(api)` 메소드를 호출해서 `serviceRegistry()` 메소드 상에서 API 를 등록하면 됩니다.

다음은 URL 이 /customapi 를 담고 있는 경우 실행될 사용자 정의 API 의 예시입니다:

```

1 import json
2 import os
3
4 from qgis.PyQt.QtCore import QBuffer, QIODevice, QTextStream, QRegularExpression
5 from qgis.server import (
6     QgsServiceRegistry,
7     QgsService,
8     QgsServerFilter,
9     QgsServerOgcApi,
10    QgsServerQueryStringParameter,

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

11     QgsServerOgcApiHandler,
12 )
13
14 from qgis.core import (
15     QgsMessageLog,
16     QgsJsonExporter,
17     QgsCircle,
18     QgsFeature,
19     QgsPoint,
20     QgsGeometry,
21 )
22
23
24 class CustomApiHandler(QgsServerOgcApiHandler):
25
26     def __init__(self):
27         super(CustomApiHandler, self).__init__()
28         self.setContentTypes([QgsServerOgcApi.HTML, QgsServerOgcApi.JSON])
29
30     def path(self):
31         return QRegularExpression("/customapi")
32
33     def operationId(self):
34         return "CustomApiXYCircle"
35
36     def summary(self):
37         return "Creates a circle around a point"
38
39     def description(self):
40         return "Creates a circle around a point"
41
42     def linkTitle(self):
43         return "Custom Api XY Circle"
44
45     def linkType(self):
46         return QgsServerOgcApi.data
47
48     def handleRequest(self, context):
49         """Simple Circle"""
50
51         values = self.values(context)
52         x = values['x']
53         y = values['y']
54         r = values['r']
55         f = QgsFeature()
56         f.setAttributes([x, y, r])
57         f.setGeometry(QgsCircle(QgsPoint(x, y), r).toCircularString())
58         exporter = QgsJsonExporter()
59         self.write(json.loads(exporter.exportFeature(f)), context)
60
61     def templatePath(self, context):
62         # The template path is used to serve HTML content
63         return os.path.join(os.path.dirname(__file__), 'circle.html')
64
65     def parameters(self, context):
66         return [QgsServerQueryStringParameter('x', True,

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

67     ↪QgsServerQueryStringParameter.Type.Double, 'X coordinate'),
        QgsServerQueryStringParameter(
68         'y', True, QgsServerQueryStringParameter.Type.Double, 'Y
↪coordinate'),
69         QgsServerQueryStringParameter('r', True,
↪QgsServerQueryStringParameter.Type.Double, 'radius')]
70
71
72 class CustomApi():
73
74     def __init__(self, serverIface):
75         api = QgsServerOgcApi(serverIface, '/customapi',
76                               'custom api', 'a custom api', '1.1')
77         handler = CustomApiHandler()
78         api.registerHandler(handler)
79         serverIface.serviceRegistry().registerApi(api)

```

PyQGIS 의 꼼수와 편법

 힌트

PyQGIS 콘솔을 사용하지 않는 경우 이 페이지에 있는 코드 조각들을 다음과 같이 가져와야 합니다:

```

1 from qgis.PyQt.QtCore import (
2     QRectF,
3 )
4
5 from qgis.core import (
6     Qgs,
7     QgsProject,
8     QgsLayerTreeModel,
9 )
10
11 from qgis.gui import (
12     QgsLayerTreeView,
13 )

```

21.1 사용자 인터페이스

룩 앤드 필 바꾸기

```

1 from qgis.PyQt.QtWidgets import QApplication
2
3 app = QApplication.instance()
4 app.setStyleSheet(".QWidget {color: blue; background-color: yellow;}")
5 # You can even read the stylesheet from a file
6 with open("testdata/file.qss") as qss_file_content:
7     app.setStyleSheet(qss_file_content.read())

```

아이콘과 제목 바꾸기

```
1 from qgis.PyQt.QtGui import QIcon
2
3 icon = QIcon("/path/to/logo/file.png")
4 iface.mainWindow().setWindowIcon(icon)
5 iface.mainWindow().setWindowTitle("My QGIS")
```

21.2 설정

QgsSettings 목록 얻기

```
1 from qgis.core import QgsSettings
2
3 qs = QgsSettings()
4
5 for k in sorted(qs.allKeys()):
6     print(k)
```

21.3 툴바

툴바 제거하기

```
1 toolbar = iface.helpToolBar()
2 parent = toolbar.parentWidget()
3 parent.removeToolBar(toolbar)
4
5 # and add again
6 parent.addToolBar(toolbar)
```

액션 툴바 제거하기

```
actions = iface.attributesToolBar().actions()
iface.attributesToolBar().clear()
iface.attributesToolBar().addAction(actions[4])
iface.attributesToolBar().addAction(actions[3])
```

21.4 메뉴 그룹

메뉴 제거하기

```
1 # for example Help Menu
2 menu = iface.helpMenu()
3 menubar = menu.parentWidget()
4 menubar.removeAction(menu.menuAction())
5
6 # and add again
7 menubar.addAction(menu.menuAction())
```


21.5 캔버스

캔버스에 접근하기

```
canvas = iface.mapCanvas()
```

캔버스 색상 바꾸기

```
from qgis.PyQt.QtCore import Qt

iface.mapCanvas().setCanvasColor(Qt.black)
iface.mapCanvas().refresh()
```

맵 업데이트 주기

```
from qgis.core import QgsSettings
# Set milliseconds (150 milliseconds)
QgsSettings().setValue("/qgis/map_update_interval", 150)
```

21.6 레이어

벡터 레이어 추가하기

```
layer = iface.addVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports_↵
↵layer", "ogr")
if not layer or not layer.isValid():
    print("Layer failed to load!")
```

활성 레이어 얻기

```
layer = iface.activeLayer()
```

모든 레이어를 목록화하기

```
from qgis.core import QgsProject

QgsProject.instance().mapLayers().values()
```

레이어 이름 얻기

```
1 from qgis.core import QgsVectorLayer
2 layer = QgsVectorLayer("Point?crs=EPSG:4326", "layer name you like", "memory")
3 QgsProject.instance().addMapLayer(layer)
4
5 layers_names = []
6 for layer in QgsProject.instance().mapLayers().values():
7     layers_names.append(layer.name())
8
9 print("layers TOC = {}".format(layers_names))
```

```
layers TOC = ['layer name you like']
```

다른 방법

```
layers_names = [layer.name() for layer in QgsProject.instance().mapLayers().values()]
print("layers TOC = {}".format(layers_names))
```

```
layers TOC = ['layer name you like']
```

Obtain layers type

```
for layer in QgsProject.instance().mapLayers().values():
    print(f"{layer.name()} of type {layer.type().name}")
```

```
layer name you like of type Vector
```

이름으로 레이어 찾기

```
from qgis.core import QgsProject

layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
print(layer.name())
```

```
layer name you like
```

활성 레이어 설정하기

```
from qgis.core import QgsProject

layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
iface.setActiveLayer(layer)
```

레이어를 주기적으로 새로고침하기

```
1 from qgis.core import QgsProject
2
3 layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
4 # Set seconds (5 seconds)
5 layer.setAutoRefreshInterval(5000)
6 # Enable data reloading
7 layer.setAutoRefreshMode(Qgis.AutoRefreshMode.ReloadData)
```

메소드 보이기

```
dir(layer)
```

피쳐 양식으로 새 피쳐 추가하기

```
1 from qgis.core import QgsFeature, QgsGeometry
2
3 feat = QgsFeature()
4 geom = QgsGeometry()
5 feat.setGeometry(geom)
6 feat.setFields(layer.fields())
7
8 iface.openFeatureForm(layer, feat, False)
```

피쳐 양식 없이 새 피쳐 추가하기

```

1 from qgis.core import QgsGeometry, QgsPointXY, QgsFeature
2
3 pr = layer.dataProvider()
4 feat = QgsFeature()
5 feat.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(10,10)))
6 pr.addFeatures([feat])

```

피처 얻기

```

for f in layer.getFeatures():
    print(f)

```

```
<qgis._core.QgsFeature object at 0x7f45cc64b678>
```

선택한 피처 얻기

```

for f in layer.selectedFeatures():
    print(f)

```

선택한 피처 ID 얻기

```

selected_ids = layer.selectedFeatureIds()
print(selected_ids)

```

선택한 피처 ID로부터 메모리 레이어 생성하기

```

from qgis.core import QgsFeatureRequest

memory_layer = layer.materialize(QgsFeatureRequest().setFilterFids(layer.
    ↳selectedFeatureIds()))
QgsProject.instance().addMapLayer(memory_layer)

```

도형 얻기

```

# Point layer
for f in layer.getFeatures():
    geom = f.geometry()
    print('%f, %f' % (geom.asPoint().y(), geom.asPoint().x()))

```

```
10.000000, 10.000000
```

도형 이동시키기

```

1 from qgis.core import QgsFeature, QgsGeometry
2 poly = QgsFeature()
3 geom = QgsGeometry.fromWkt("POINT(7 45)")
4 geom.translate(1, 1)
5 poly.setGeometry(geom)
6 print(poly.geometry())

```

```
<QgsGeometry: Point (8 46)>
```

좌표계 설정하기

```
from qgis.core import QgsProject, QgsCoordinateReferenceSystem
```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```
for layer in QgsProject.instance().mapLayers().values():
    layer.setCrs(QgsCoordinateReferenceSystem('EPSG:4326'))
```

좌표계 보기

```
1 from qgis.core import QgsProject
2
3 for layer in QgsProject.instance().mapLayers().values():
4     crs = layer.crs().authid()
5     layer.setName('{} {}'.format(layer.name(), crs))
```

필드 열 숨기기

```
1 from qgis.core import QgsEditorWidgetSetup
2
3 def fieldVisibility (layer, fname):
4     setup = QgsEditorWidgetSetup('Hidden', {})
5     for i, column in enumerate(layer.fields()):
6         if column.name() == fname:
7             layer.setEditorWidgetSetup(idx, setup)
8             break
9     else:
10        continue
```

WKT 에서 나온 레이어

```
1 from qgis.core import QgsVectorLayer, QgsFeature, QgsGeometry, QgsProject
2
3 layer = QgsVectorLayer('Polygon?crs=epsg:4326', 'Mississippi', 'memory')
4 pr = layer.dataProvider()
5 poly = QgsFeature()
6 geom = QgsGeometry.fromWkt("POLYGON ((-88.82 34.99,-88.09 34.89,-88.39 30.34,-89.57
↳30.18,-89.73 31,-91.63 30.99,-90.87 32.37,-91.23 33.44,-90.93 34.23,-90.30 34.99,-
↳88.82 34.99))")
7 poly.setGeometry(geom)
8 pr.addFeatures([poly])
9 layer.updateExtents()
10 QgsProject.instance().addMapLayers([layer])
```

GeoPackage 에서 벡터 레이어를 모두 불러오기

```
1 from qgis.core import QgsDataProvider
2
3 fileName = "testdata/sublayers.gpkg"
4 layer = QgsVectorLayer(fileName, "test", "ogr")
5 subLayers = layer.dataProvider().subLayers()
6
7 for subLayer in subLayers:
8     name = subLayer.split(QgsDataProvider.SUBLAYER_SEPARATOR)[1]
9     uri = "%s|layername=%s" % (fileName, name,)
10    # Create layer
11    sub_vlayer = QgsVectorLayer(uri, name, 'ogr')
12    # Add layer to map
13    QgsProject.instance().addMapLayer(sub_vlayer)
```

타일 레이어 (XYZ 레이어) 불러오기

```

1 from qgis.core import QgsRasterLayer, QgsProject
2
3 def loadXYZ(url, name):
4     rasterLyr = QgsRasterLayer("type=xyz&url=" + url, name, "wms")
5     QgsProject.instance().addMapLayer(rasterLyr)
6
7 urlWithParams = 'https://tile.openstreetmap.org/%7Bz%7D/%7Bx%7D/%7By%7D.png&zmax=19&
8 ↪zmin=0&crs=EPSG3857'
9 loadXYZ(urlWithParams, 'OpenStreetMap')

```

레이어를 모두 제거하기

```
QgsProject.instance().removeAllMapLayers()
```

모두 제거하기

```
QgsProject.instance().clear()
```

21.7 TOC(Table of Contents)

체크한 레이어에 접근하기

```
iface.mapCanvas().layers()
```

컨텍스트 메뉴 제거하기

```

1 ltv = iface.layerTreeView()
2 mp = ltv.menuProvider()
3 ltv.setMenuProvider(None)
4 # Restore
5 ltv.setMenuProvider(mp)

```

21.8 고급 TOC

루트 노드

```

1 from qgis.core import QgsVectorLayer, QgsProject, QgsLayerTreeLayer
2
3 root = QgsProject.instance().layerTreeRoot()
4 node_group = root.addGroup("My Group")
5
6 layer = QgsVectorLayer("Point?crs=EPSG:4326", "layer name you like", "memory")
7 QgsProject.instance().addMapLayer(layer, False)
8
9 node_group.addLayer(layer)
10
11 print(root)
12 print(root.children())

```

첫 번째 하위 노드에 접근하기

```

1 from qgis.core import QgsLayerTreeGroup, QgsLayerTreeLayer, QgsLayerTree
2
3 child0 = root.children()[0]
4 print (child0.name())
5 print (type(child0))
6 print (isinstance(child0, QgsLayerTreeLayer))
7 print (isinstance(child0.parent(), QgsLayerTree))

```

```

My Group
<class 'qgis._core.QgsLayerTreeGroup'>
False
True

```

그룹과 노드 찾기

```

1 from qgis.core import QgsLayerTreeGroup, QgsLayerTreeLayer
2
3 def get_group_layers(group):
4     print('- group: ' + group.name())
5     for child in group.children():
6         if isinstance(child, QgsLayerTreeGroup):
7             # Recursive call to get nested groups
8             get_group_layers(child)
9         else:
10            print(' - layer: ' + child.name())
11
12
13 root = QgsProject.instance().layerTreeRoot()
14 for child in root.children():
15     if isinstance(child, QgsLayerTreeGroup):
16         get_group_layers(child)
17     elif isinstance(child, QgsLayerTreeLayer):
18         print ('- layer: ' + child.name())

```

```

- group: My Group
- layer: layer name you like

```

이름으로 그룹 찾기

```
print (root.findGroup("My Group"))
```

```
<QgsLayerTreeGroup: My Group>
```

ID 로 레이어 찾기

```
print(root.findLayer(layer.id()))
```

```
<QgsLayerTreeLayer: layer name you like>
```

레이어 추가하기

```

1 from qgis.core import QgsVectorLayer, QgsProject
2
3 layer1 = QgsVectorLayer("Point?crs=EPSG:4326", "layer name you like 2", "memory")
4 QgsProject.instance().addMapLayer(layer1, False)
5 node_layer1 = root.addLayer(layer1)

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

6 # Remove it
7 QgsProject.instance().removeMapLayer(layer1)

```

그룹 추가하기

```

1 from qgis.core import QgsLayerTreeGroup
2
3 node_group2 = QgsLayerTreeGroup("Group 2")
4 root.addChildNode(node_group2)
5 QgsProject.instance().mapLayersByName("layer name you like")[0]

```

불러온 레이어 이동시키기

```

1 layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
2 root = QgsProject.instance().layerTreeRoot()
3
4 myLayer = root.findLayer(layer.id())
5 myClone = myLayer.clone()
6 parent = myLayer.parent()
7
8 myGroup = root.findGroup("My Group")
9 # Insert in first position
10 myGroup.insertChildNode(0, myClone)
11
12 parent.removeChildNode(myLayer)

```

불러온 레이어를 특정 그룹으로 이동시키기

```

1 QgsProject.instance().addMapLayer(layer, False)
2
3 root = QgsProject.instance().layerTreeRoot()
4 myGroup = root.findGroup("My Group")
5 myOriginalLayer = root.findLayer(layer.id())
6 myLayer = myOriginalLayer.clone()
7 myGroup.insertChildNode(0, myLayer)
8 parent.removeChildNode(myOriginalLayer)

```

활성 레이어의 가시성을 켜고끄기

```

root = QgsProject.instance().layerTreeRoot()
node = root.findLayer(layer.id())
new_state = Qt.Checked if node.isVisible() == Qt.Unchecked else Qt.Unchecked
node.setItemVisibilityChecked(new_state)

```

그룹이 선택되었는지 여부

```

1 def isMyGroupSelected( groupName ):
2     myGroup = QgsProject.instance().layerTreeRoot().findGroup( groupName )
3     return myGroup in iface.layerTreeView().selectedNodes()
4
5 print(isMyGroupSelected( 'my group name' ))

```

```
False
```

노드 펼치기

```
print(myGroup.isExpanded())
myGroup.setExpanded(False)
```

숨긴 노드 켜기

```
1 from qgis.core import QgsProject
2
3 model = iface.layerTreeView().layerTreeModel()
4 ltv = iface.layerTreeView()
5 root = QgsProject.instance().layerTreeRoot()
6
7 layer = QgsProject.instance().mapLayersByName('layer name you like')[0]
8 node = root.findLayer(layer.id())
9
10 index = model.node2index( node )
11 ltv.setRowHidden( index.row(), index.parent(), True )
12 node.setCustomProperty( 'nodeHidden', 'true' )
13 ltv.setCurrentIndex(model.node2index(root))
```

노드 신호

```
1 def onWillAddChildren(node, indexFrom, indexTo):
2     print ("WILL ADD", node, indexFrom, indexTo)
3
4 def onAddedChildren(node, indexFrom, indexTo):
5     print ("ADDED", node, indexFrom, indexTo)
6
7 root.willAddChildren.connect(onWillAddChildren)
8 root.addedChildren.connect(onAddedChildren)
```

레이어 제거하기

```
root.removeLayer(layer)
```

그룹 제거하기

```
root.removeChildNode(node_group2)
```

새 TOC 생성하기

```
1 root = QgsProject.instance().layerTreeRoot()
2 model = QgsLayerTreeModel(root)
3 view = QgsLayerTreeView()
4 view.setModel(model)
5 view.show()
```

노드 이동시키기

```
cloned_group1 = node_group.clone()
root.insertChildNode(0, cloned_group1)
root.removeChildNode(node_group)
```

노드 이름 바꾸기

```
cloned_group1.setName("Group X")
node_layer1.setName("Layer X")
```


21.9 공간 처리 알고리즘

알고리즘 목록 얻기

```

1 from qgis.core import QgsApplication
2
3 for alg in QgsApplication.processingRegistry().algorithms():
4     if 'buffer' == alg.name():
5         print("{}: {} --> {}".format(alg.provider().name(), alg.name(), alg.
    ↳ displayName()))

```

```
QGIS (native c++):buffer --> Buffer
```

알고리즘 도움말 얻기

랜덤하게 선택하기

```

from qgis import processing
processing.algorithmHelp("native:buffer")

```

```
...
```

알고리즘 실행하기

이 예시의 경우, 프로젝트에 추가된 임시 메모리 레이어에 결과를 저장합니다.

```

from qgis import processing
result = processing.run("native:buffer", {'INPUT': layer, 'OUTPUT': 'memory:'})
QgsProject.instance().addMapLayer(result['OUTPUT'])

```

```
Processing(0): Results: {'OUTPUT': 'output_d27a2008_970c_4687_b025_f057abbd7319'}
```

알고리즘이 몇 개나 있는가?

```
len(QgsApplication.processingRegistry().algorithms())
```

제공자가 몇 개나 있는가?

```

from qgis.core import QgsApplication

len(QgsApplication.processingRegistry().providers())

```

표현식이 몇 개나 있는가?

```

from qgis.core import QgsExpression

len(QgsExpression.Functions())

```

21.10 장식자 (decorator)

저작권 (Copyright)

```

1  from qgis.PyQt.Qt import QTextDocument
2  from qgis.PyQt.QtGui import QFont
3
4  mQFont = "Sans Serif"
5  mQFontsize = 9
6  mLabelQString = "© QGIS 2019"
7  mMarginHorizontal = 0
8  mMarginVertical = 0
9  mLabelQColor = "#FF0000"
10
11  INCHES_TO_MM = 0.0393700787402 # 1 millimeter = 0.0393700787402 inches
12  case = 2
13
14  def add_copyright(p, text, xOffset, yOffset):
15      p.translate( xOffset , yOffset )
16      text.drawContents(p)
17      p.setWorldTransform( p.worldTransform() )
18
19  def _on_render_complete(p):
20      deviceHeight = p.device().height() # Get paint device height on which this
21      ↪ painter is currently painting
22      deviceWidth = p.device().width() # Get paint device width on which this painter
23      ↪ is currently painting
24      # Create new container for structured rich text
25      text = QTextDocument()
26      font = QFont()
27      font.setFamily(mQFont)
28      font.setPointSize(int(mQFontsize))
29      text.setDefaultFont(font)
30      style = "<style type=\"text/css\"> p {color: " + mLabelQColor + "}</style>"
31      text.setHtml( style + "<p>" + mLabelQString + "</p>" )
32      # Text Size
33      size = text.size()
34
35      # RenderMillimeters
36      pixelsInchX = p.device().logicalDpiX()
37      pixelsInchY = p.device().logicalDpiY()
38      xOffset = pixelsInchX * INCHES_TO_MM * int(mMarginHorizontal)
39      yOffset = pixelsInchY * INCHES_TO_MM * int(mMarginVertical)
40
41      # Calculate positions
42      if case == 0:
43          # Top Left
44          add_copyright(p, text, xOffset, yOffset)
45
46      elif case == 1:
47          # Bottom Left
48          yOffset = deviceHeight - yOffset - size.height()
49          add_copyright(p, text, xOffset, yOffset)
50
51      elif case == 2:
52          # Top Right
53          xOffset = deviceWidth - xOffset - size.width()

```

(다음 페이지에 계속)

(이전 페이지에서 계속)

```

52     add_copyright(p, text, xOffset, yOffset)
53
54     elif case == 3:
55         # Bottom Right
56         yOffset = deviceHeight - yOffset - size.height()
57         xOffset = deviceWidth - xOffset - size.width()
58         add_copyright(p, text, xOffset, yOffset)
59
60     elif case == 4:
61         # Top Center
62         xOffset = deviceWidth / 2
63         add_copyright(p, text, xOffset, yOffset)
64
65     else:
66         # Bottom Center
67         yOffset = deviceHeight - yOffset - size.height()
68         xOffset = deviceWidth / 2
69         add_copyright(p, text, xOffset, yOffset)
70
71     # Emitted when the canvas has rendered
72     iface.mapCanvas().renderComplete.connect(_on_render_complete)
73     # Repaint the canvas map
74     iface.mapCanvas().refresh()

```

21.11 작성자 (composer)

이름으로 조판 출력하기

```

1 composerTitle = 'MyComposer' # Name of the composer
2
3 project = QgsProject.instance()
4 projectLayoutManager = project.layoutManager()
5 layout = projectLayoutManager.layoutByName(composerTitle)

```

21.12 소스

- QGIS Python (PyQGIS) API
- QGIS C++ API
- 스택 오버플로의 QGIS 질문들
- 클라스 칼손 (Klas Karlsson) 의 스크립트